

Rechnerstrukturen Teil II

Microarchitektur

Dr.-Ing. Christian Hakert / Dr. rer. nat. Mario Günzel

TU Dortmund, Fakultät für Informatik, Lehrstuhl 12

Kontext

Die Wissenschaft Informatik befasst sich mit der Darstellung, Speicherung, Übertragung und **Verarbeitung** von Informationen

[Gesellschaft für Informatik]

Externe Architektur → interne Architektur

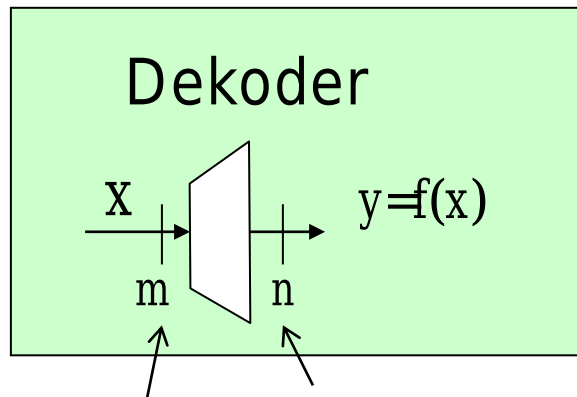
Bislang: Sicht des Programmierers auf den Prozessor:

- Befehlsschnittstelle
- **externe Architektur**

Jetzt: Interne Realisierung:

- **interne Architektur**
- Mikroarchitektur
- Realisierung mit Register-Transfer-Strukturen
(der Inhalt welchen Registers wird in welches andere Register transferiert?)

Komponenten von RT-Strukturen: Dekoder



Anzahl Bits der Bitvektoren

Beispiel 1: 1 aus n-Dekoder

$$f(x) = "00... \overset{y_i}{1} ... 00" \text{ wenn } \text{nat}(x) = i$$

z.B.

$$f(x) = \begin{cases} "0...0001" & \text{wenn } x = "0...00" \\ "0...0010" & \text{wenn } x = "0...01" \\ "0...0100" & \text{wenn } x = "0...10" \\ "0...1000" & \text{wenn } x = "0...11" \\ \dots & \dots \end{cases}$$

Beispiel 2: Prioritätsencoder ($m > n$)

$$\text{nat}(f(x)) = \begin{cases} i, & \text{wenn } i \text{ der größte Index ist, für den } x_i = '1' \text{ ist} \\ \text{undefiniert} & \text{für } x = "0...00" \end{cases}$$

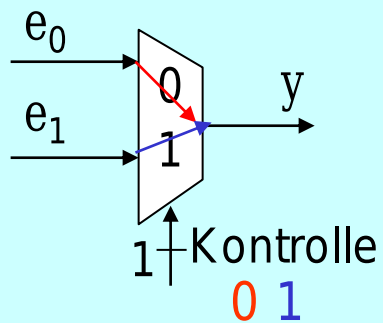
z.B.: $y = "101"$, $\text{nat}(y) = 5$ für $x = "00\underline{1}01011"$

5 0

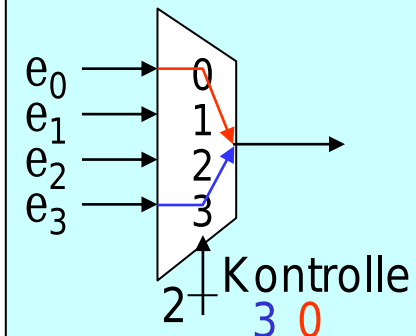
$\text{nat}(a) = \sum a_i 2^i$: (Abbildung Bitvektor $\rightarrow$ natürliche Zahl)

Komponenten von RT-Strukturen: Multiplexer

z.B. 2 auf 1 Multiplexer



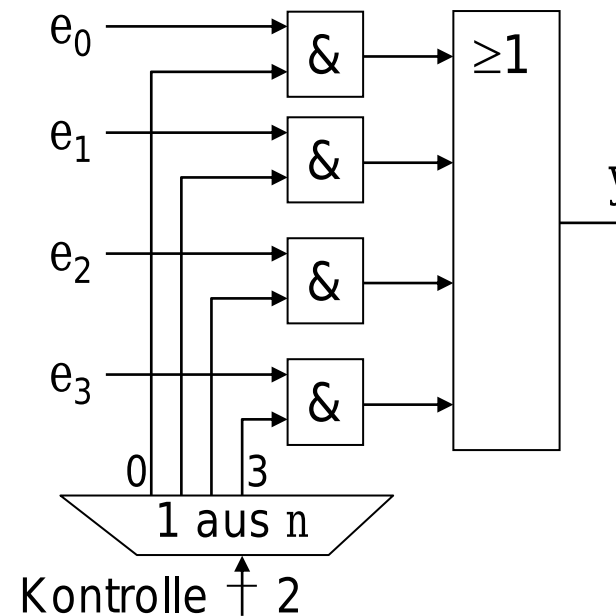
z.B. 4 auf 1 Multiplexer



$$y = e_i \text{ für } \text{nat}(\text{Kontrolle}) = i$$

e_i, y : einzelne Bits oder Bitvektoren

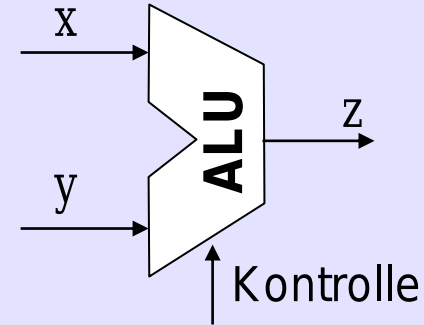
Mögliche Realisierung (für einzelnes Bit)



Komponenten von RT-Strukturen: ALUs

Addierer, arithmetisch/
logische Einheiten (ALUs)

$$z = f(x, y, \text{Kontrolle})$$



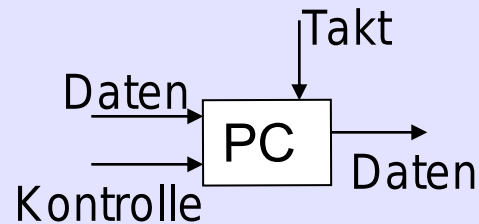
Beispiel

$$z = \begin{cases} x \wedge y & \text{wenn } nat(\text{Kontrolle}) = 0 \\ x \vee y & \text{wenn } nat(\text{Kontrolle}) = 1 \\ x + y & \text{wenn } nat(\text{Kontrolle}) = 2 \\ x - y & \text{wenn } nat(\text{Kontrolle}) = 6 \end{cases}$$

x, y, z, Kontrolle:
Bitvektoren

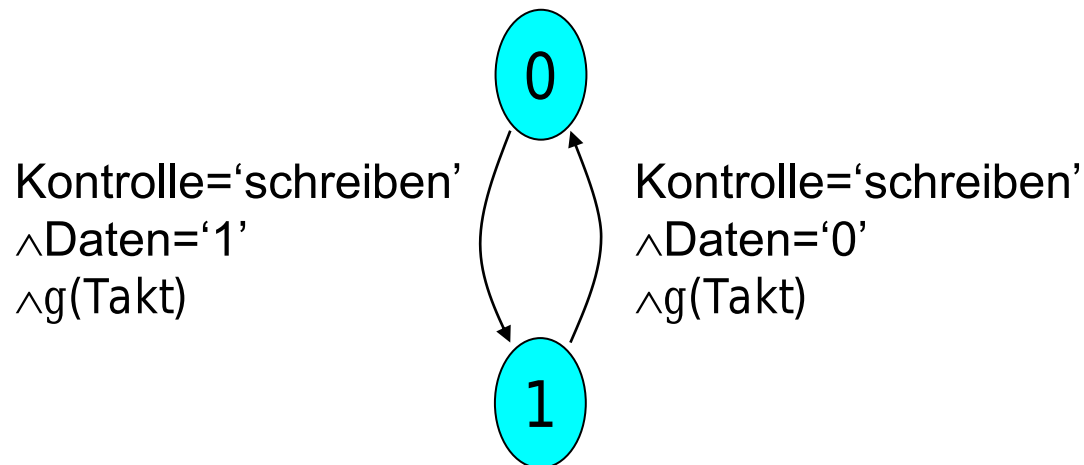
Komponenten von RT-Strukturen: Register

Register



Übernimmt mit dem Takt die Daten, sofern der Kontrolleingang auf "schreiben" gesetzt ist.
Daten: einzelne Bits oder Bitvektoren

Für jedes einzelne Bit:

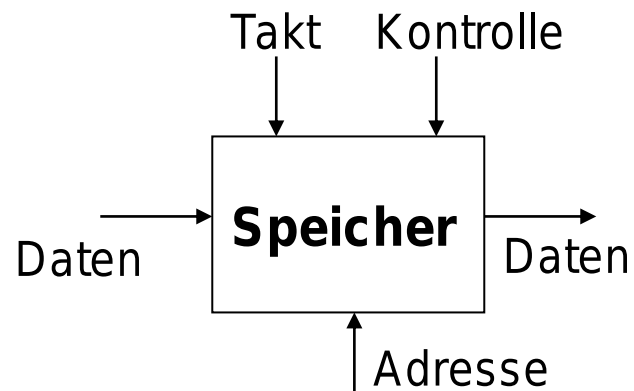


g: modelliert unterschiedliche Taktabhängigkeit

- Flankenabhängigkeit: Wechsel nur bei pos. ("positiv flankengetriggert") oder neg. Flanke
- oder Pegelabhängigkeit: Wechsel sofern Takt='1' oder Takt='0' ist

Komponenten von RT-Strukturen: Speicher

Speicher

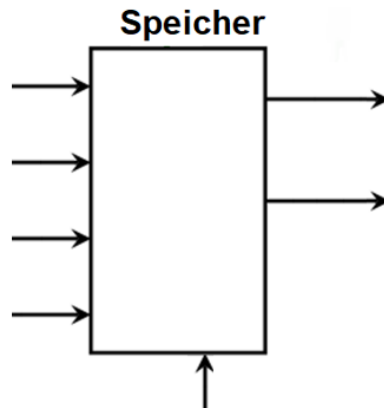


Liest ständig die am Adresseingang ausgewählte Speicherzelle aus und stellt ihren Wert am Datenausgang mit gewisser Verzögerung zur Verfügung. Übernimmt mit dem Takt die Daten in die ausgewählte Speicherzelle, sofern der Kontrolleingang auf „Schreiben“ gesetzt ist.

Wir stellen den 1-aus-n-Adressdekoder immer explizit dar, um den Adresseingang zu identifizieren.

Multiportspeicher

Multiport-Speicher



Besitzt mehrere Adresseingänge, die zu einem zugeordneten **Port** gehören.

Jeder Leseport stellt ständig die gelesene Speicherzelle am Ausgang bereit;
jeder Schreibport übernimmt die Eingangsdaten in die ausgewählte Speicherzelle, sofern der Kontrolleingang auf "schreiben" gestellt ist.
Konflikt, falls mehrere Schreibports dieselbe Zelle auswählen (soll vermieden werden; falls es doch vorkommt: z.B. UND-Verknüpfung der Eingangsdaten)

Zusammenfassung

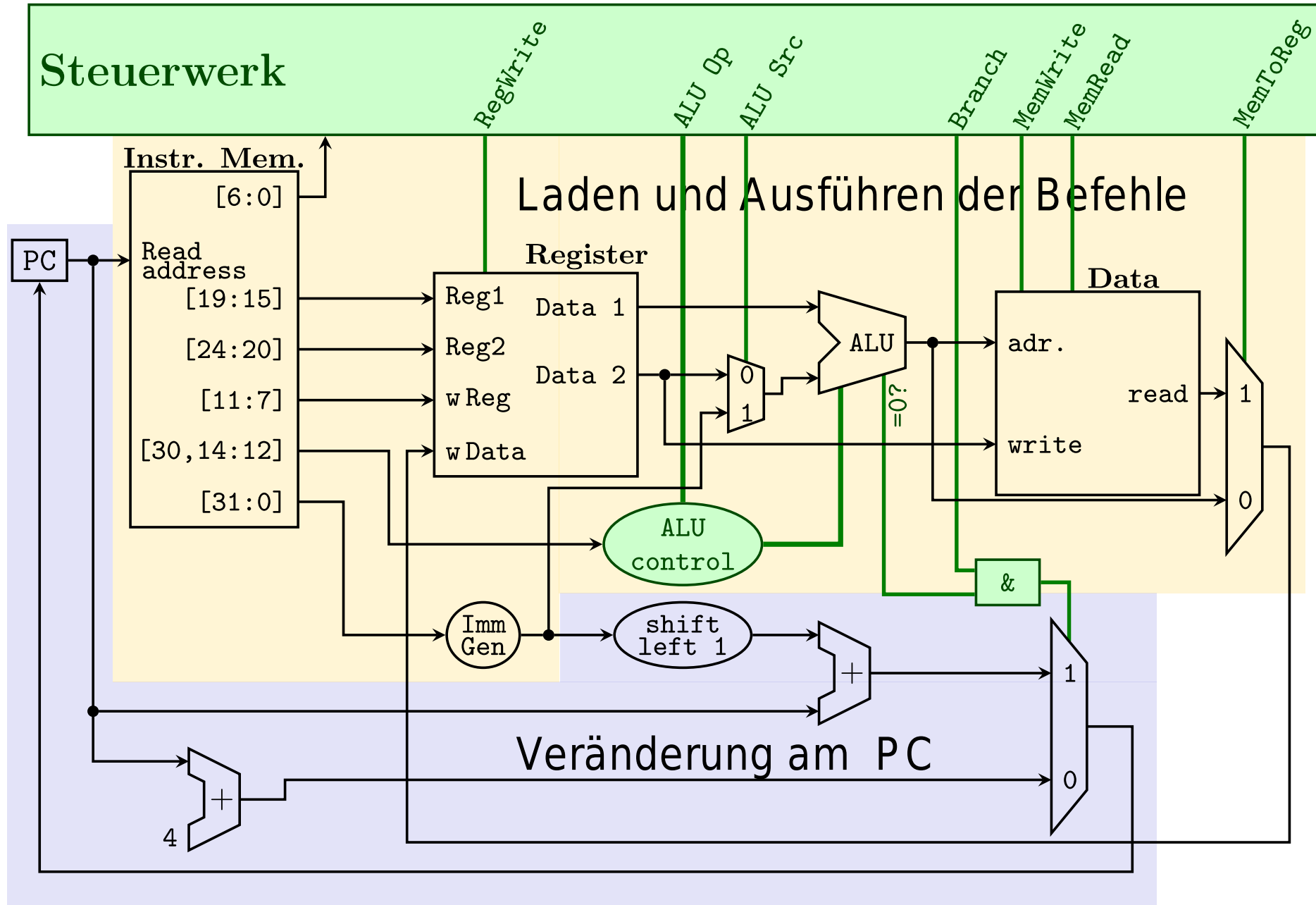
Jetzt Sicht auf die interne Architektur:

- Komponenten von RT-Strukturen
 - Dekoder
 - Multiplexer
 - ALUs
 - Register
 - Speicher

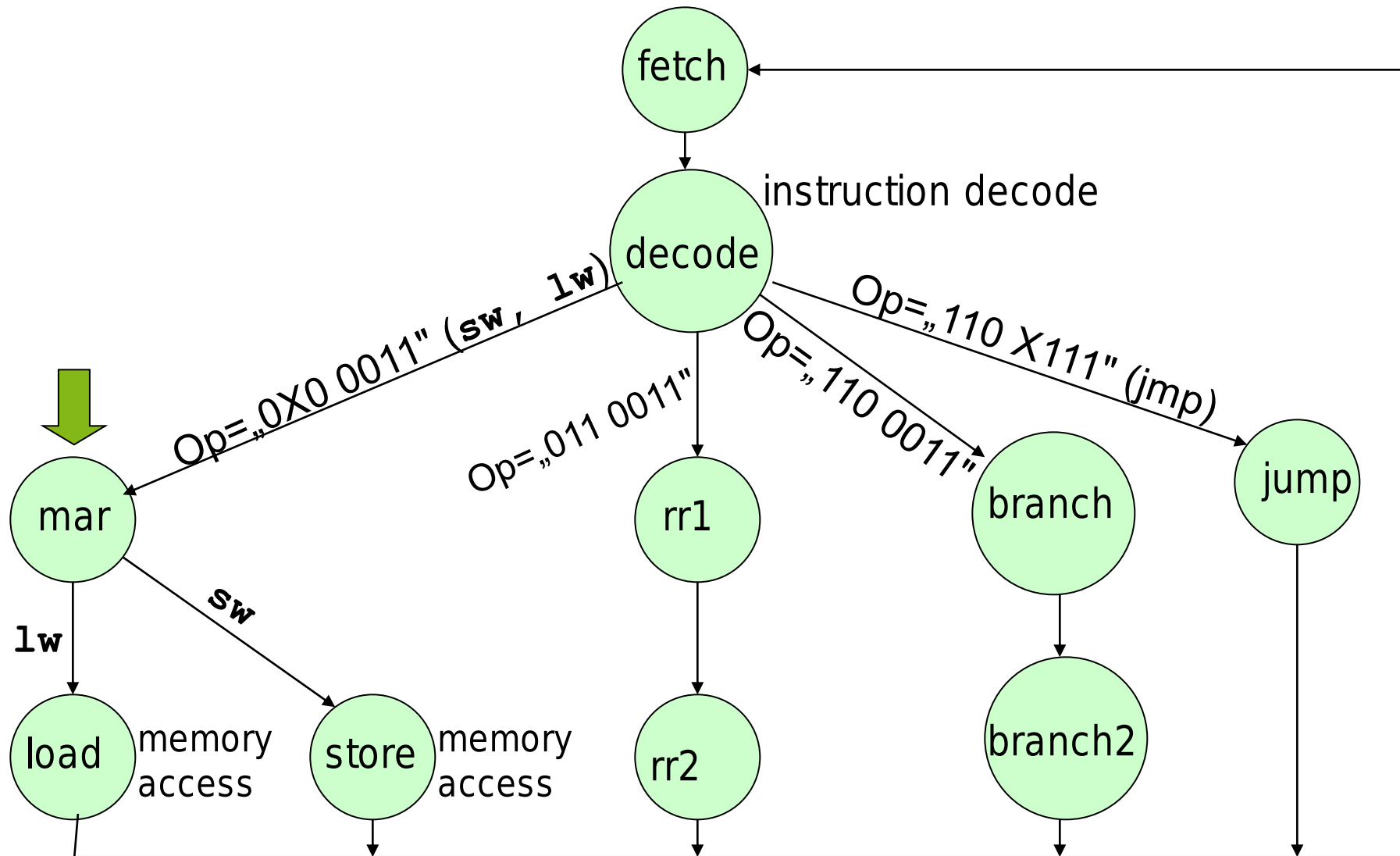
Eine Mikroarchitektur für die RISC-V-Maschine

- Automatische Erzeugung von Mikroarchitekturen aus ISA heraus (μ Architektursynthese): wurde untersucht, ist aber
 - a. im Rahmen einer Erstsemestervorlesung zu kompliziert
 - b. für unsere einfachen Befehle auch nicht nötig.
 - Mit Erfahrung (Betrachtung wesentlicher Komponenten/Verbindungen):
 - manueller μ Architekturentwurf.
 - Benötigt Minimum an Komponenten.
 - Enthält ein Rechenwerk und ein Steuerwerk.
 - Verifikation: $\forall$ Befehle: nachweisen, dass die μ Architektur den Befehl ausführen kann.
- Vorgehen für die RISC-V-Maschine

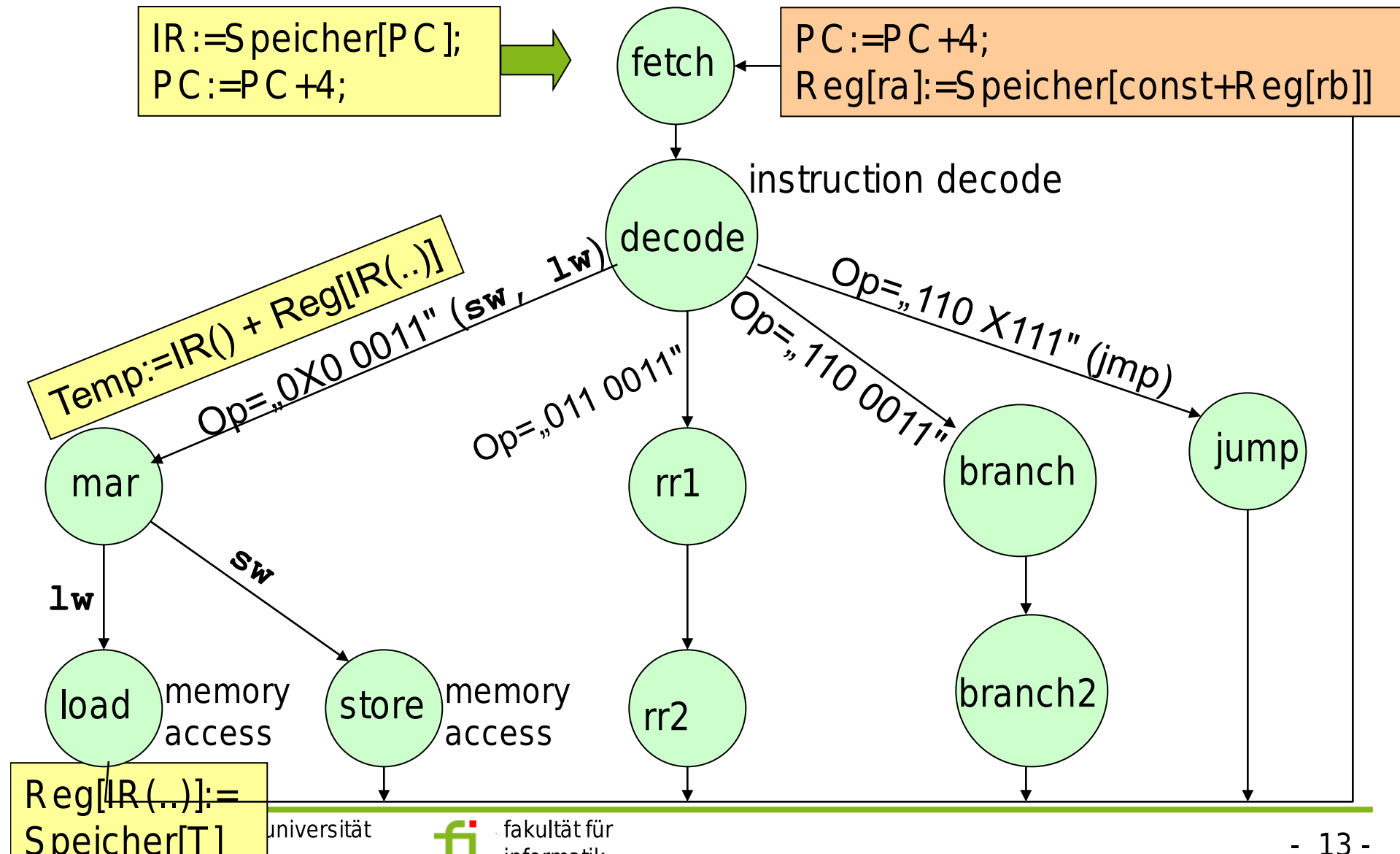
Steuerwerk



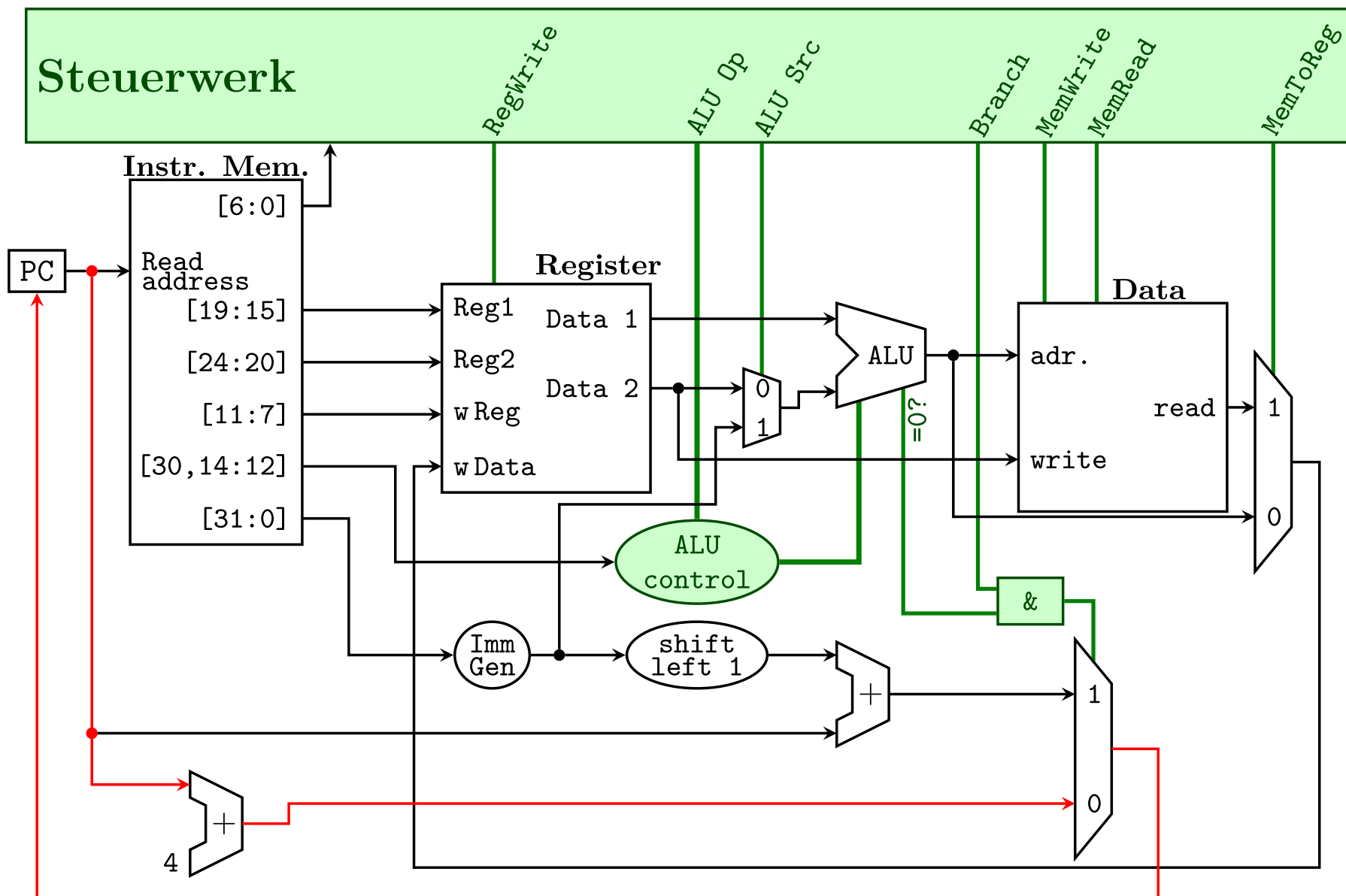
Zustandsgraph der Ausführung einiger RISC-V-Befehle



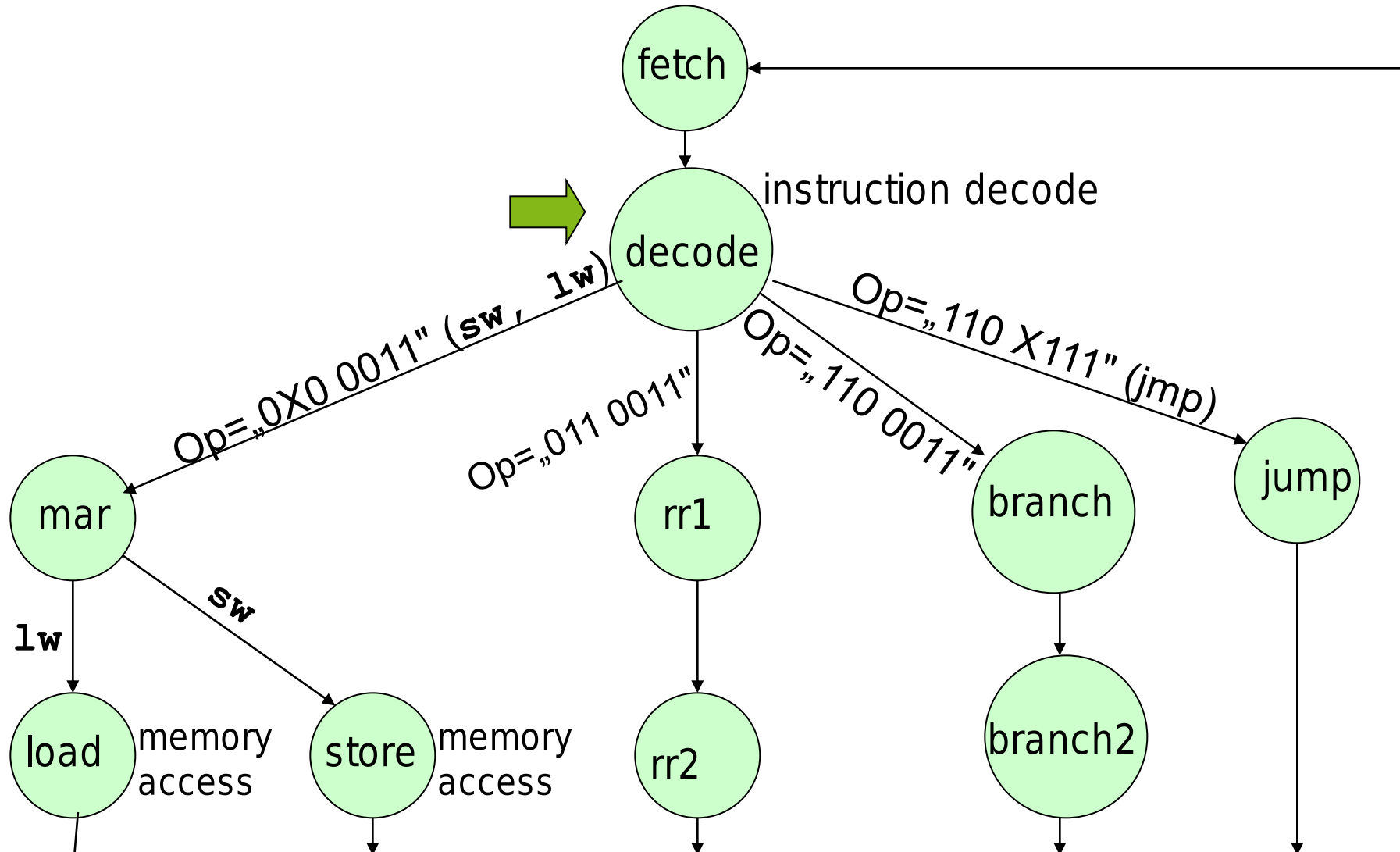
Überprüfung der Ausführbarkeit ➡ Zustandsgraph der Ausführung einiger RISC-V-Befehle



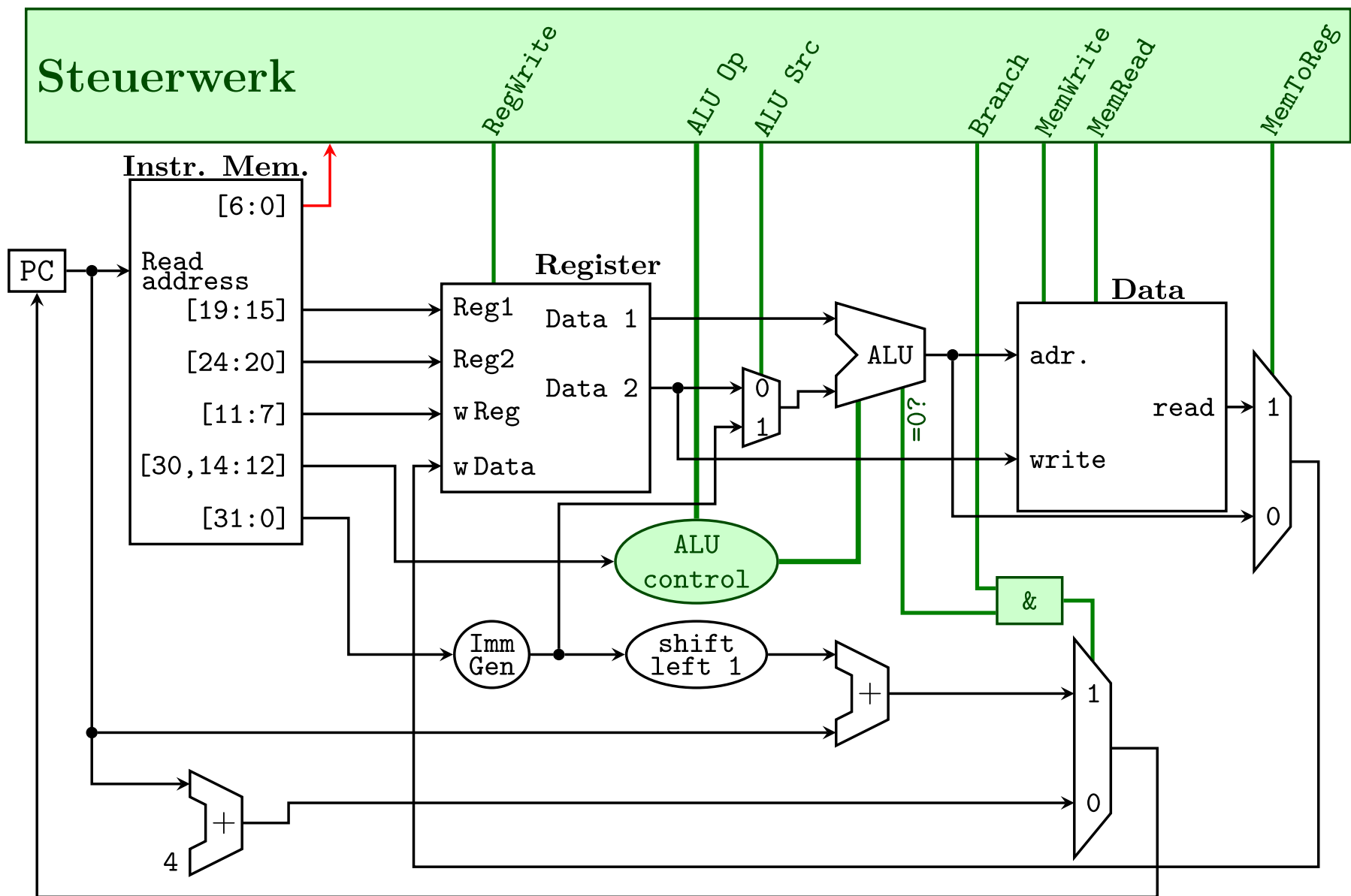
fetch



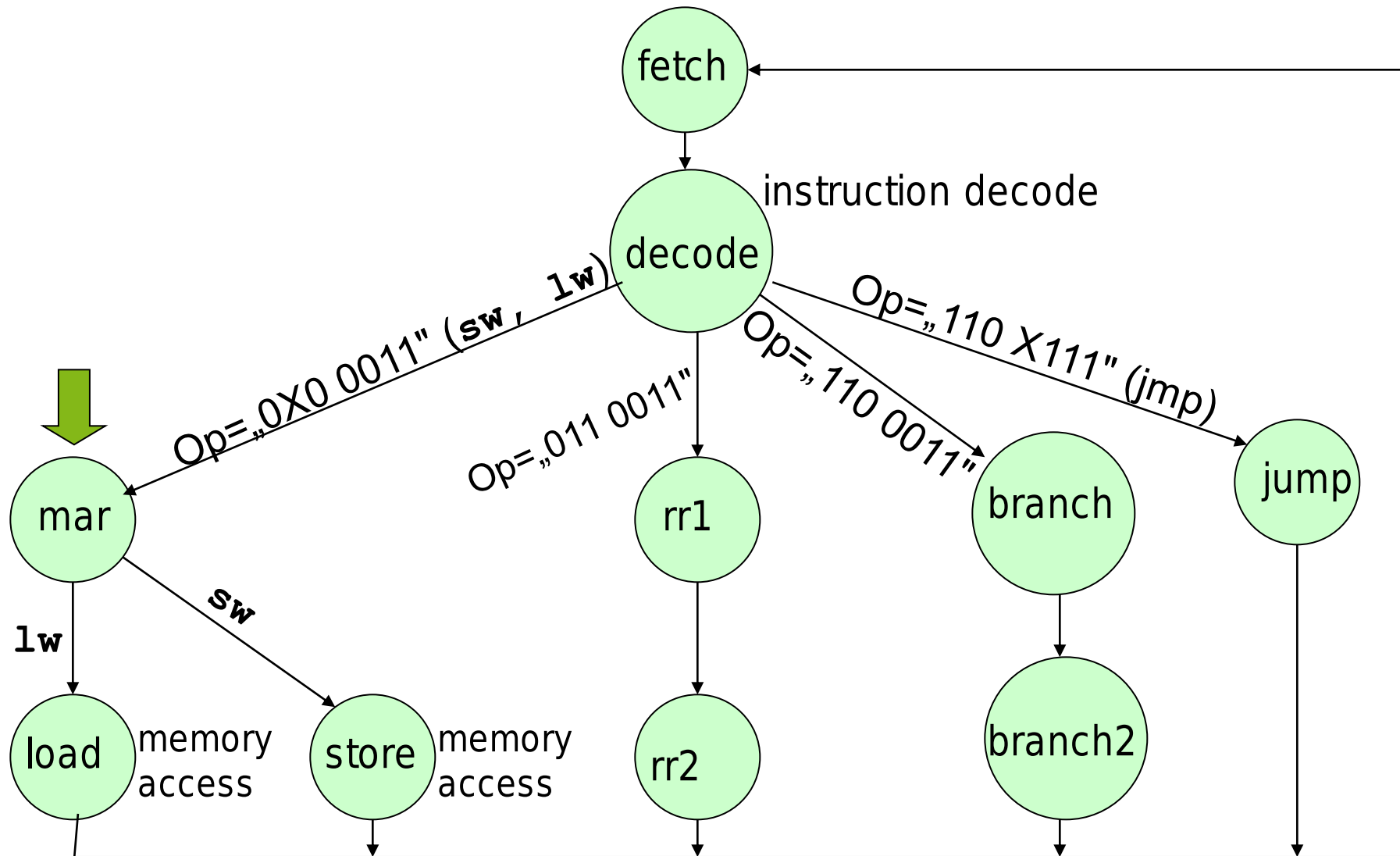
Zustandsgraph der Ausführung einiger RISC-V-Befehle



decode



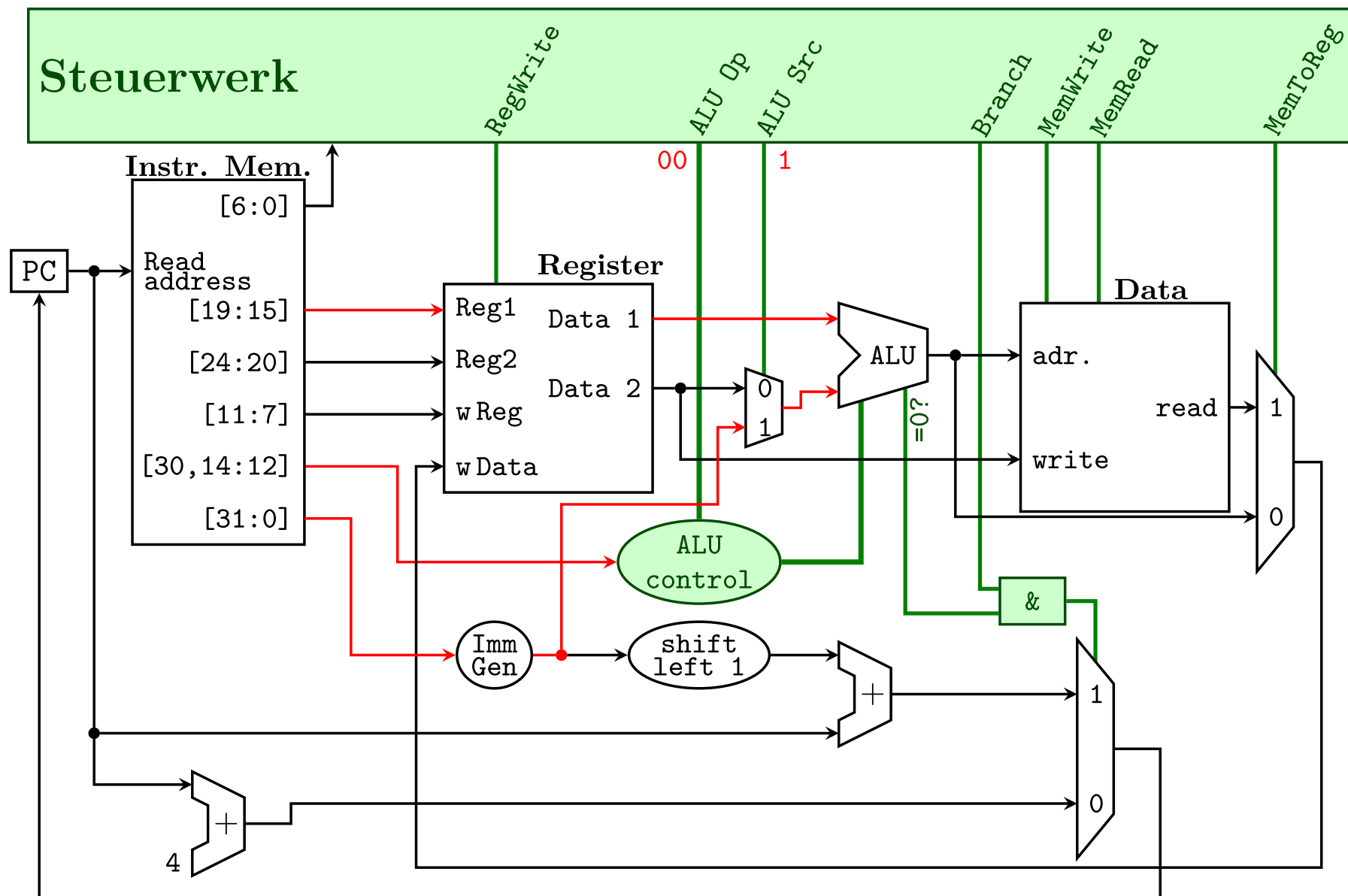
Zustandsgraph der Ausführung einiger RISC-V-Befehle



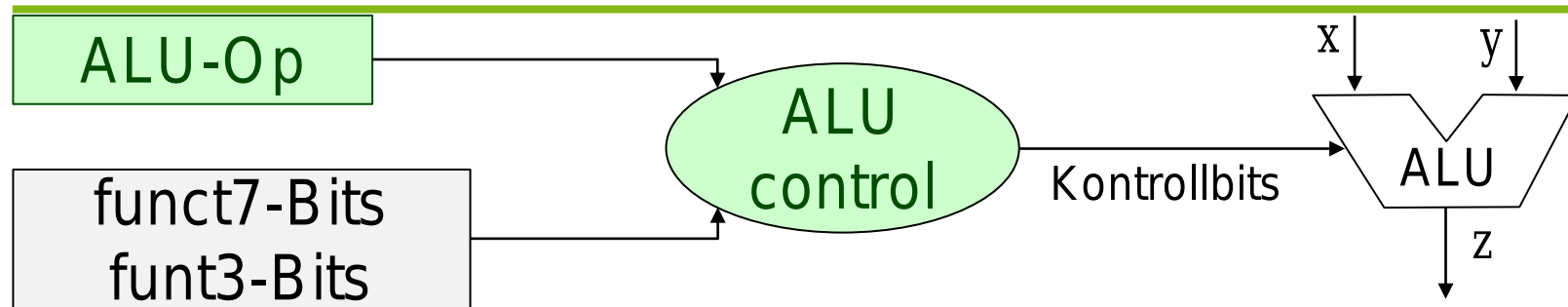
Darstellung von Befehlen

	Größe [Bit]	7	5	5	3	5	7
Arithmetik	Format R	funct7	rs2	rs1	funct3	rd	op
lw	Format I	imm(11:0)		rs1	funct3	rd	op
sw	Format S	imm (11:5)	rs2	rs1	funct3	imm(4:0)	op

mar

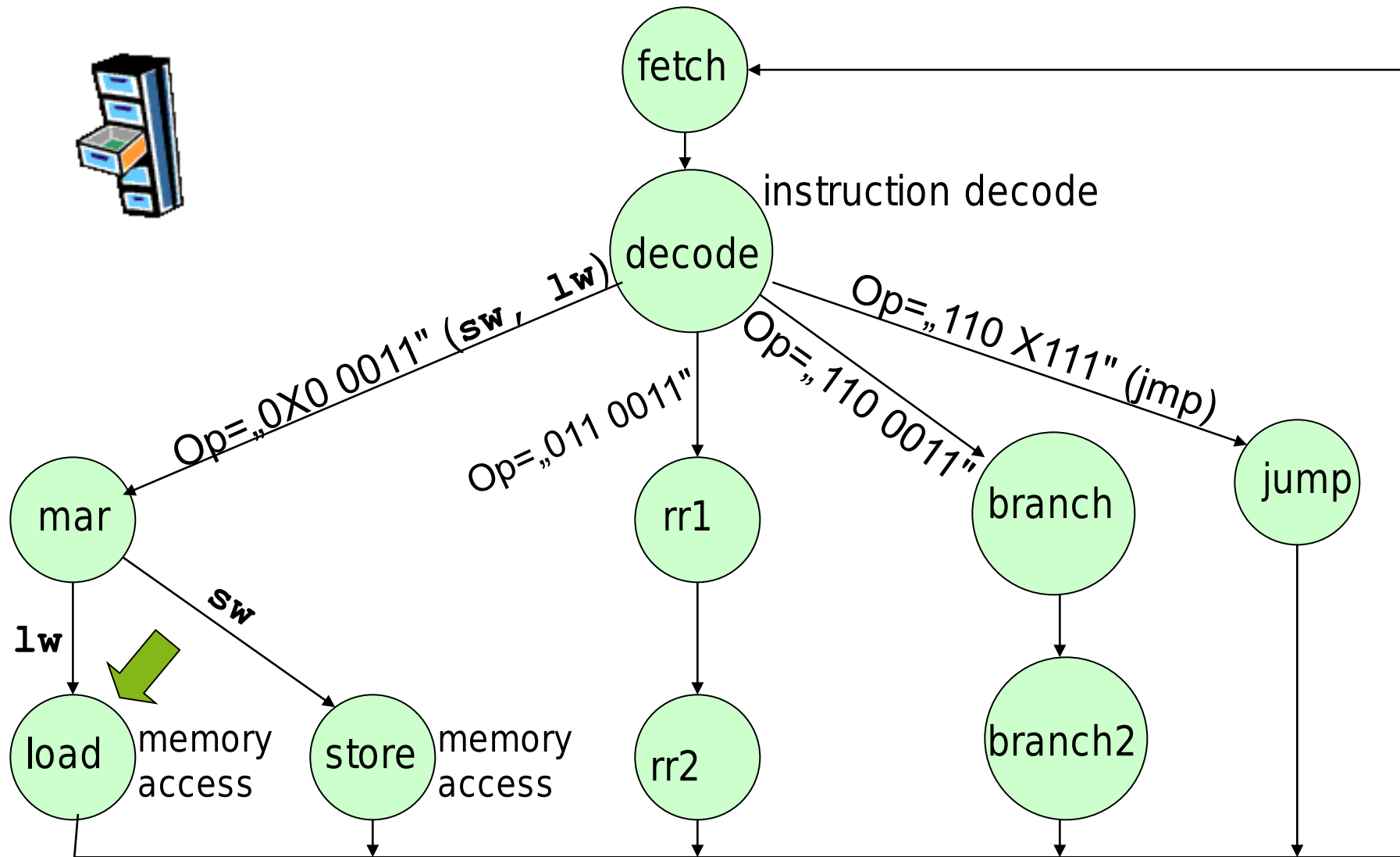


Funktionalität der ALU

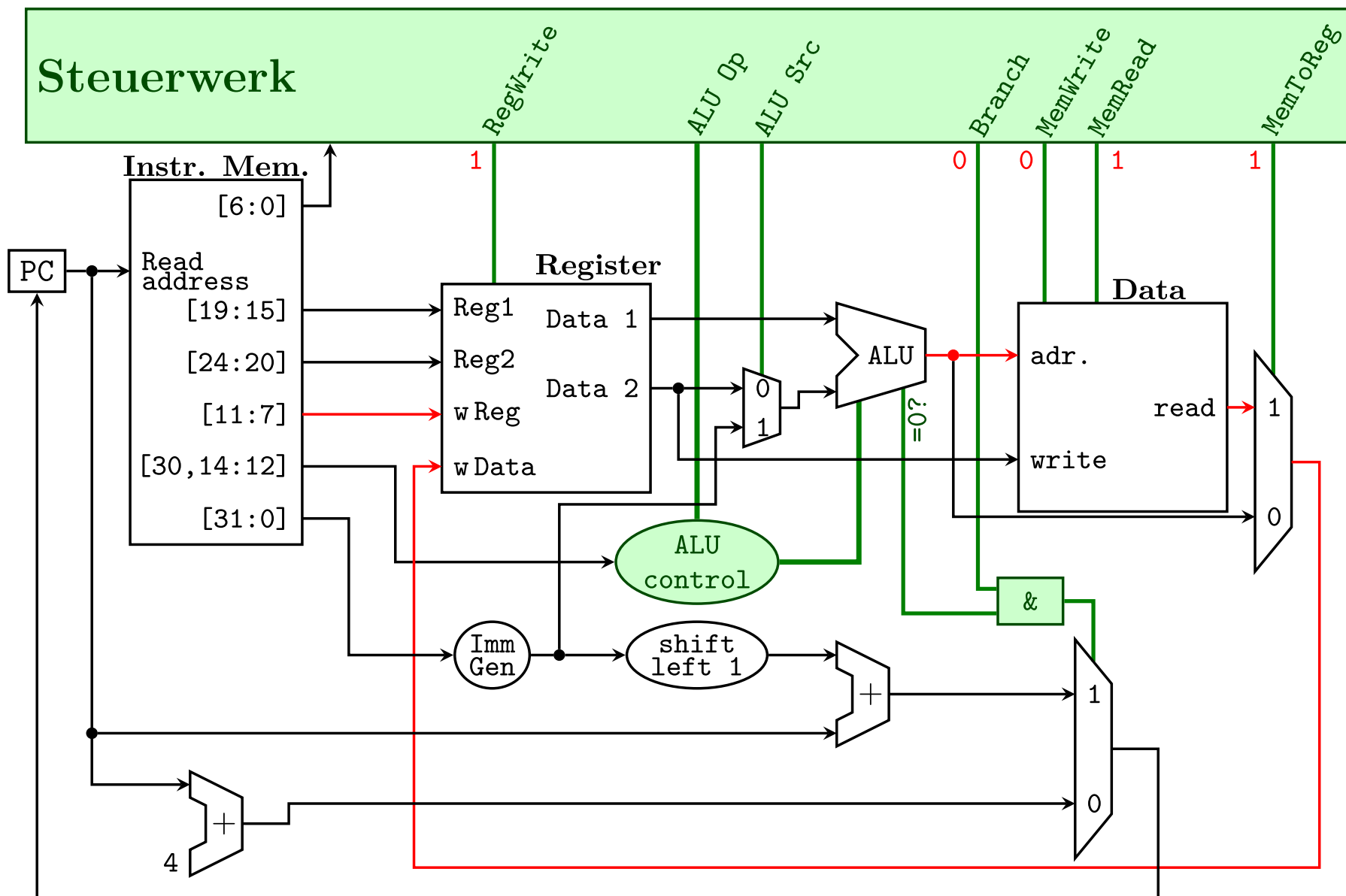


Befehl (opcode)	Operation	Kontroll bits	funct7 Feld	funct3 Feld	Gewünschte ALU Operation	ALU-Op
Lw	Load word	0010	xxxxxxx	xxx	Addition	00
Sw	Store word	0010	xxxxxxx	xxx	Addition	00
Beq	Sprung, wenn gleich	0110	xxxxxxx	xxx	Subtraktion	01
R-Type	Addition	0010	0000000	000	Addition	10
R-Type	Subtraktion	0110	0100000	000	Subtraktion	10
R-Type	Logisch AND	0000	0000000	111	AND	10
R-Type	Logisch OR	0001	0000000	110	OR	10

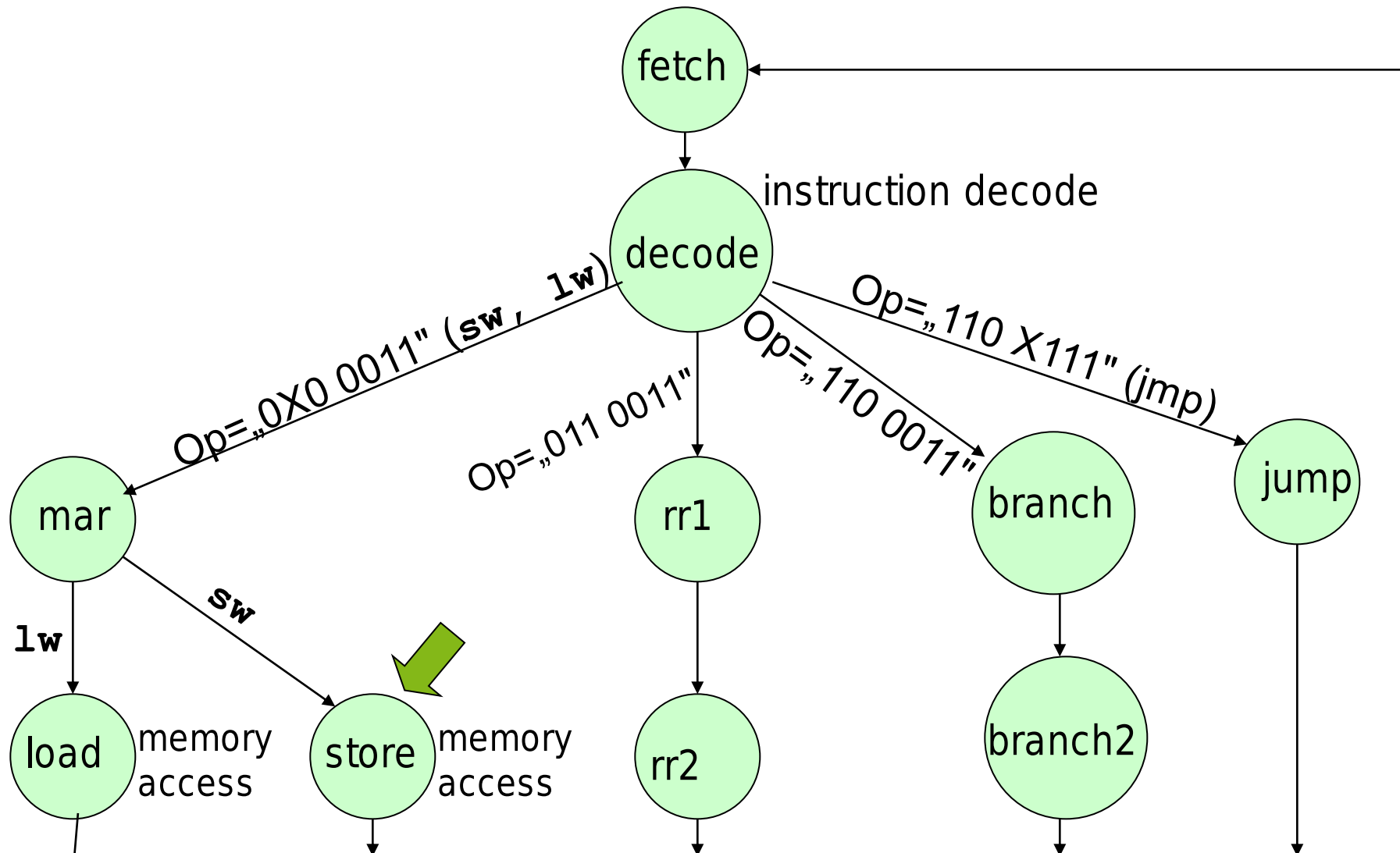
Zustandsgraph der Ausführung einiger RISC-V-Befehle



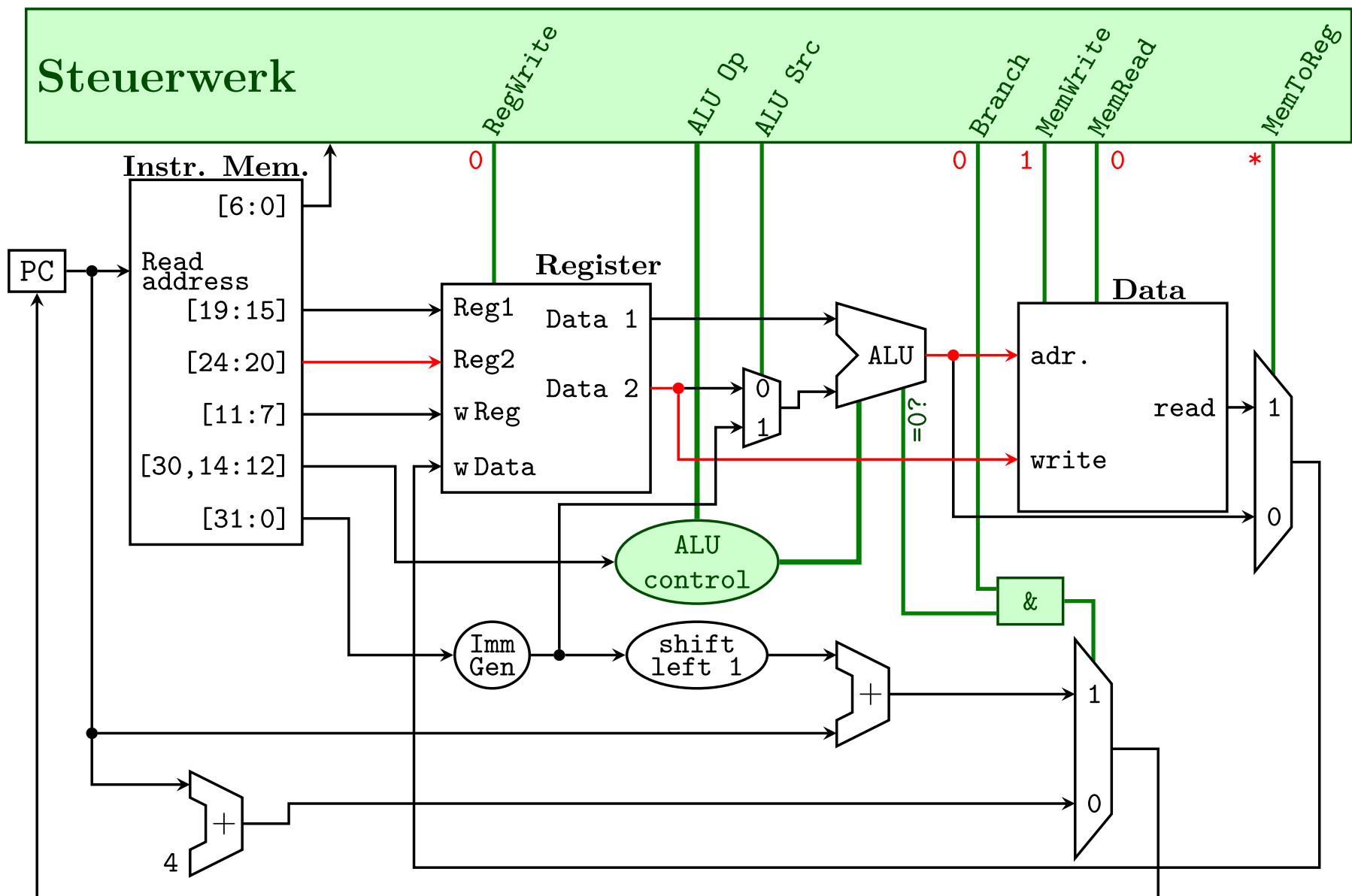
load



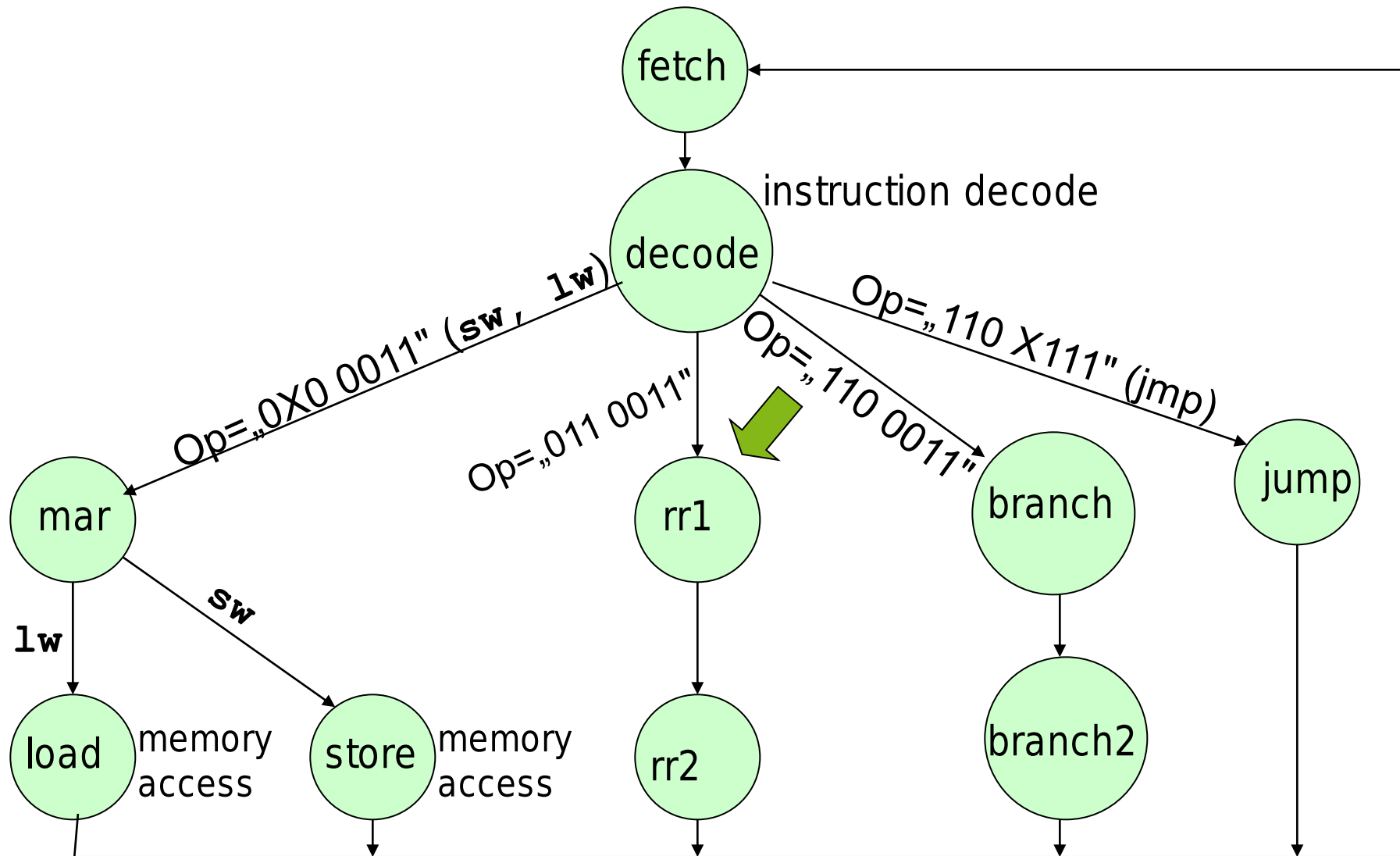
Zustandsgraph der Ausführung einiger RISC-V-Befehle



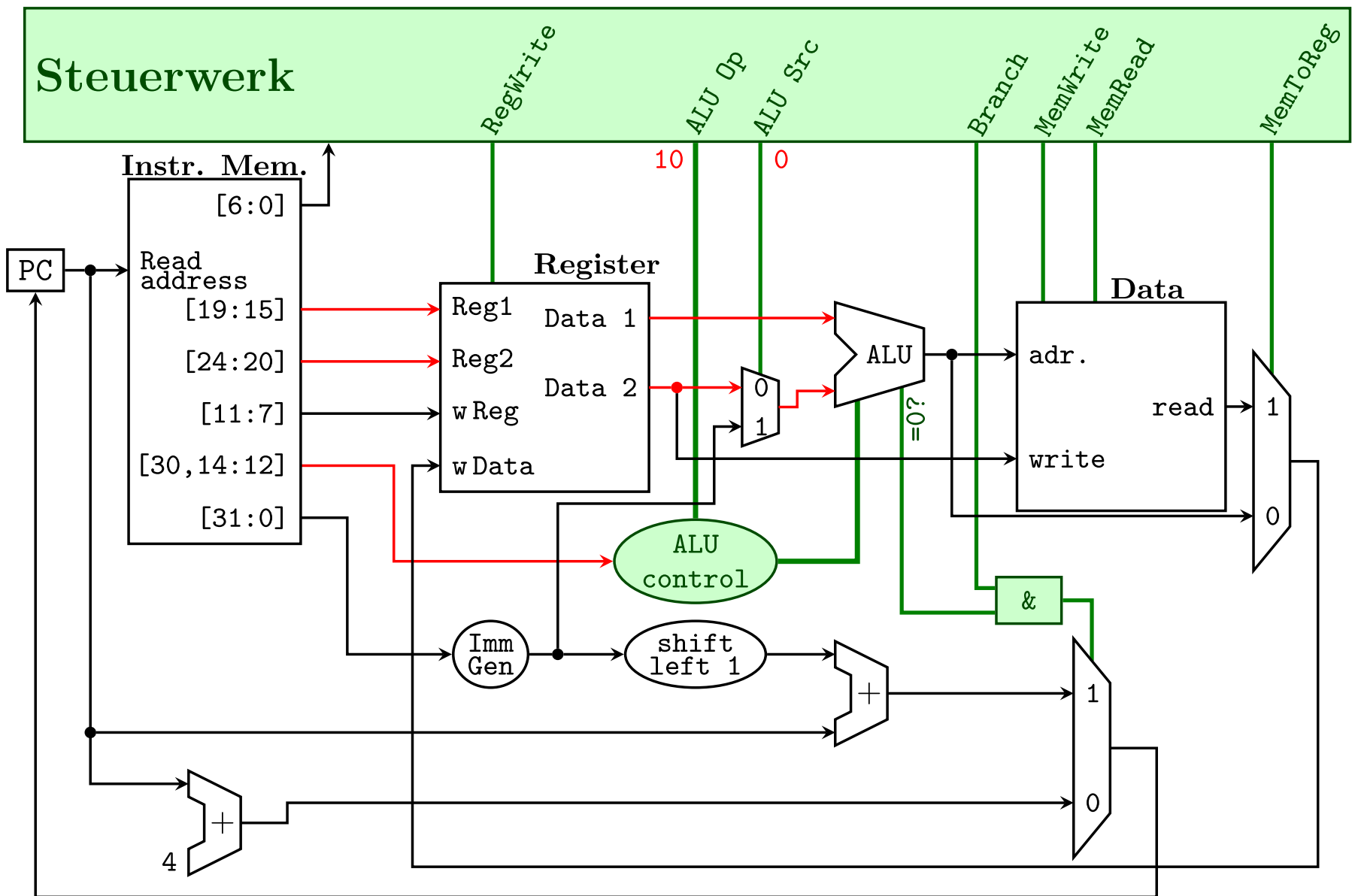
store



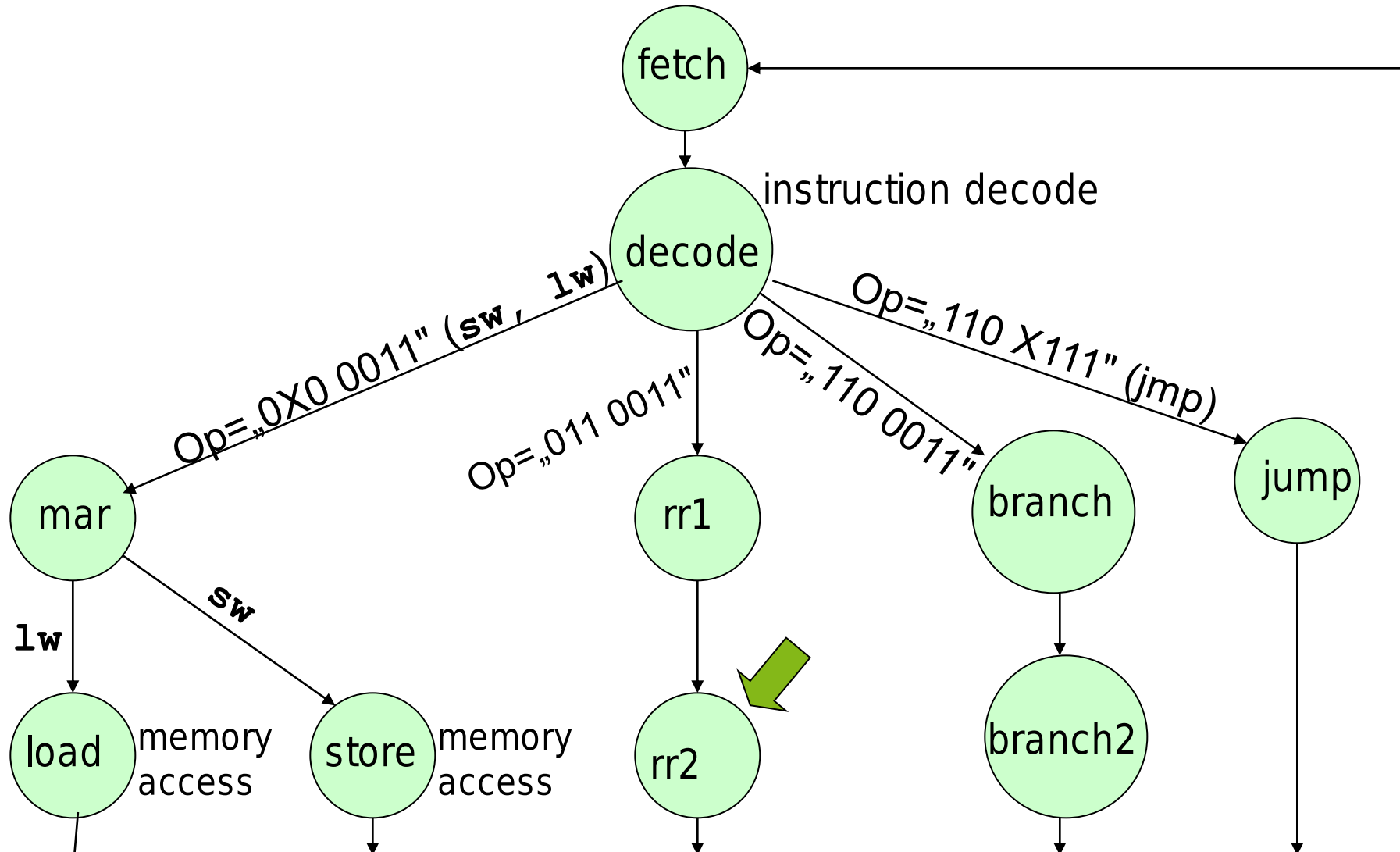
Zustandsgraph der Ausführung einiger RISC-V-Befehle



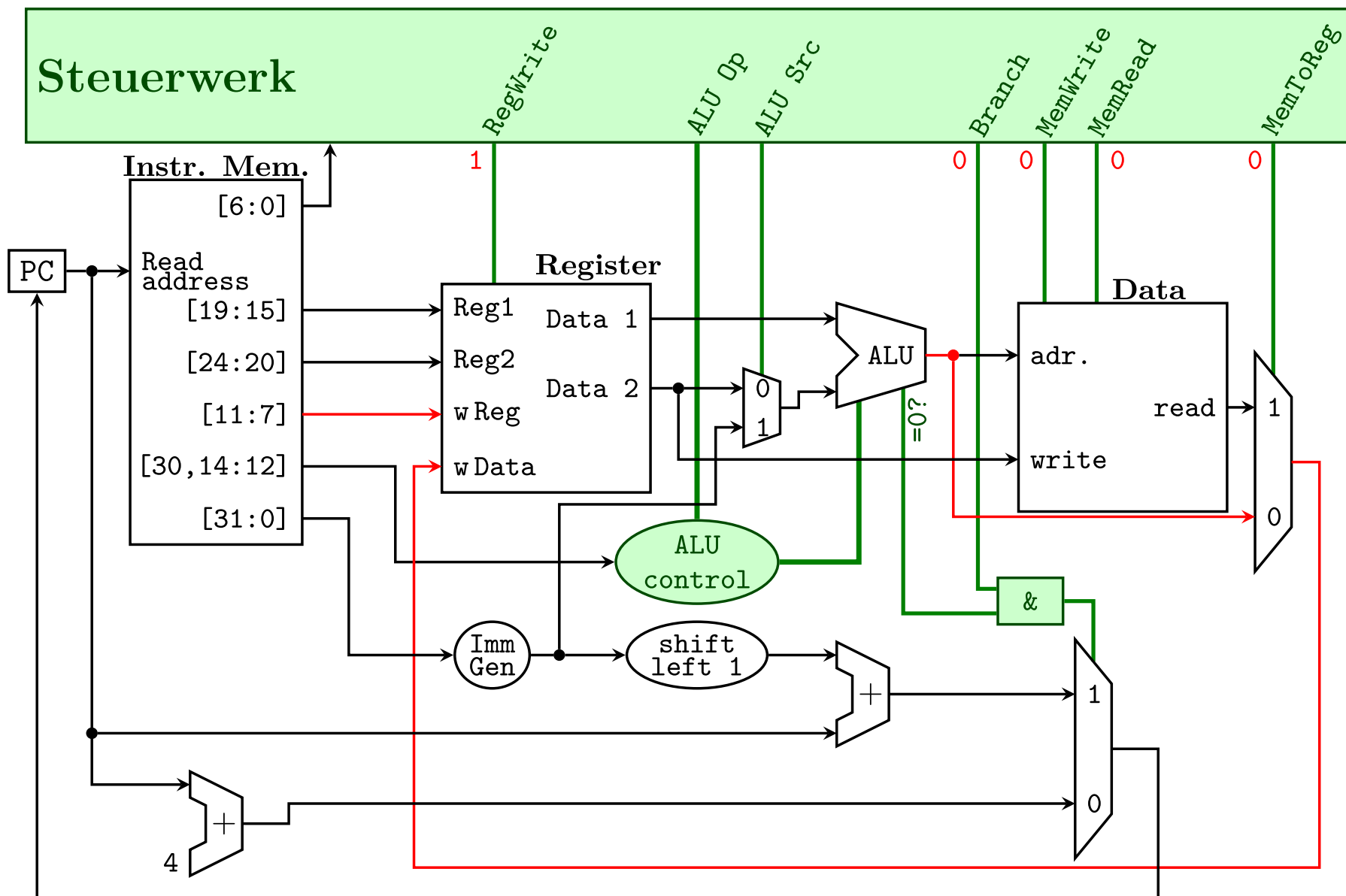
rr1



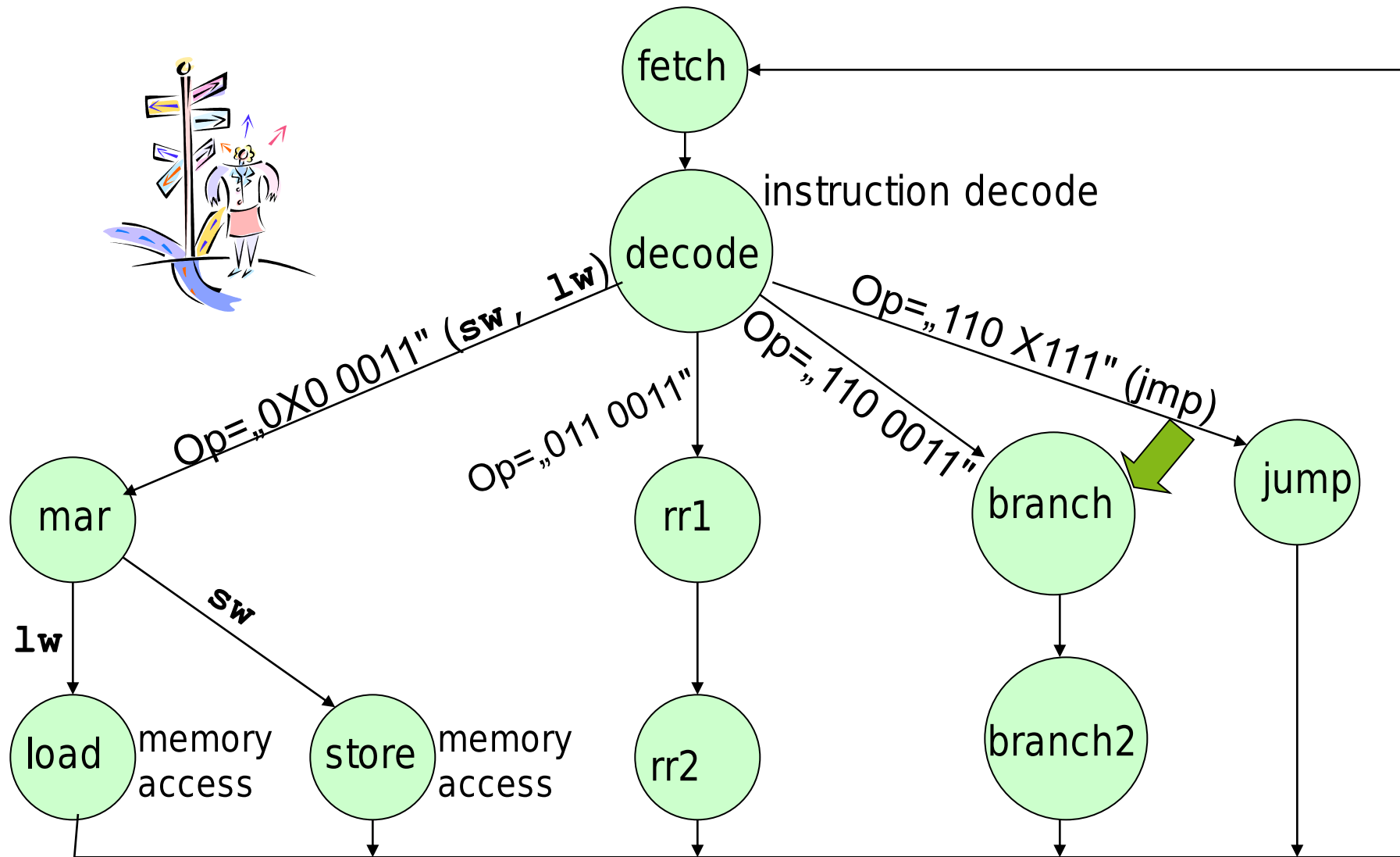
Zustandsgraph der Ausführung einiger RISC-V-Befehle



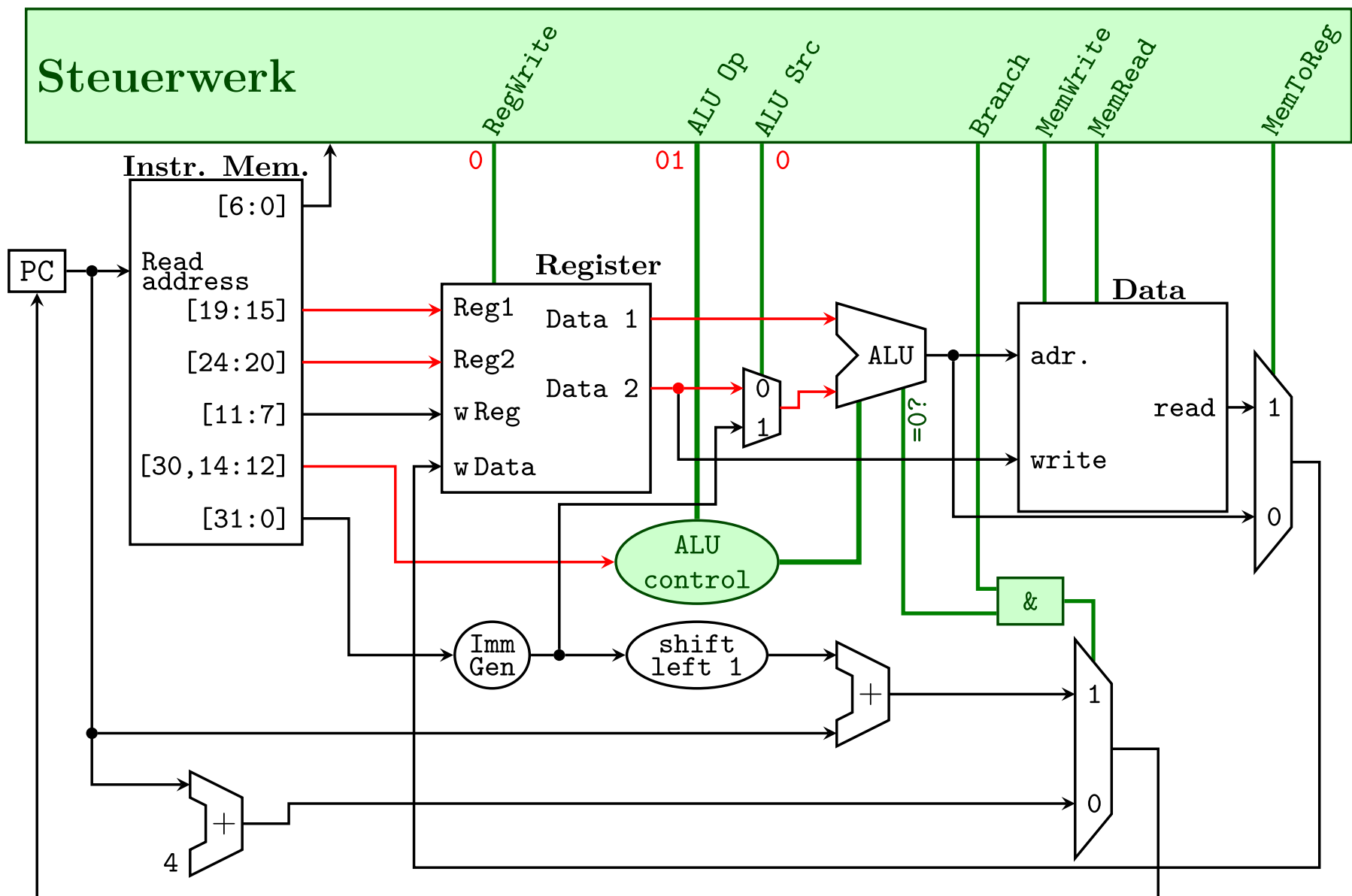
rr2



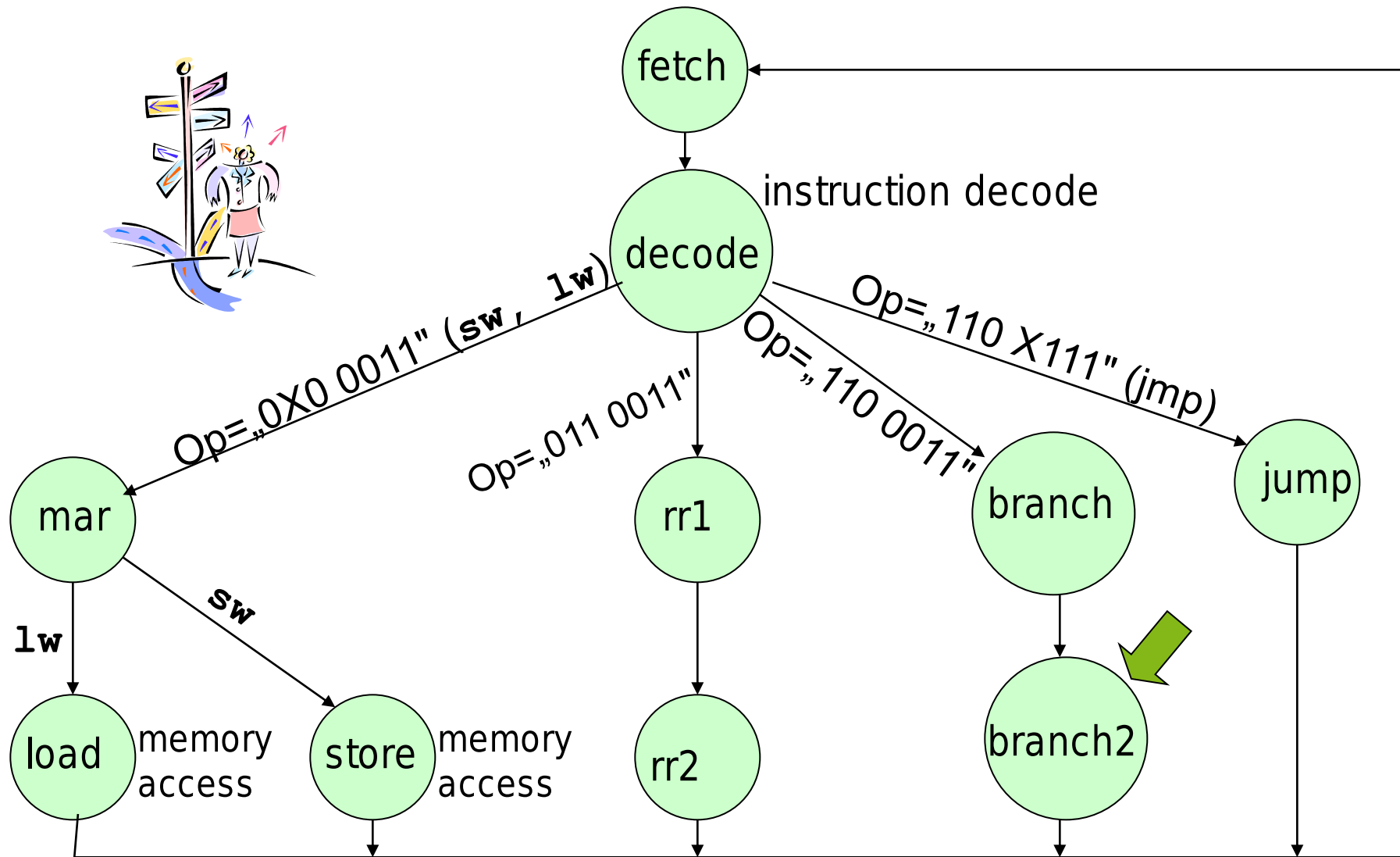
Zustandsgraph der Ausführung einiger RISC-V-Befehle



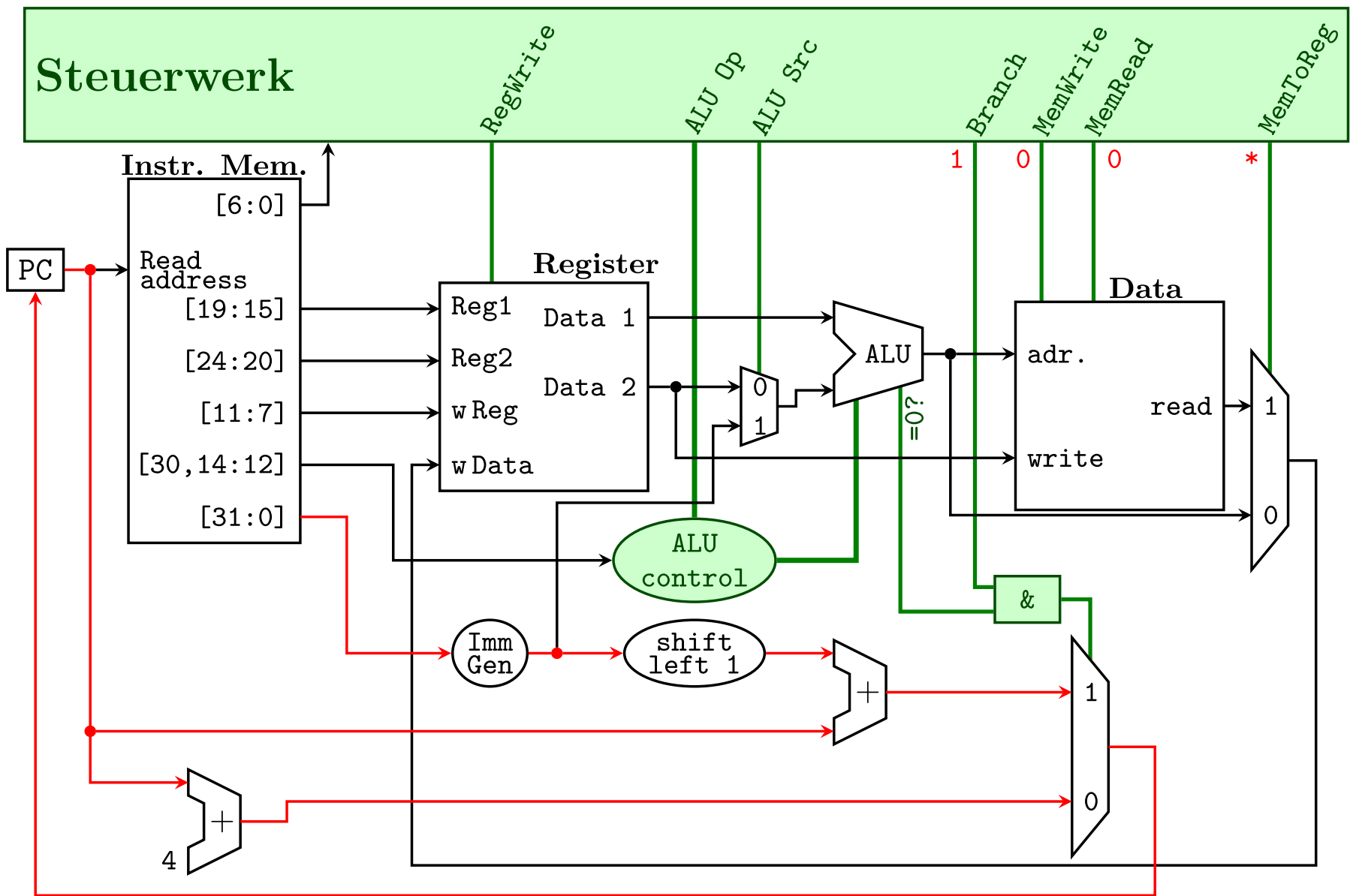
branch1



Zustandsgraph der Ausführung einiger RISC-V-Befehle



branch2



Quiz

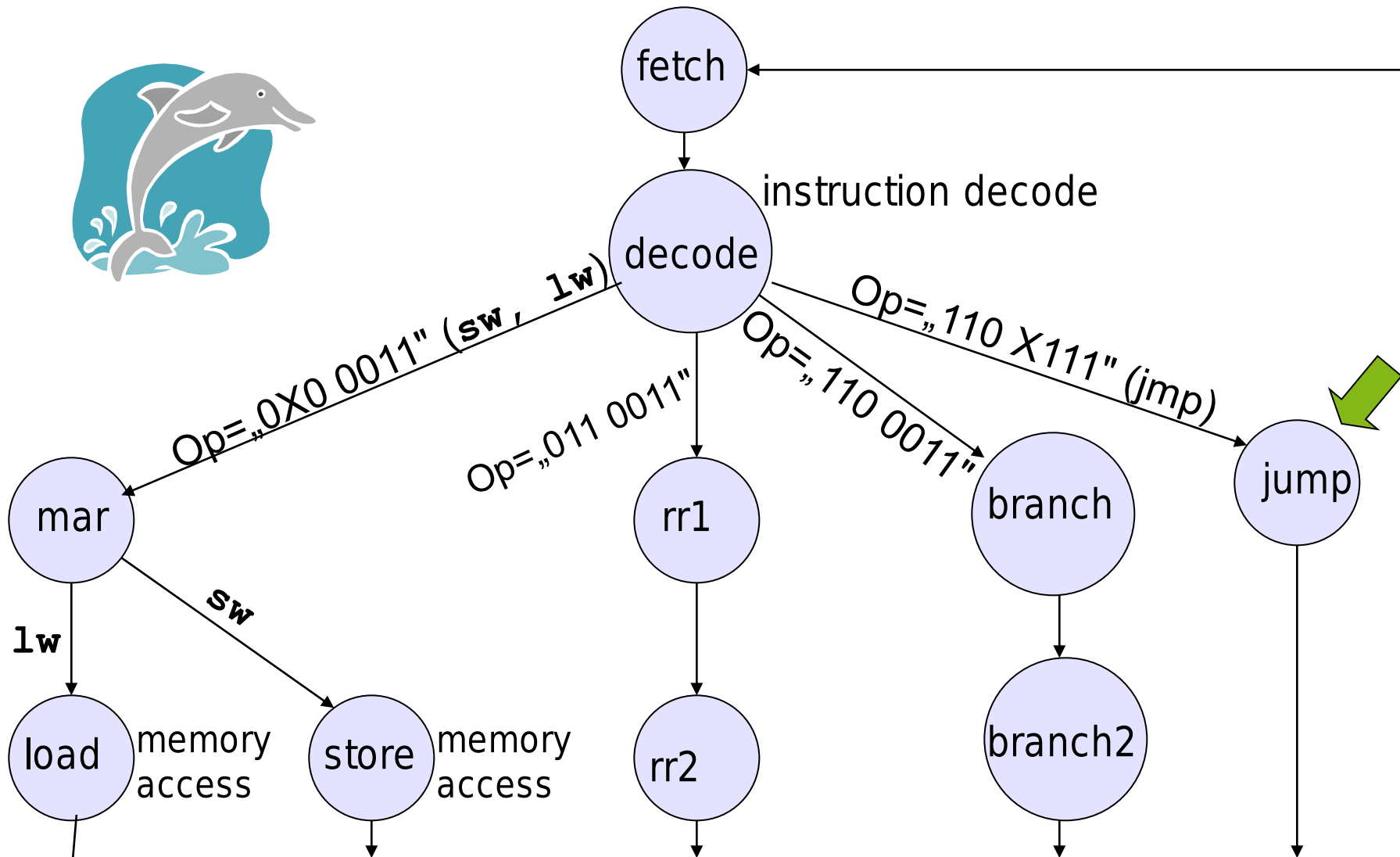
0x400000: beq x8, x9, 0x08

Welche Befehlsadresse wird nach beq (0x400000) ausgeführt, wenn x8 und x9 gleich sind?

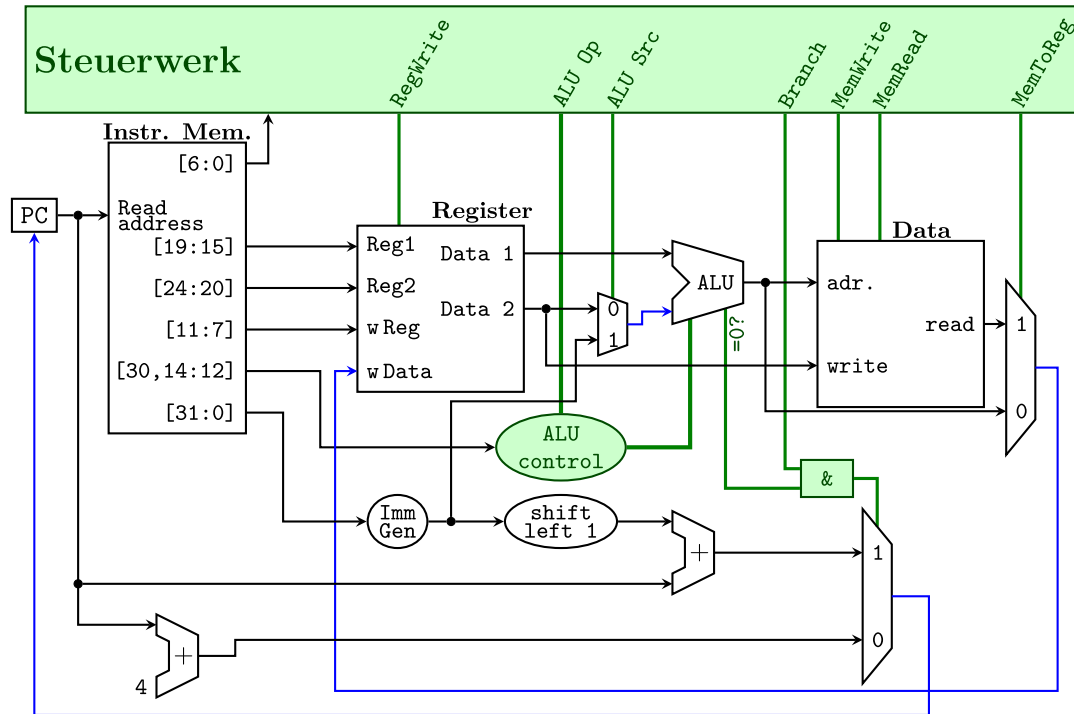
Format: **beq rs1, rs2, imm**
wobei **imm** ein Offset von 2^{12} ist ($2^{11}-1$ bis -2^{11})

**# Falls Reg[rs1] = Reg[rs2], PC := PC + sign_ext(imm & “0“, 32),
sonst PC := PC+4**

Zustandsgraph der Ausführung einiger RISC-V-Befehle



jump



Diese Befehlsart kann mit dem Steuerwerk nicht dargestellt werden.

⇒ Es gibt keine Möglichkeit, den PC in einem Register zu sichern.

Befehle vom Typ U (lui, auipc) können ebenfalls nicht dargestellt werden.

⇒ Es gibt keine direkte Verbindung vom MUX-Ausgang zum wData-Eingang, um eine Immediate zu speichern

⇒ Die Architektur ist nicht perfekt, illustriert aber die Arbeitsweise in verständlicher Form

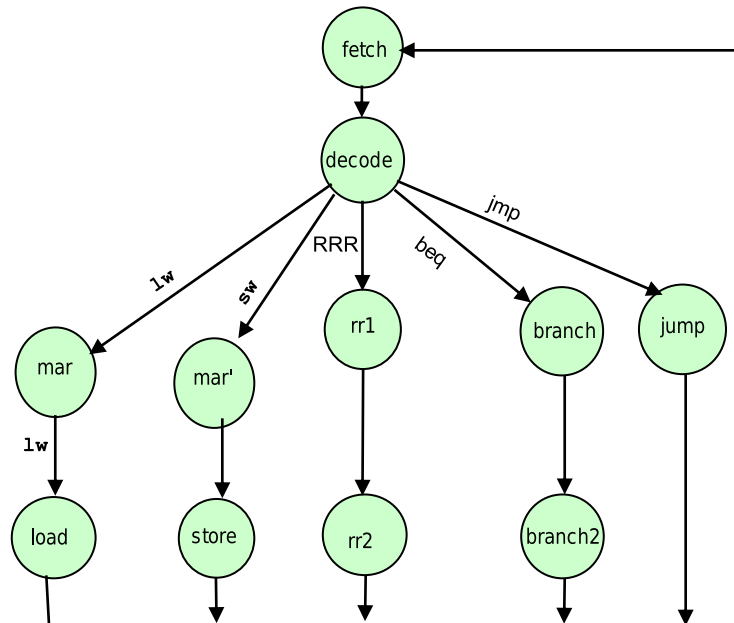
Mögliche Ergänzung des Steuerwerks zur Realisierung von `lui` und `jal` Befehlen



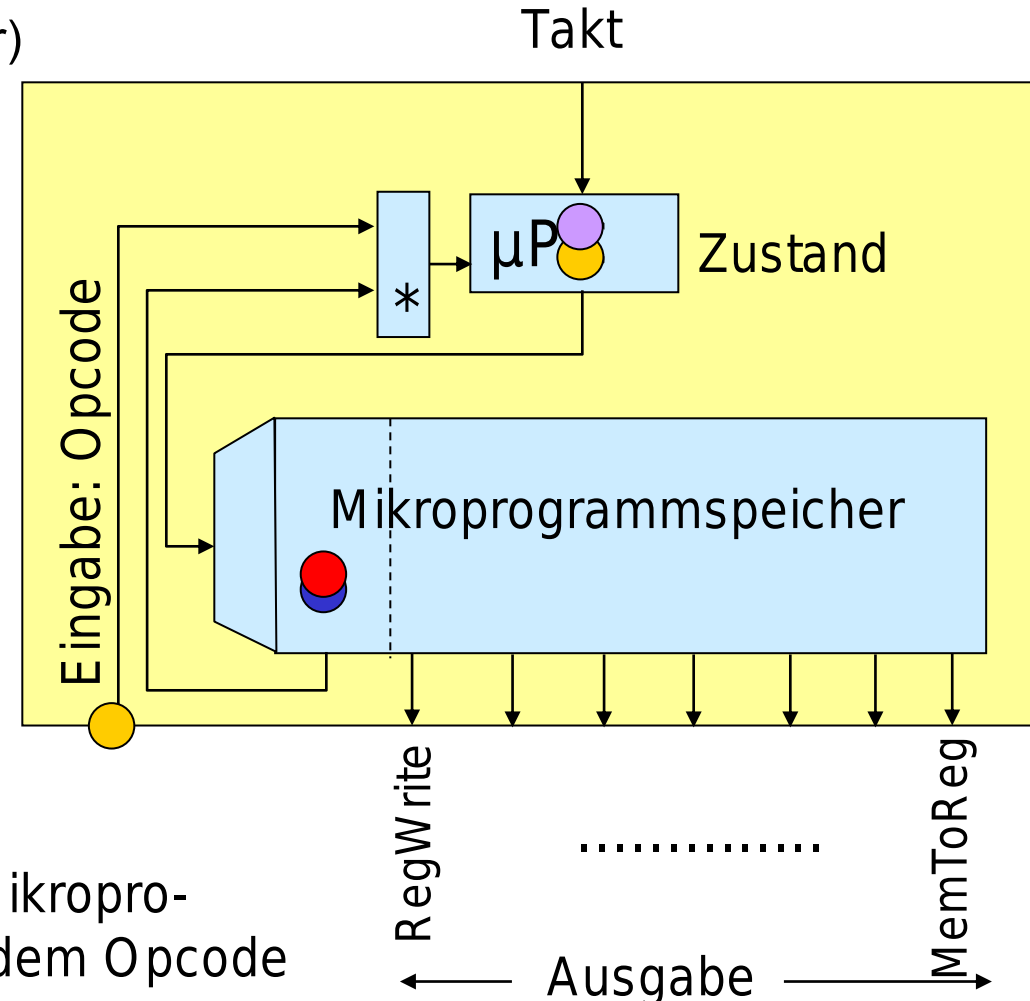
Das Steuerwerk

Verhalten, vereinfacht

(mar' vermeidet Verzweigung bei mar)

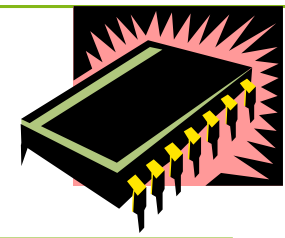


Struktur



* Folgezustand bestimmt durch Mikroprogrammspeicher, bei decode aus dem Opcode

Inhalt des Mikroprogrammspeichers



Zustand (bestimmt Adresse im Speicher)	Folge- Zustand bzw. – Zustände	+ Art der Bestimmung des Folgezustands	RegWrite	ALU Op	ALU Src	Branch	MemWrite	MemRead	MemToReg
fetch	decode								
decode	f(Opcode)								
mar, mar'	load, store			00	1				
load	fetch		1			0	0	1	1
store	fetch		0			0	1	0	*
rr1	rr2			10	0				
rr2	fetch		1			0	0	0	0
branch1	branch2		0	01	0				
branch2	fetch					1	0	0	*

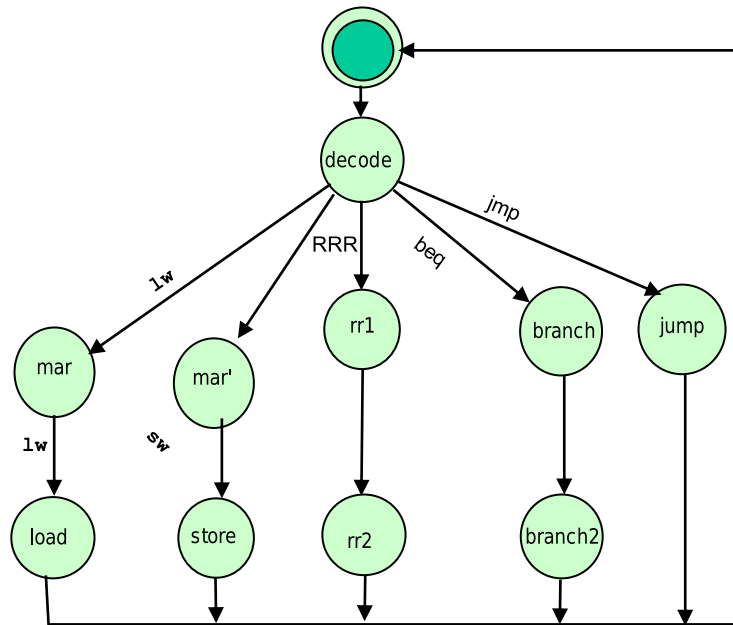
[] Für aktuellen Status nicht relevant

* Belegung irrelevant

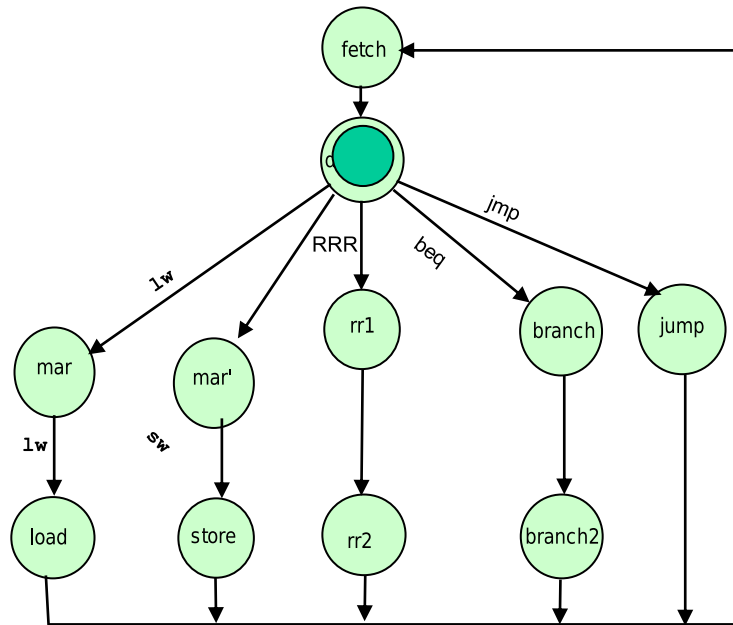
MemRead wird immer auf 0 gesetzt, um unnötige Lesezugriffe (Kosten) zu vermeiden

Stattdessen könnte hier auch ein * stehen.

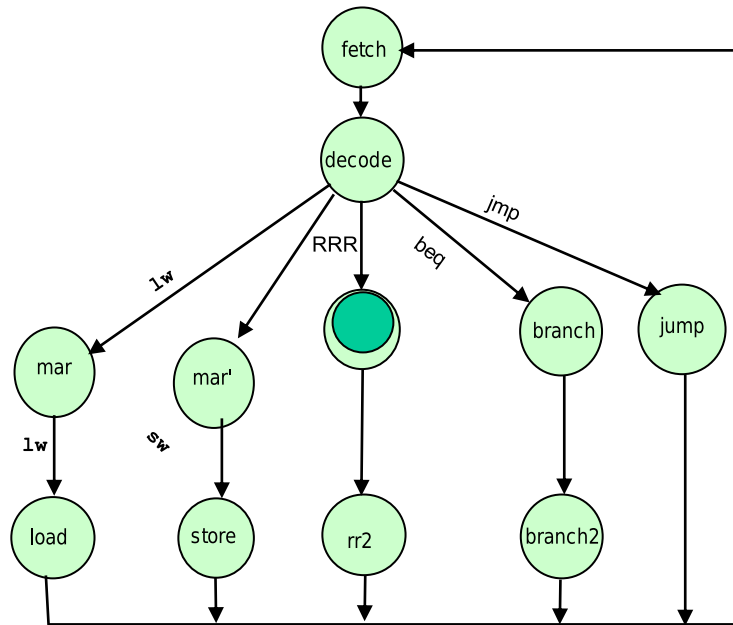
Mikroprogrammierung



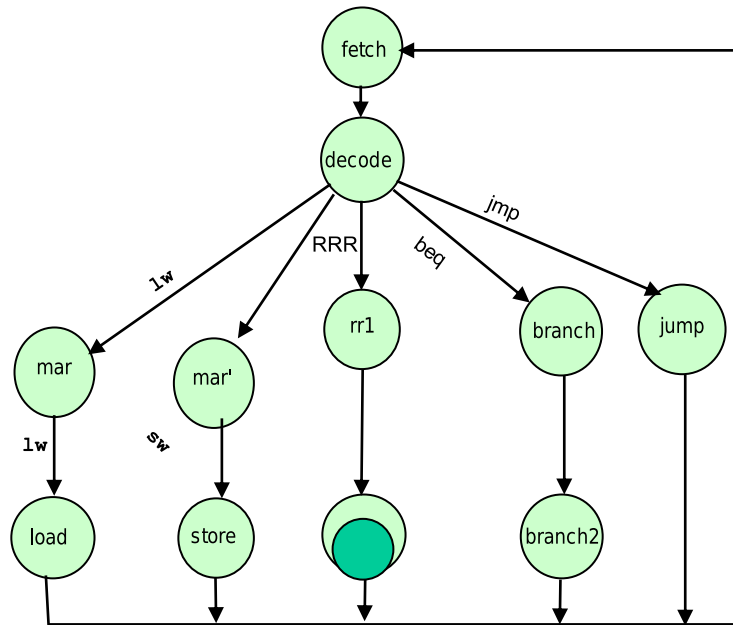
Mikroprogrammierung



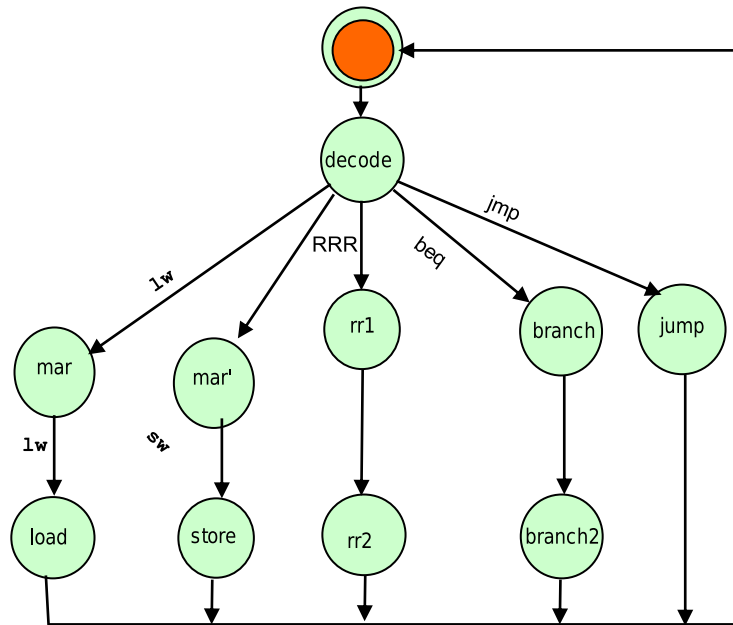
Mikroprogrammierung



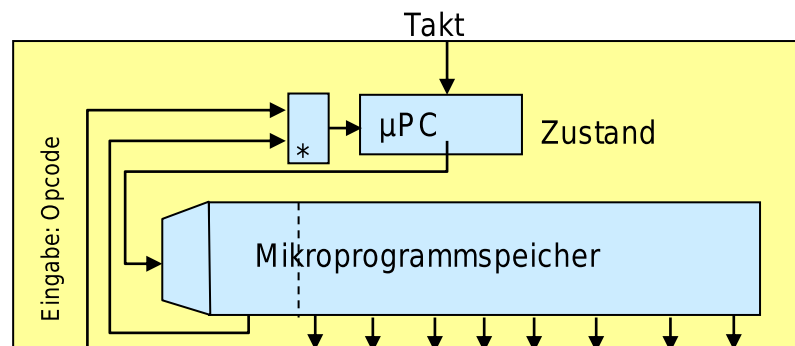
Mikroprogrammierung



Mikroprogrammierung



Struktur



Zusammenfassung

Mikroprogrammierung gestattet die strukturierte Realisierung von Rechensystemen aus RT-Struktur-Komponenten

Vorteile:

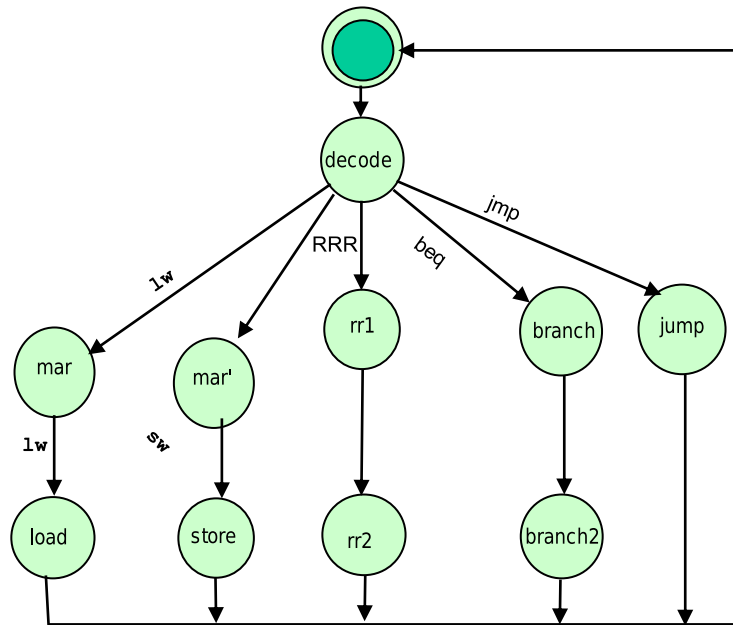
- einfache, strukturierte Realisierung auch großer, komplexer Befehlssätze
- leichte Änderbarkeit

Nachteile:

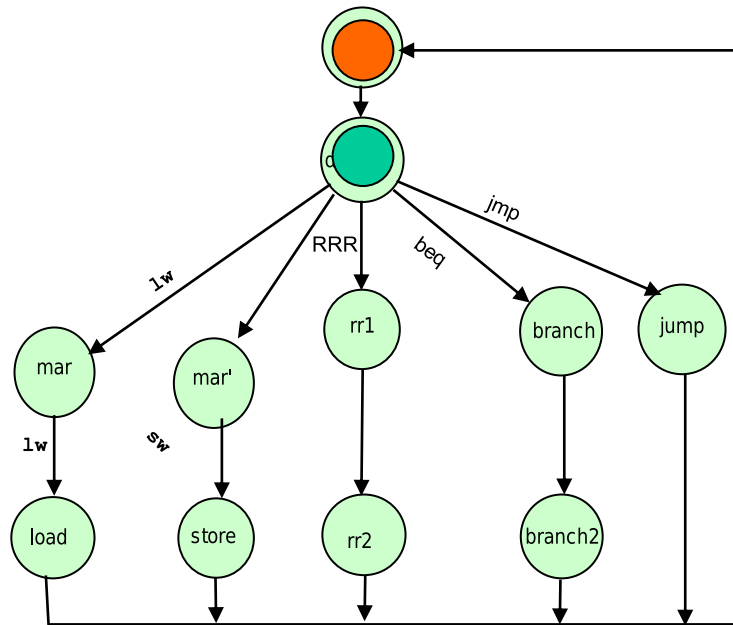
- Overhead (*fetch* und *decode* enthalten keine Operationen des auszuführenden Programms)
- Große CPI-Werte

- → Versuch, Mikroprogramme zu vermeiden

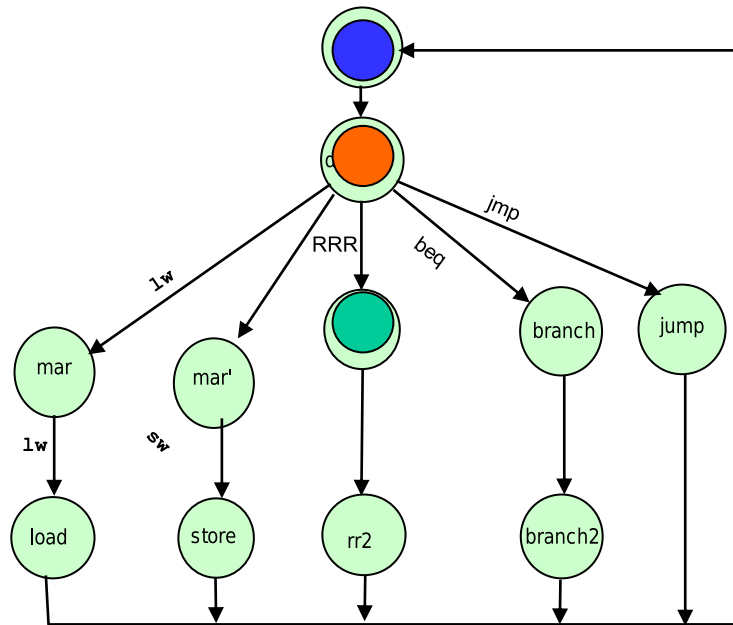
Mikroprogrammierung → Fließbandverarbeitung



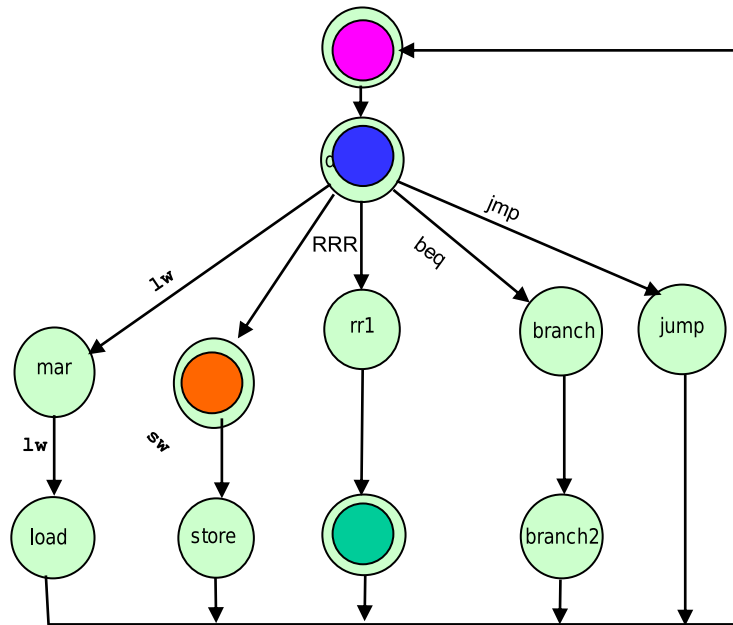
Mikroprogrammierung → Fließbandverarbeitung



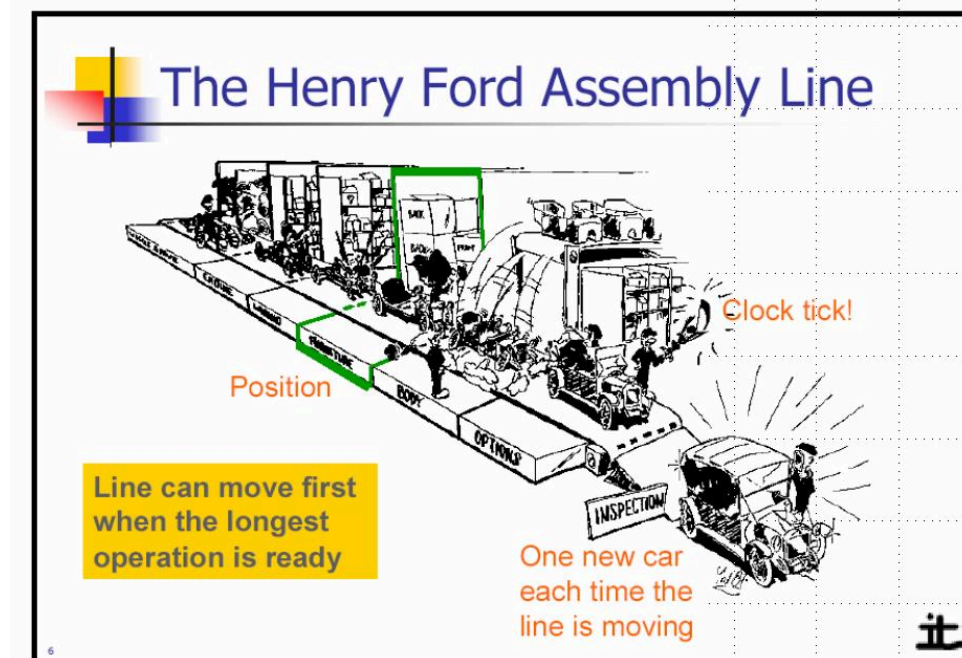
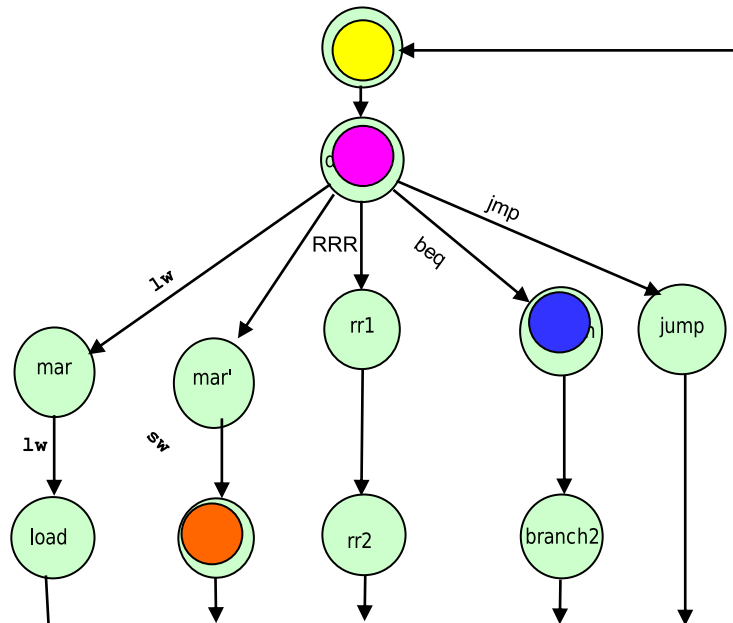
Mikroprogrammierung → Fließbandverarbeitung



Mikroprogrammierung → Fließbandverarbeitung



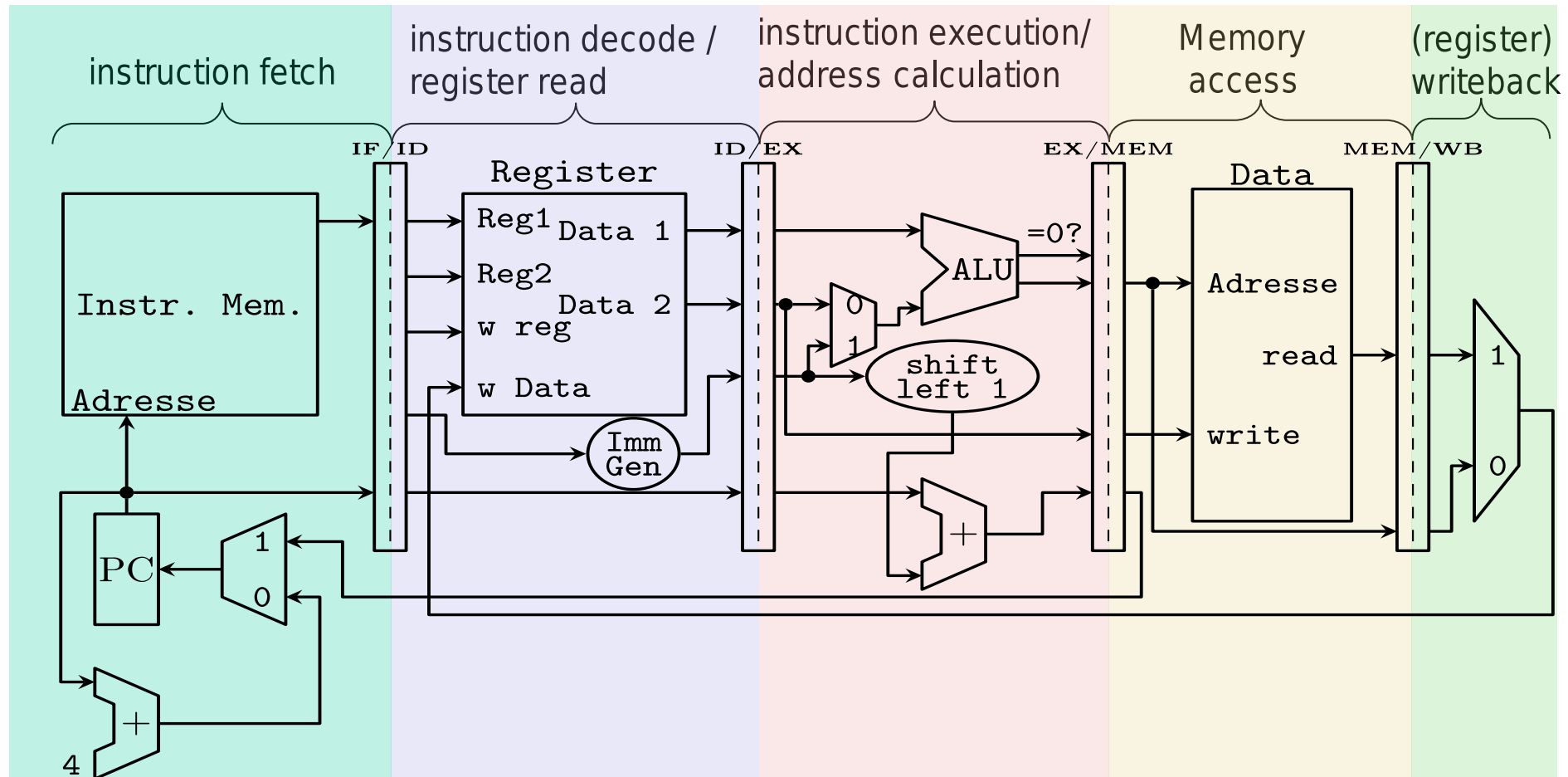
Mikroprogrammierung → Fließbandverarbeitung



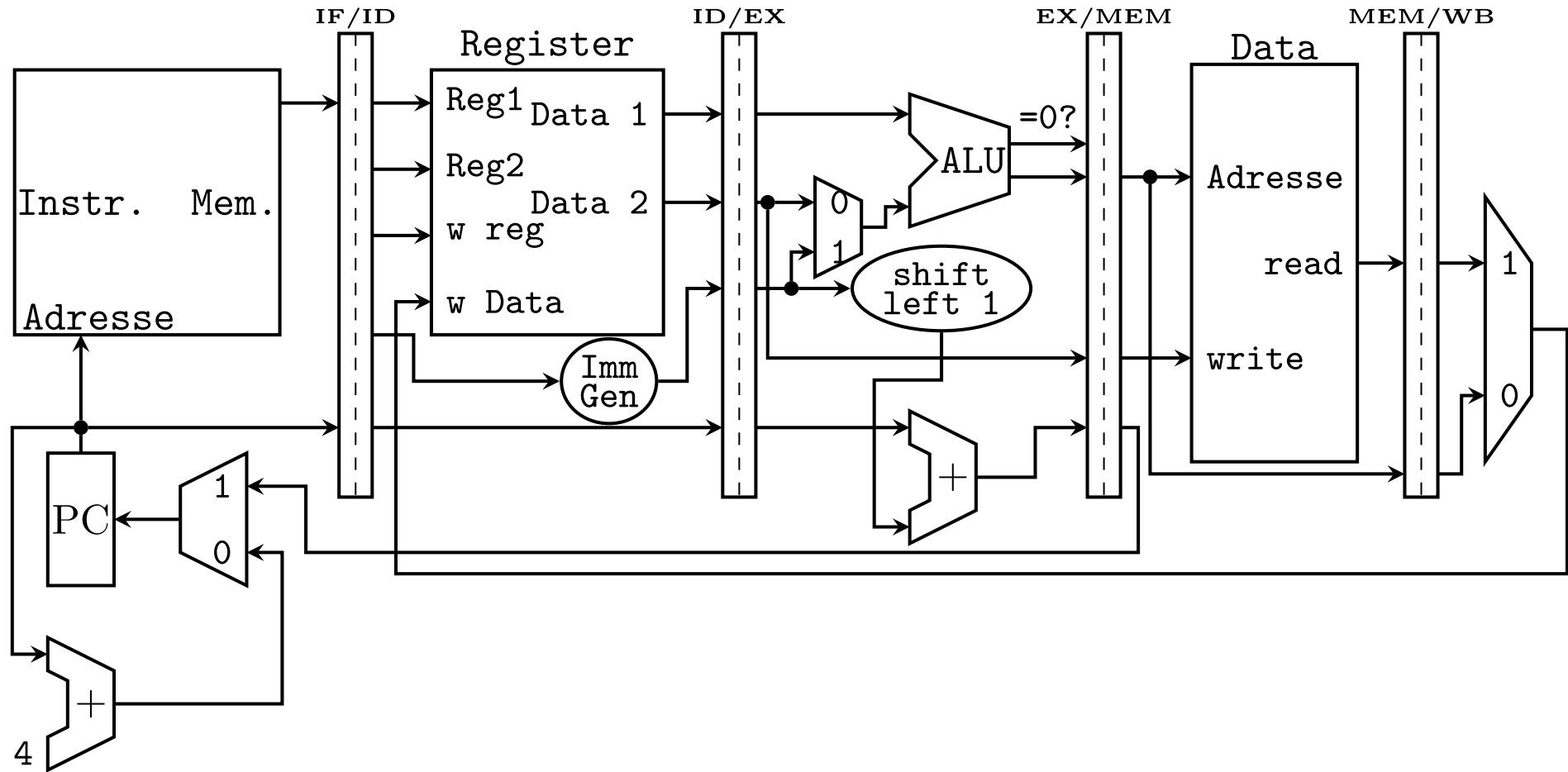
www.it.lth.se/courses/dsi/material/Lectures/Lecture6.pdf

2.3.2 Fließbandverarbeitung

Fließband-Architektur (engl. pipeline architecture): Bearbeitung mehrerer Befehle gleichzeitig, analog zu Fertigungsfließbändern. Beispiel RISC-V:



Änderungen gegenüber der Struktur ohne Fließband



- Aufteilung des Rechenwerks in Fließbandstufen, Trennung durch Pufferregister

- Steuerwerk nicht dargestellt (deutlich komplexer als bei Mikroprogrammierung)

Aufgaben der einzelnen Phasen bzw. Stufen

- **Befehlsholphase**

Lesen des aktuellen Befehls; separater Speicher, zur Vermeidung von Konflikten mit Datenzugriffen (→ Cache).

- **Dekodier- und Register-Lese-Phase**

Lesen der Register möglich wegen fester Plätze für Nr.

- **Ausführungs- und Adressberechnungsphase**

Berechnung arithmetischer Funktion bzw. Adresse für Speicherzugriff.

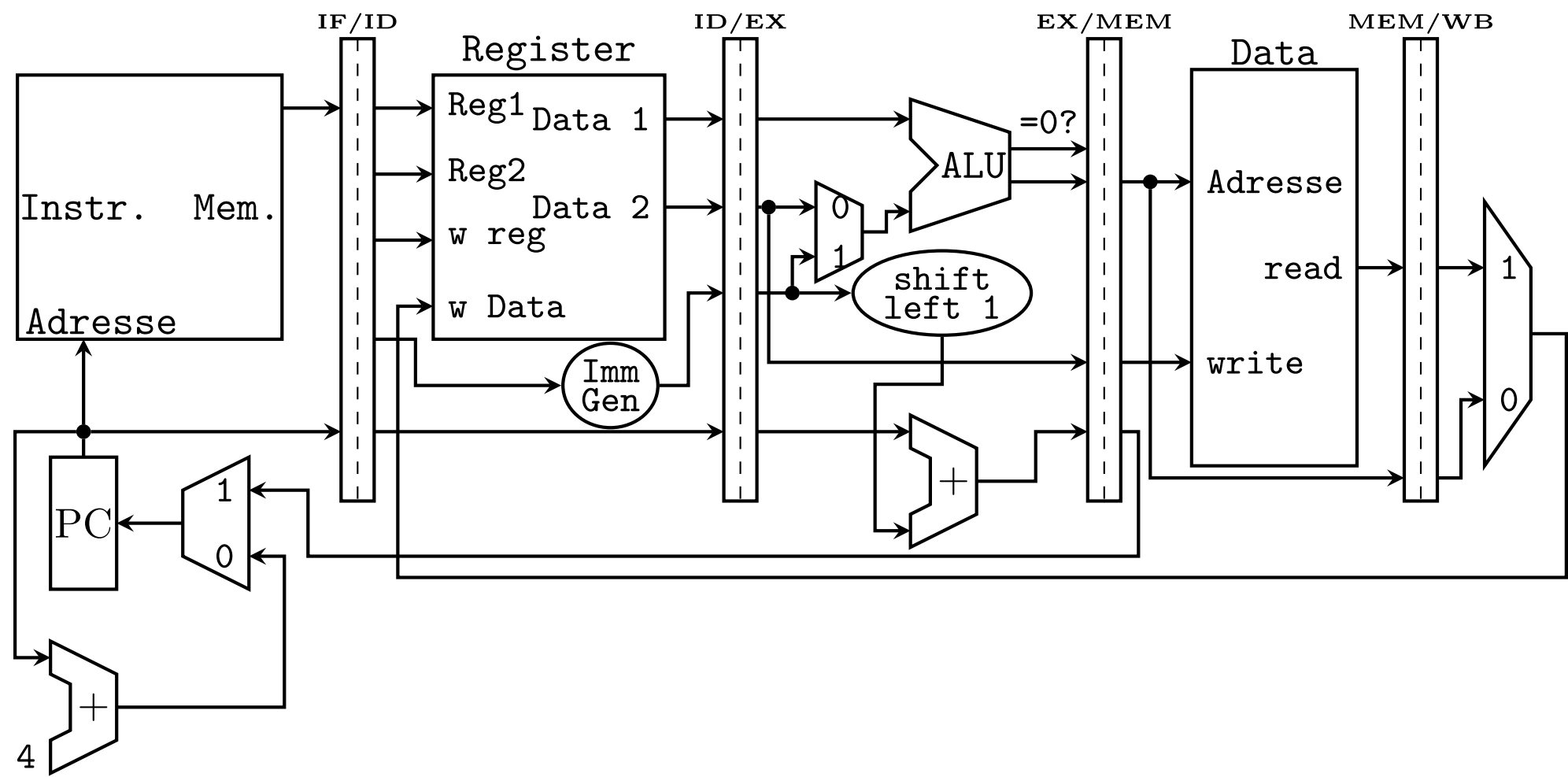
- **Speicherzugriffsphase**

Wird nur bei Lade- und Speicherbefehlen benötigt.

- **Abspeicherungsphase**

Speichern in Register, bei Speicherbefehlen nicht benötigt.

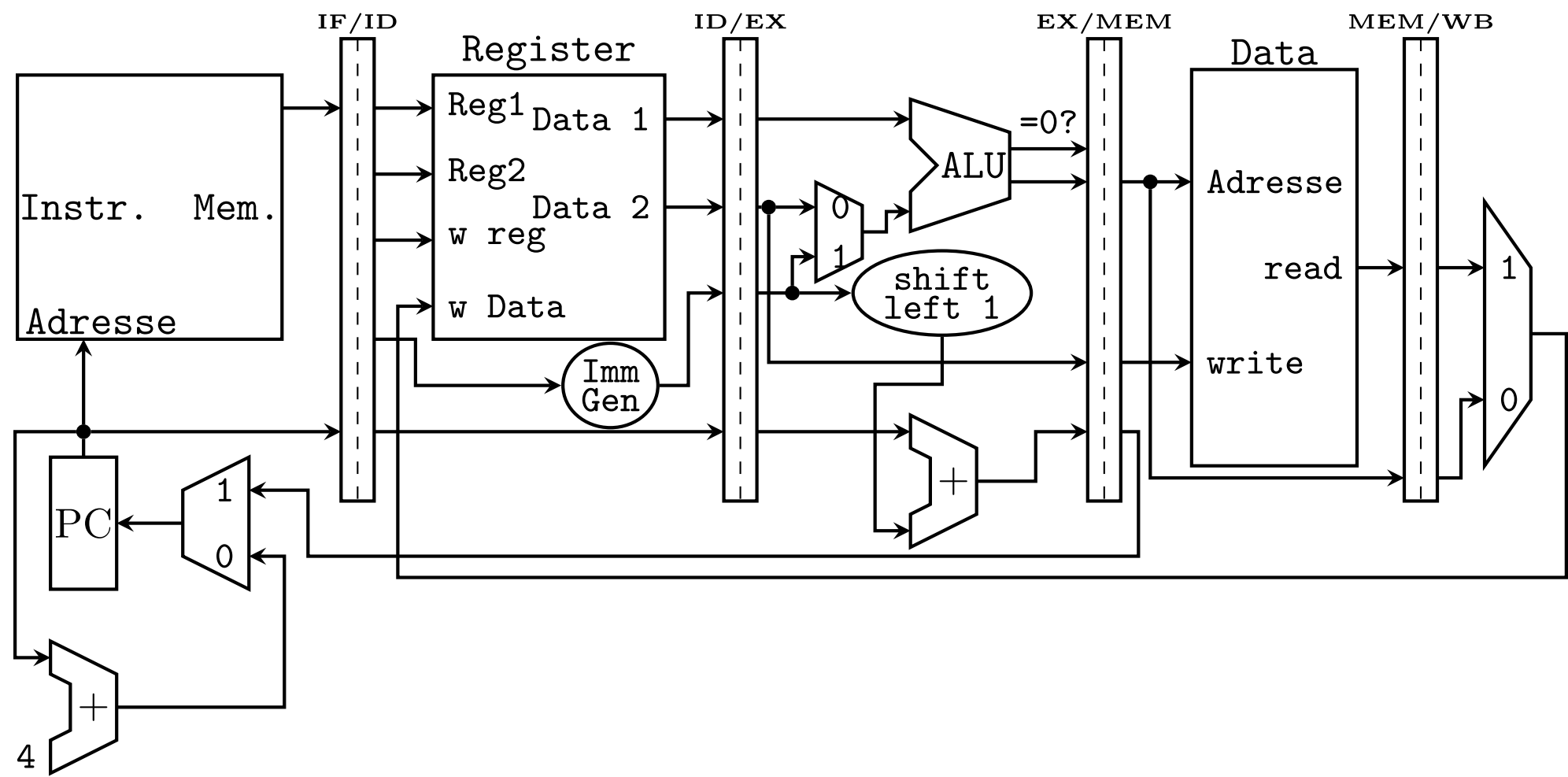
Idealer Fließbanddurchlauf



Zyklus 1

Befehl1

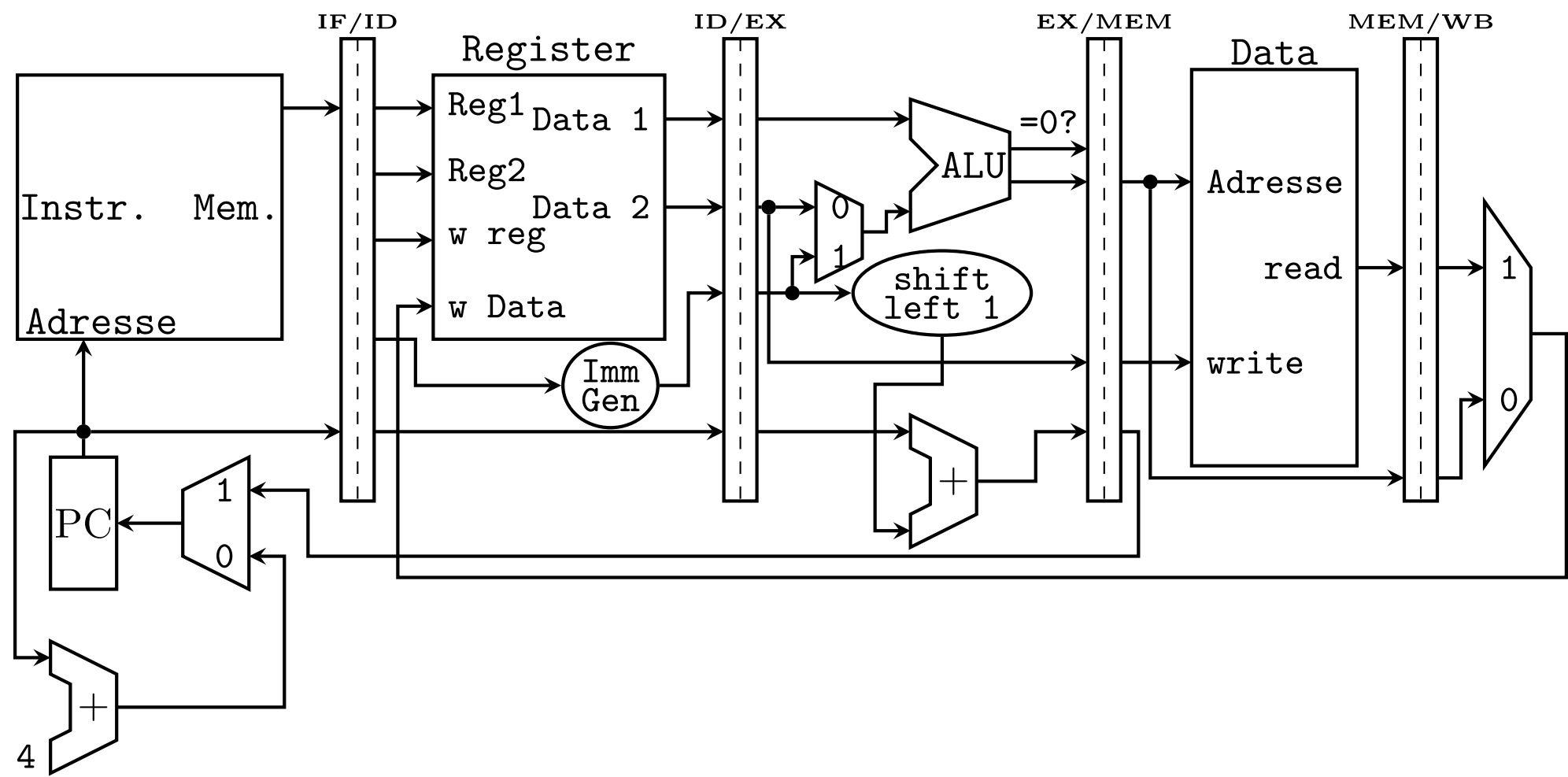
Idealer Fließbanddurchlauf



Zyklus 2

Befehl2
Befehl1

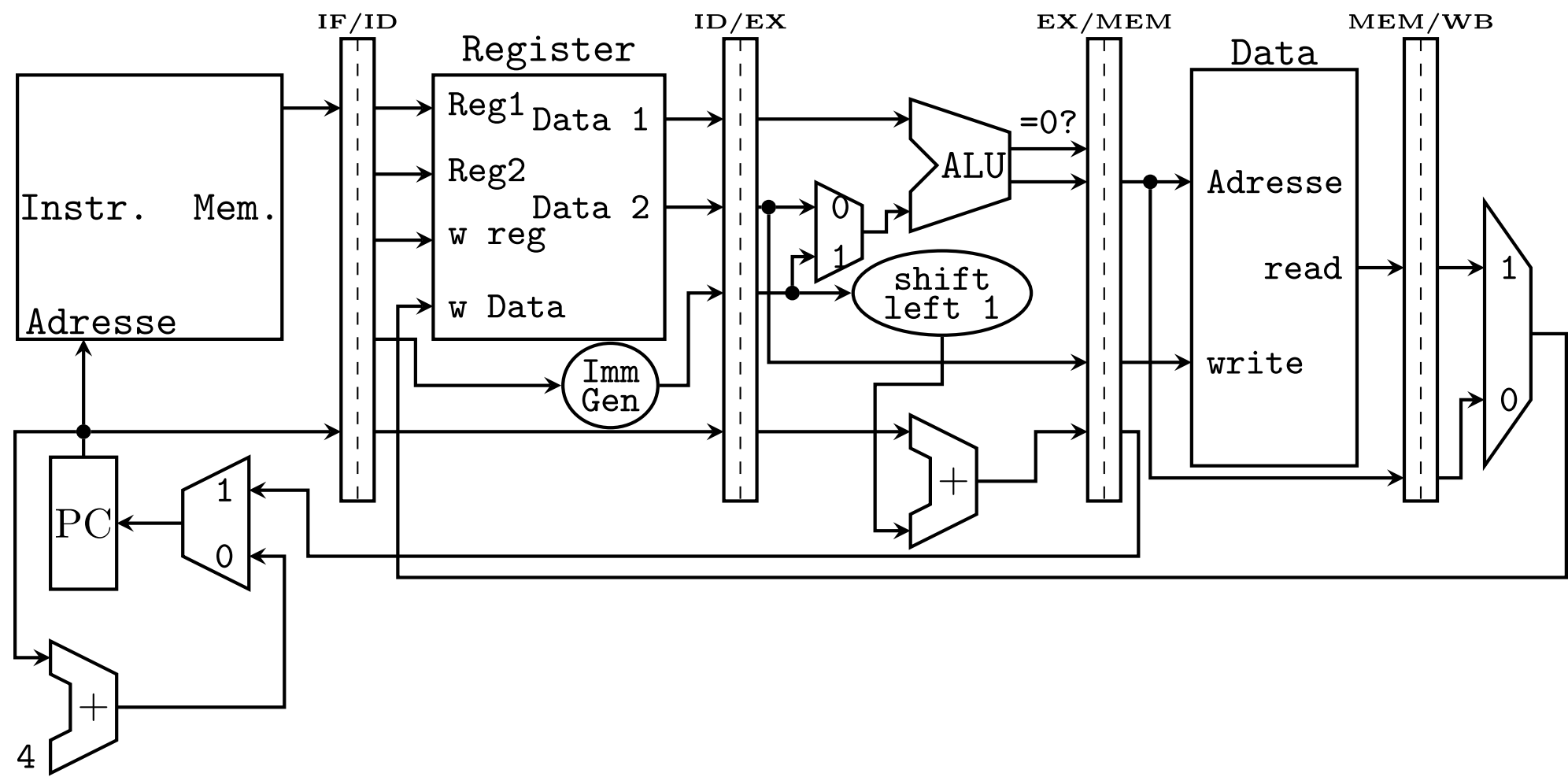
Idealer Fließbanddurchlauf



Zyklus 3

Befehl3
Befehl2
Befehl1

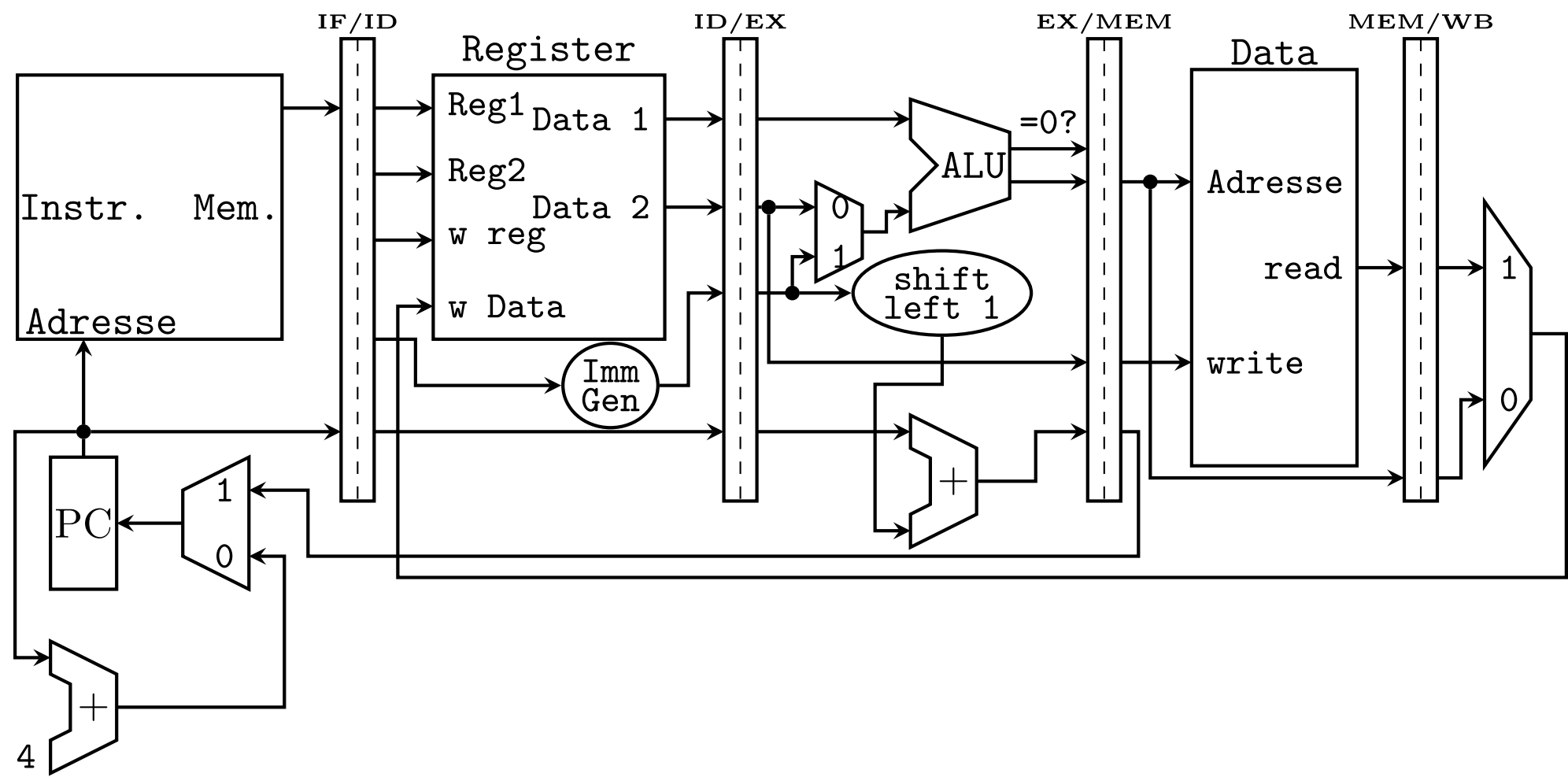
Idealer Fließbanddurchlauf



Zyklus 4

Befehl4
Befehl3
Befehl2
Befehl1

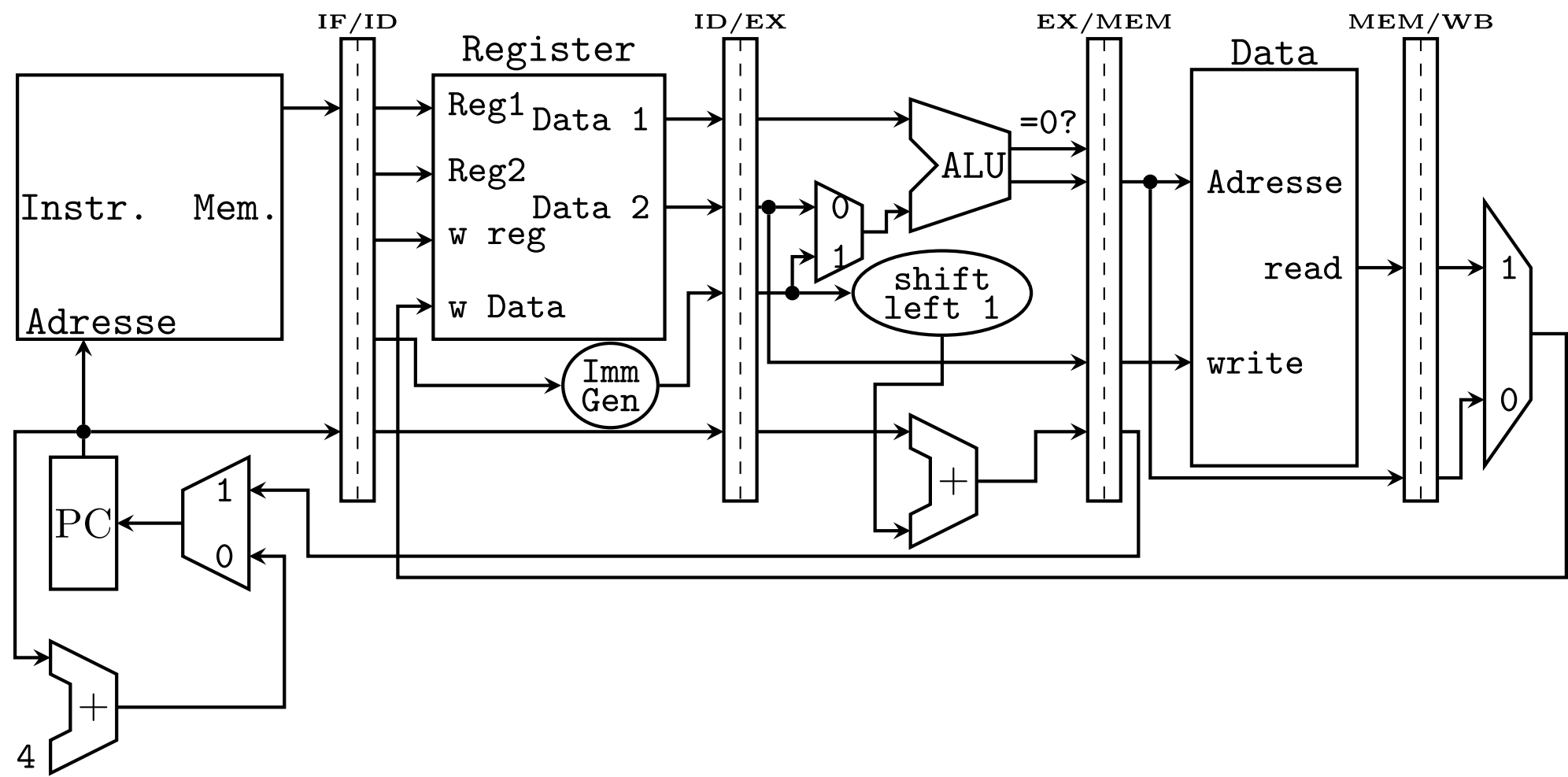
Idealer Fließbanddurchlauf



Zyklus 5

Befehl5
Befehl4
Befehl3
Befehl2
Befehl1

Idealer Fließbanddurchlauf



Zyklus 6

Befehl6
Befehl5
Befehl4
Befehl3
Befehl2

Pipeline-Hazards

Structural hazards

(deutsch: strukturelle Abhängigkeiten oder Gefährdungen).

Verschiedene Fließbandstufen müssen auf dieselbe Hardware-Komponente zugreifen, weil diese nur sehr aufwändig oder überhaupt nicht zu duplizieren ist.

Beispiele:

Speicherzugriffe, sofern für Daten und Befehle nicht über separate Pufferspeicher (caches) eine weitgehende Unabhängigkeit erreicht wird.

Bei Gleitkommaeinheiten lässt sich häufig nicht mit jedem Takt eine neue Operation starten (zu teuer).

→ Eventuell Anhalten des Fließbandes (pipeline stall) nötig.

Datenabhängigkeiten (1)

Gegeben sei eine Folge von Maschinen-Befehlen.

Ein Befehl j heißt von einem vorausgehenden Befehl i **datenabhängig**, wenn i Daten bereitstellt, die j benötigt.

Beispiel:

add **x12**, **x2**, **x3**

Die letzten 4 Befehle sind vom **add**-Befehl wegen **x12** datenabhängig

sub **x4**, **x5**, **x12**

and **x6**, **x12**, **x12**

or x12, x12, x9

xor x10, x12, x11

Diese Art der Abhängigkeit heißt (bei Hennessy und anderen) *read after write*- (oder RAW-) Abhängigkeit

Datenabhängigkeiten (2)

Gegeben sei wieder eine Folge von Maschinen-Befehlen.

Ein Befehl i heißt von einem **nachfolgenden** Befehl j **antidatenabhängig**, falls j eine Speicherzelle beschreibt, die von i noch gelesen werden müsste.

Beispiel:

add **x12**, **x2**, **x3**

Die Befehle **sub** und **and** sind vom **or**-Befehl wegen **x12** antidatenabhängig

sub **x4**, **x5**, **x12**

and **x6**, **x12**, **x12**

or x12, x12, x9

xor x10, x12, x11

Diese Art der Abhängigkeit heißt (bei Hennessy und anderen) *write after read*- (oder WAR-) Abhängigkeit

Datenabhängigkeiten (3)

Gegeben sei wieder eine Folge von Maschinen-Befehlen.

Zwei Befehle i und j heißen voneinander **Ausgabe-abhängig**, falls i und j dieselbe Speicherzelle beschreiben

Beispiel:

add **x12**, **x2**, **x3**

Die Befehle **add** und **or** sind wegen **x12** voneinander ausgabeabhängig

sub **x4**, **x5**, **x12**

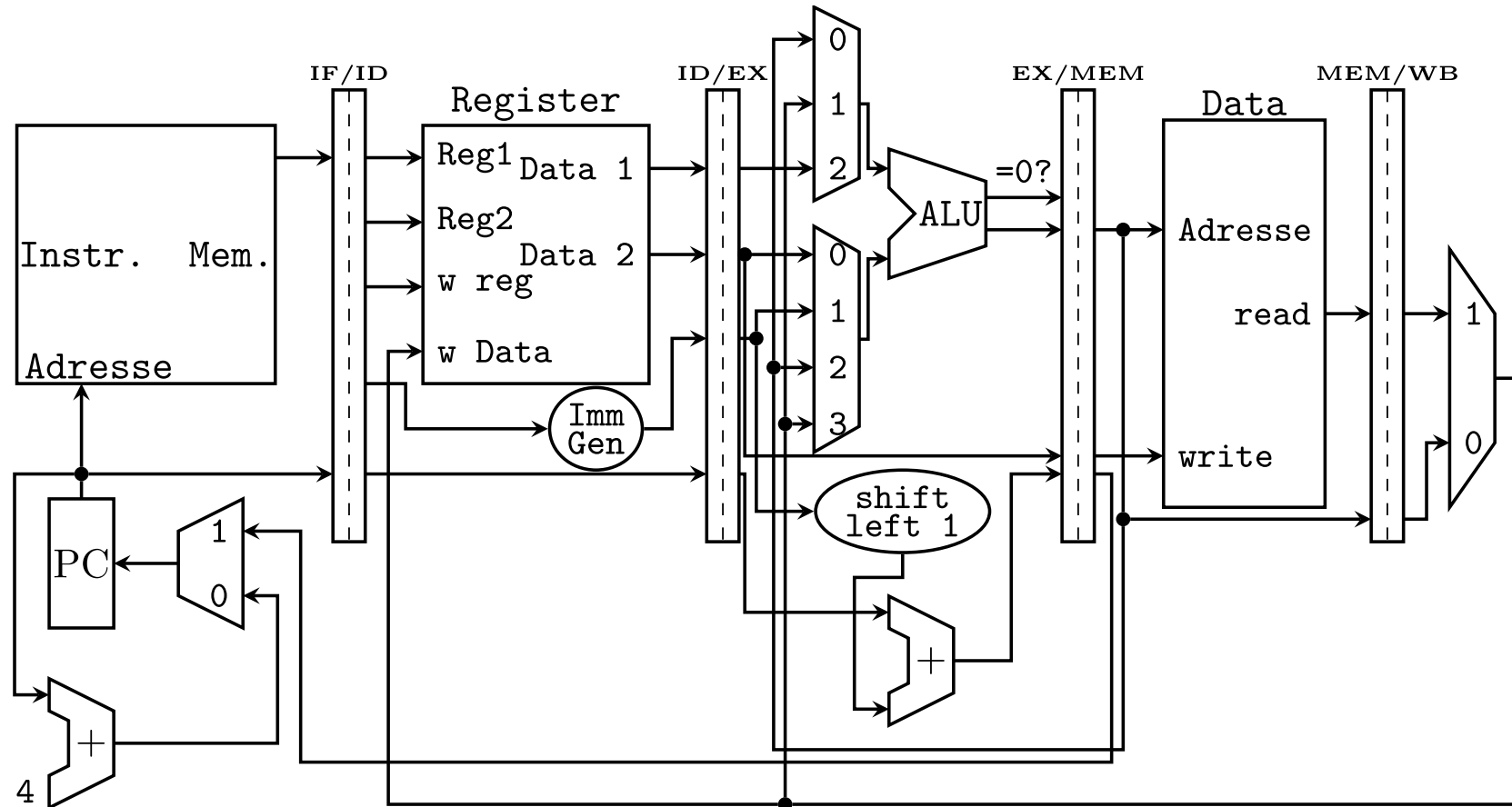
and **x6**, **x12**, **x12**

or **x12**, **x12**, **x9**

xor x10, x12, x11

Diese Art der Abhängigkeit heißt (bei Hennessy und anderen) *write after write-* (oder WAW-) Abhängigkeit

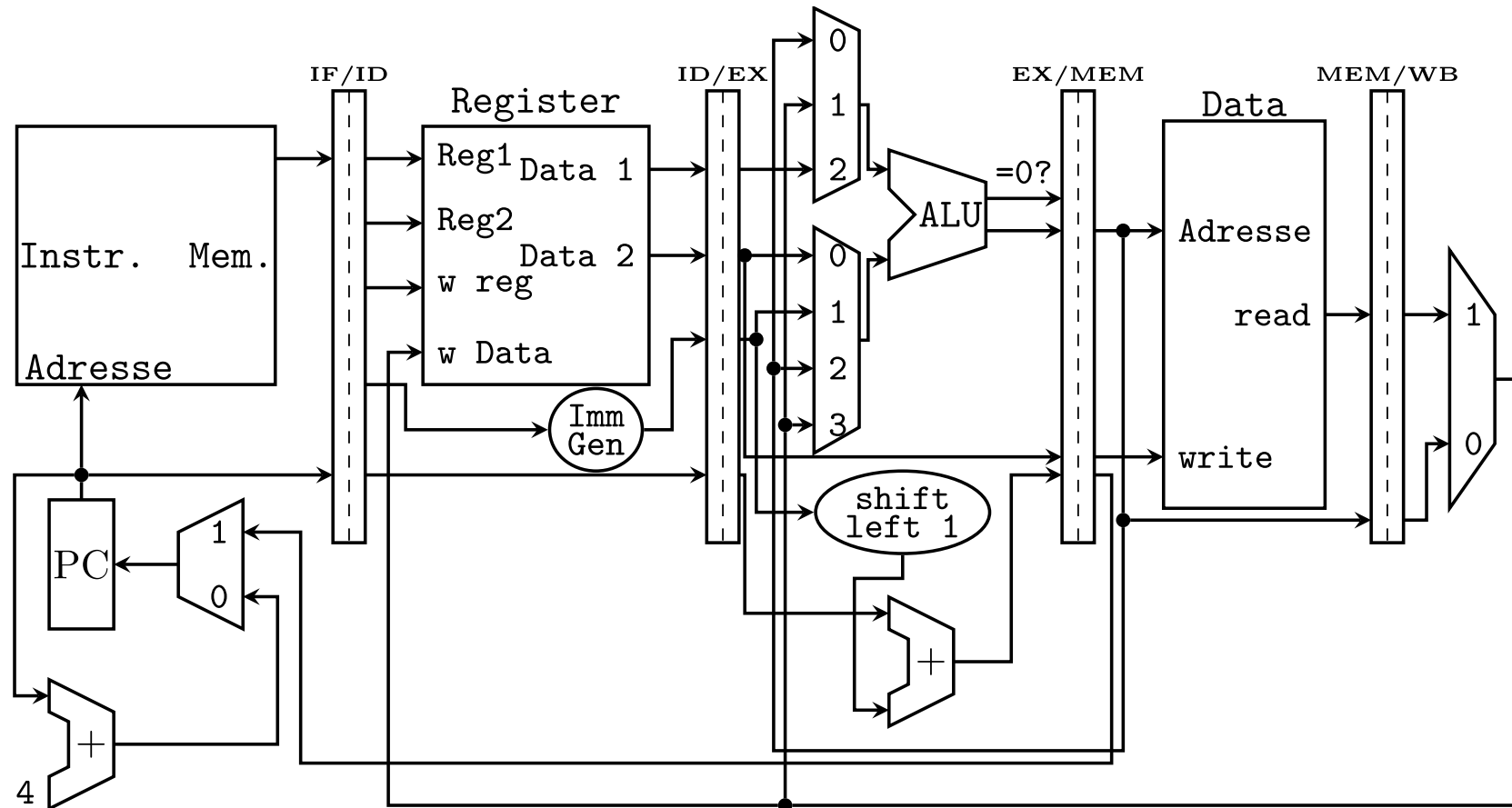
Bypässe, forwarding: Behandlung des *data hazards* bei and und sub



Zyklus 1

add x1,x2,x3

Bypässe, forwarding: Behandlung des *data hazards* bei and und sub

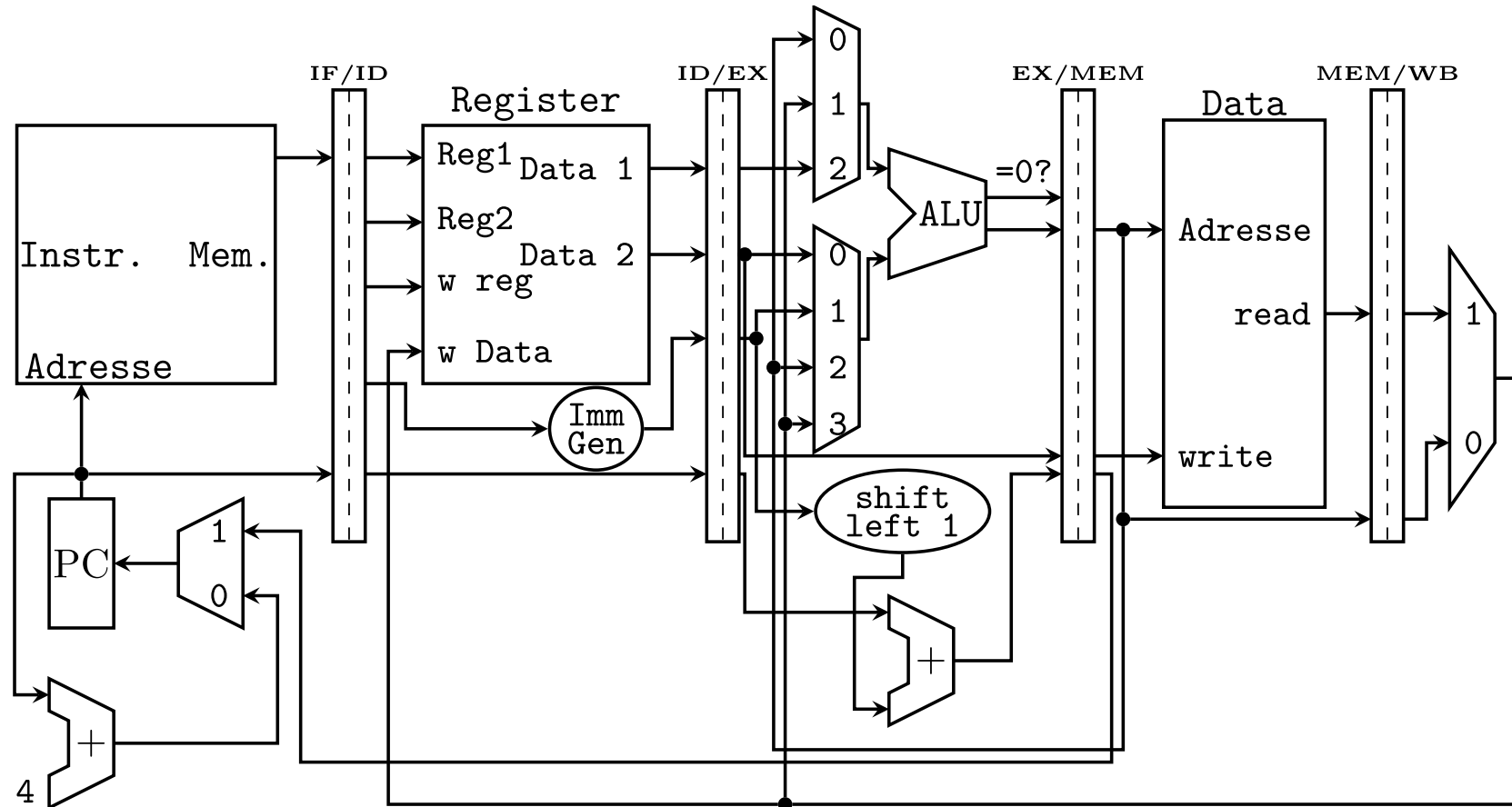


Zyklus 2

sub x4,x5,x1

add x1,x2,x3

Bypässe, forwarding: Behandlung des *data hazards* bei and und sub



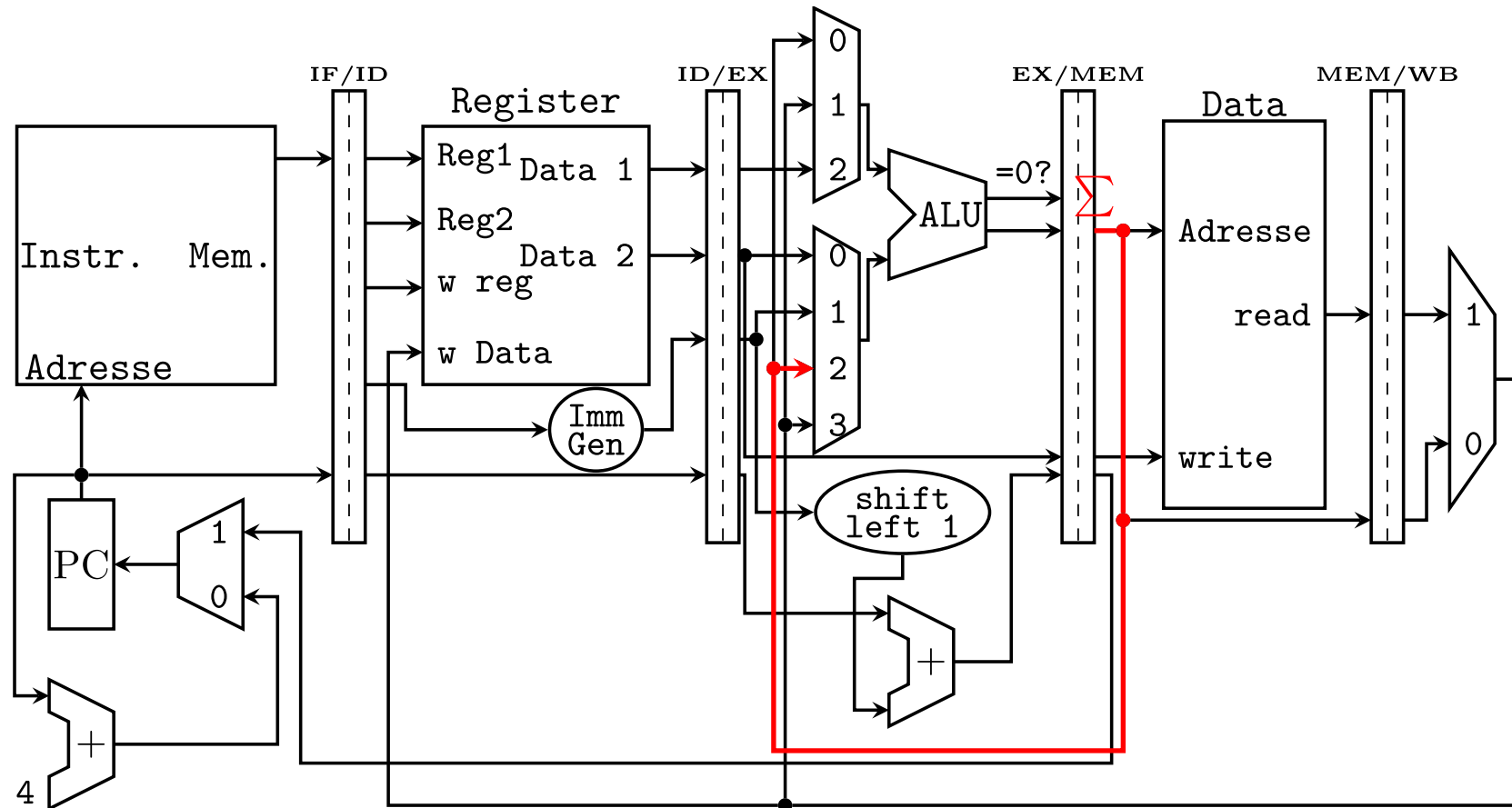
Zyklus 3

and x6,x1,x7

sub x4,x5,x1

add x1,x2,x3

Bypässe, forwarding: Behandlung des *data hazards* bei and und sub



Zyklus 4

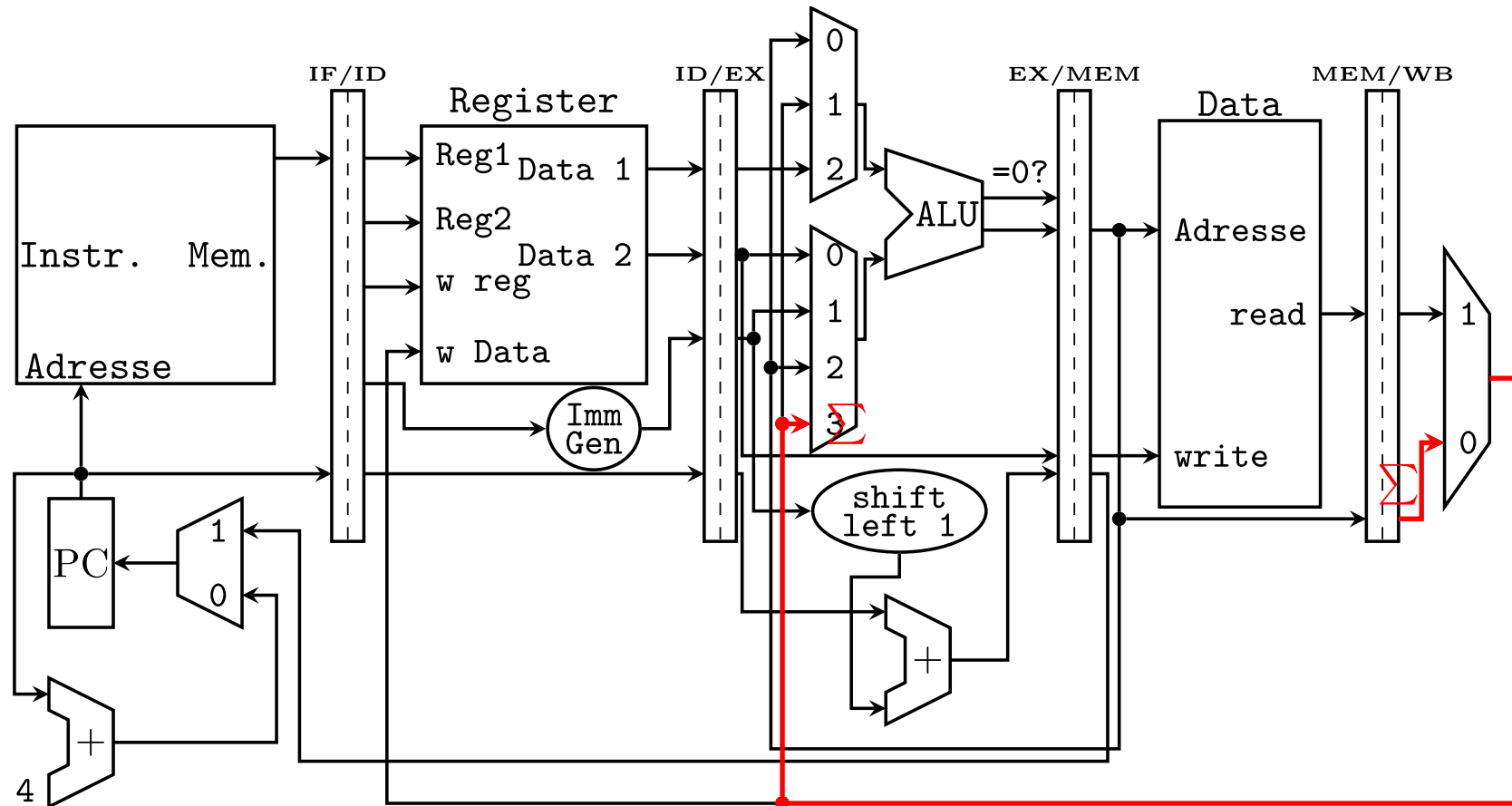
or x8,x1,x9

and x6,x1,x7

sub x4,x5,x1

add x1,x2,x3

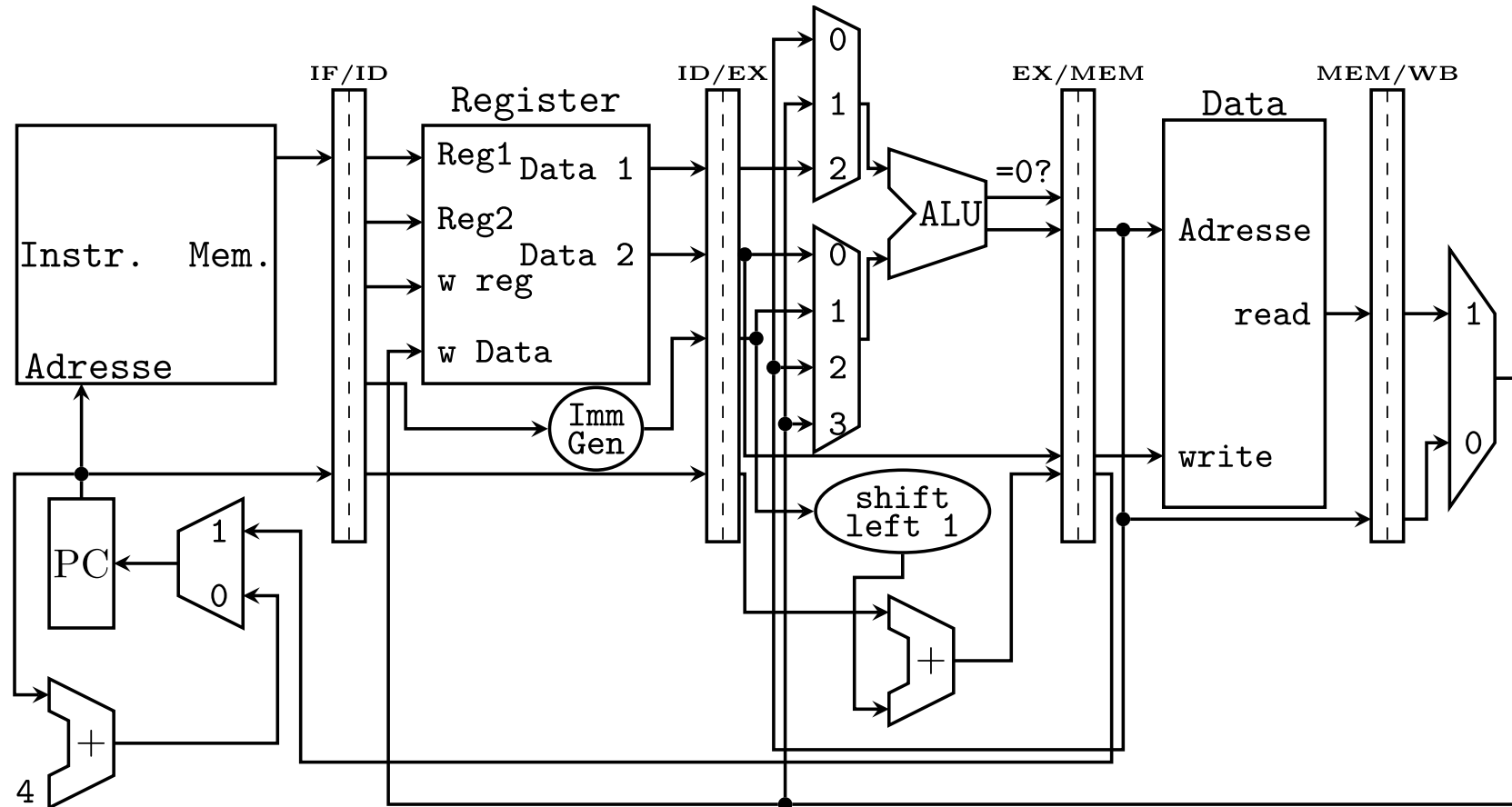
Bypässe, *forwarding*: Behandlung des *data hazards* bei and und sub



Zyklus 5

```
xor
x10, x1, x11
or  x8, x1, x9
and x6, x1, x7
sub x4, x5, x1
add x1, x2, x3
```

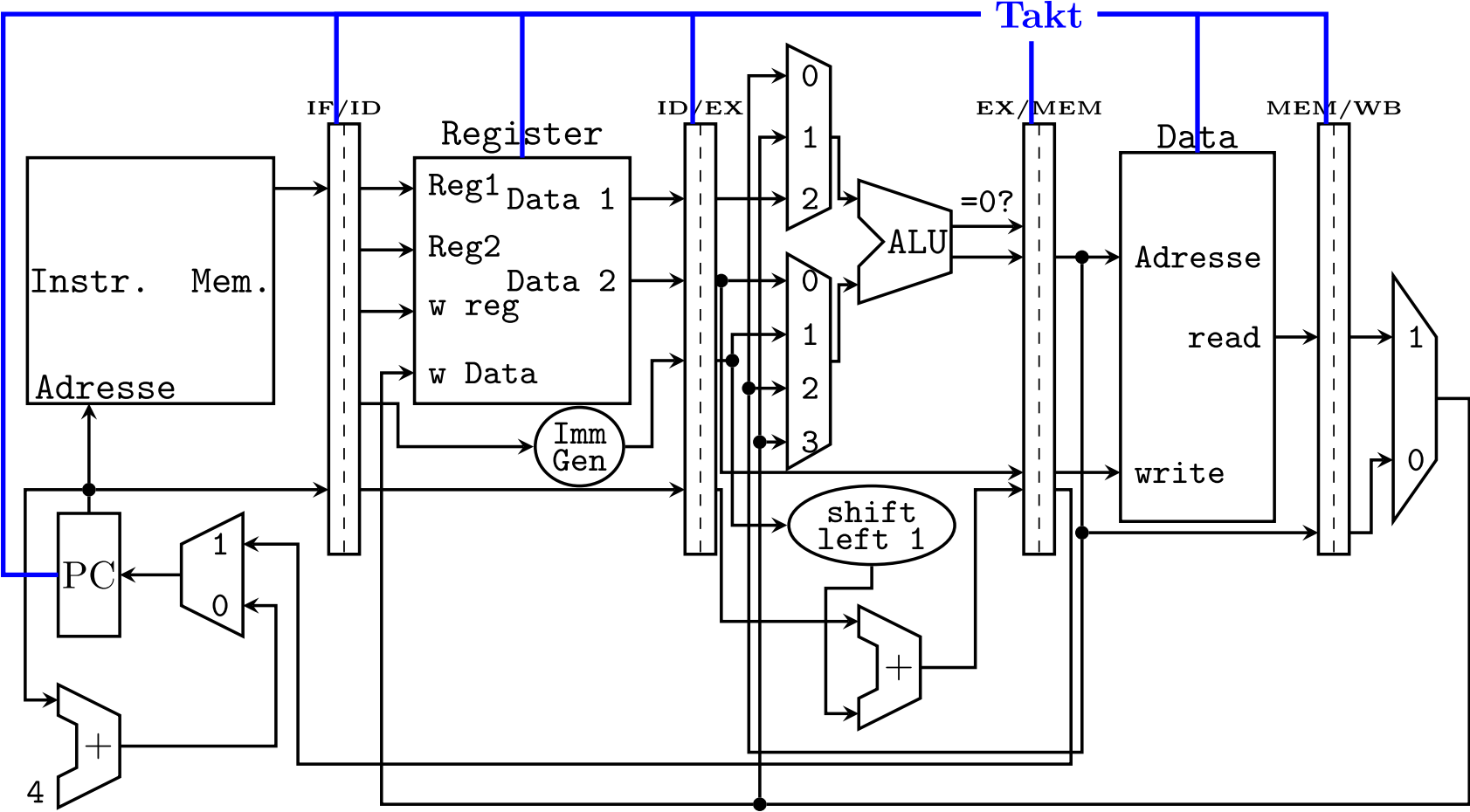
Bypässe, forwarding: Behandlung des *data hazards* bei and und sub



Zyklus 6

?
xor x10,x1,x11
or x8,x1,x9
and x6,x1,x7
sub x4,x5,x1

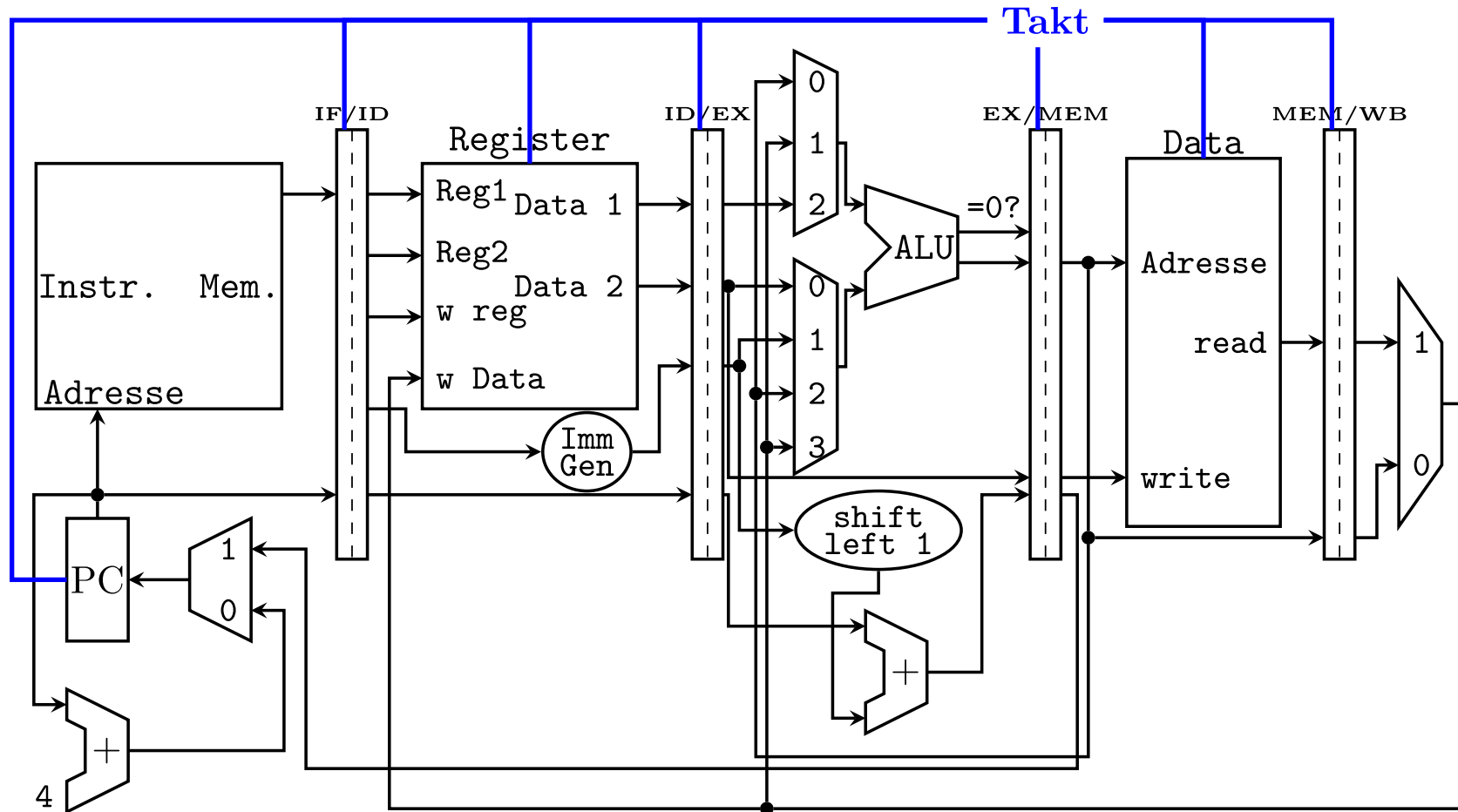
Taktung zur Behandlung des data hazards bei or



Zyklus 1

add x1,x2,x3

Taktung zur Behandlung des data hazards bei or

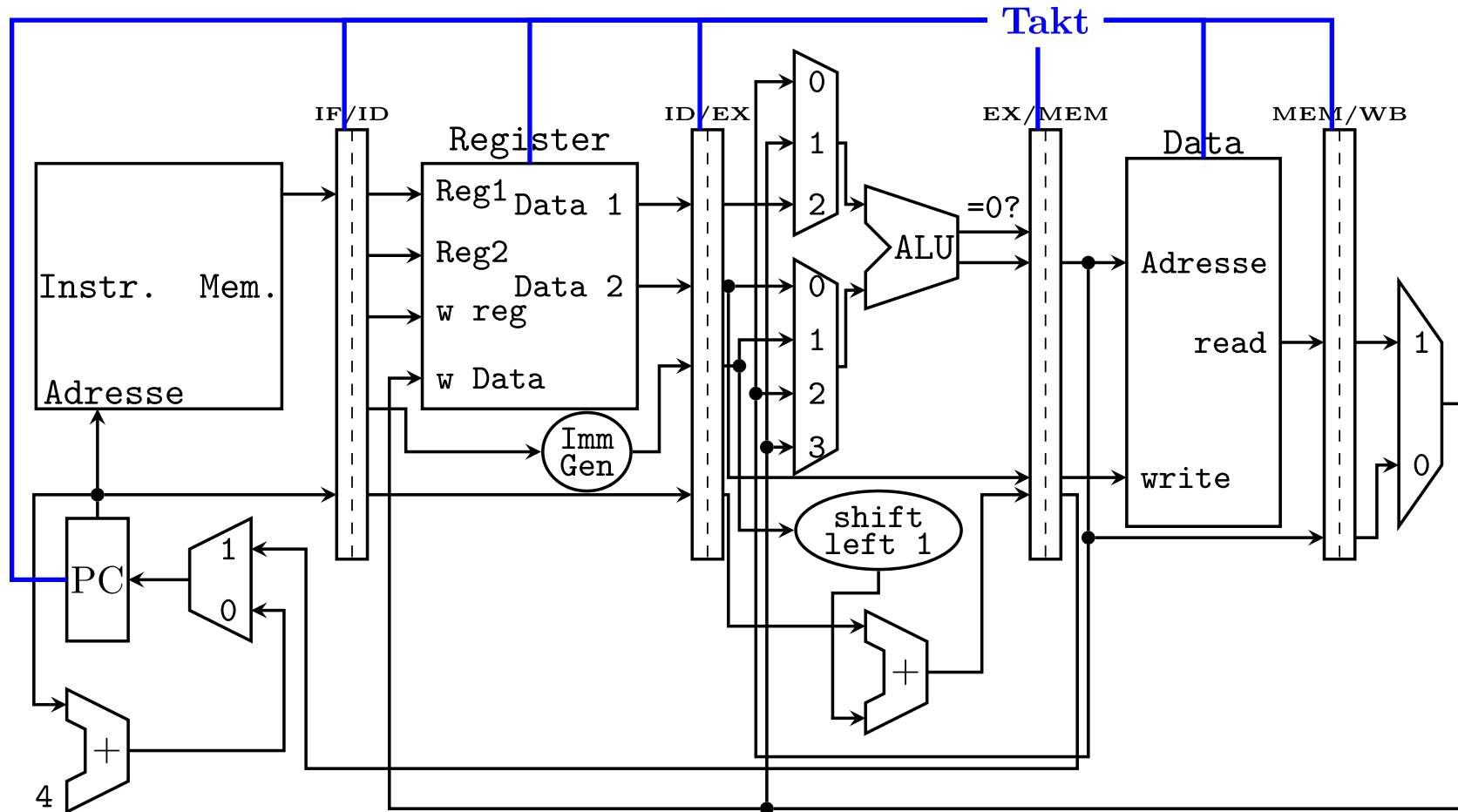


Zyklus 2

sub x4,x5,x1

add x1,x2,x3

Taktung zur Behandlung des data hazards bei or



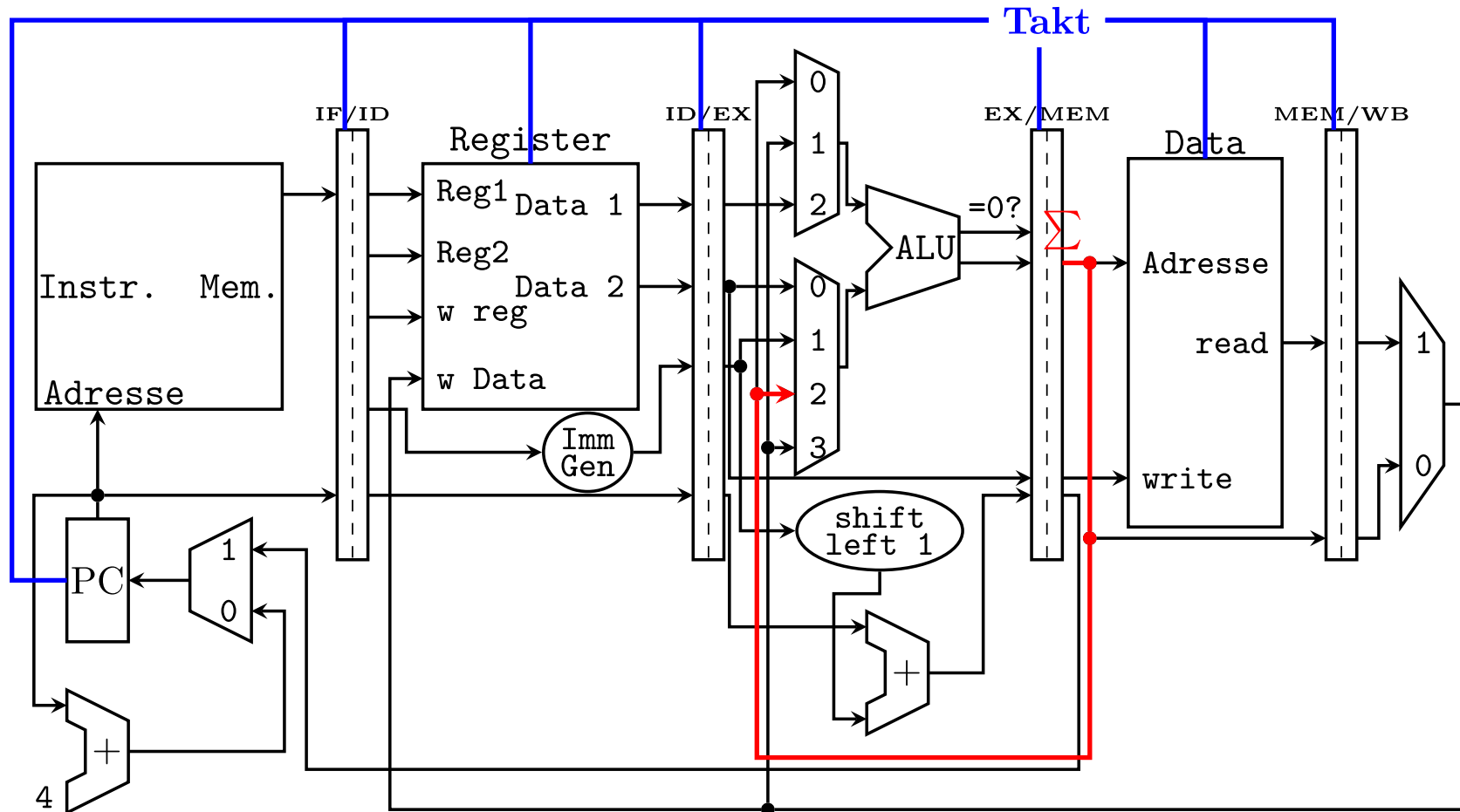
Zyklus 3

and x6,x1,x7

sub x4,x5,x1

add x1,x2,x3

Taktung zur Behandlung des data hazards bei or



Zyklus 4

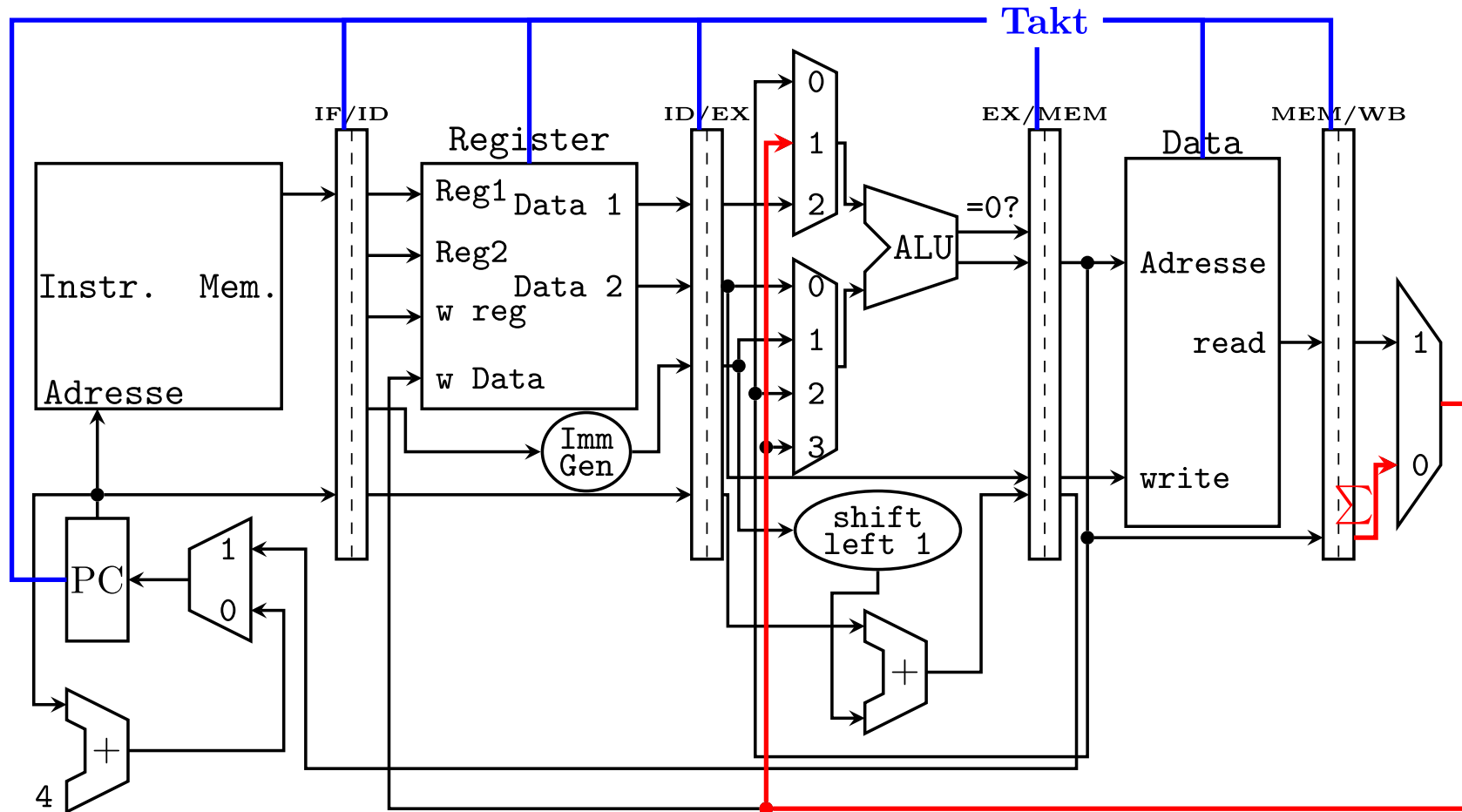
or x8,x1,x9

and x6,x1,x7

sub x4,x5,x1

add x1,x2,x3

Taktung zur Behandlung des data hazards bei or



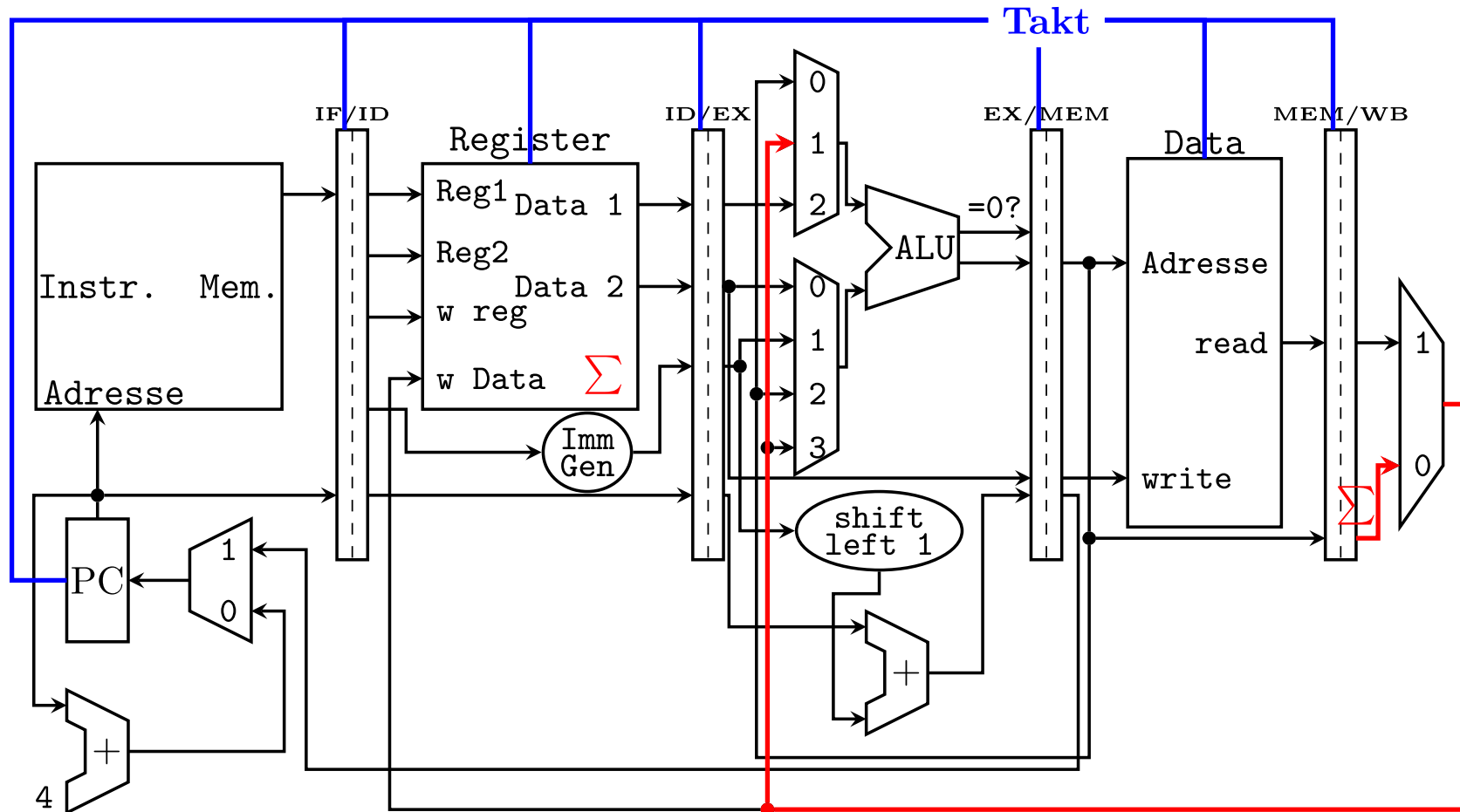
Zyklus 5

xor x10,x1,x11
or x8,x1,x9
and x6,x1,x7
sub x4,x5,x1
add x1,x2,x3

- Hat der `or` Befehl bereits den richtigen Wert erhalten?

- Idee: Zyklus aufteilen
- Puffer werden mit positiver Flanke getaktet, andere Elemente mit negativer

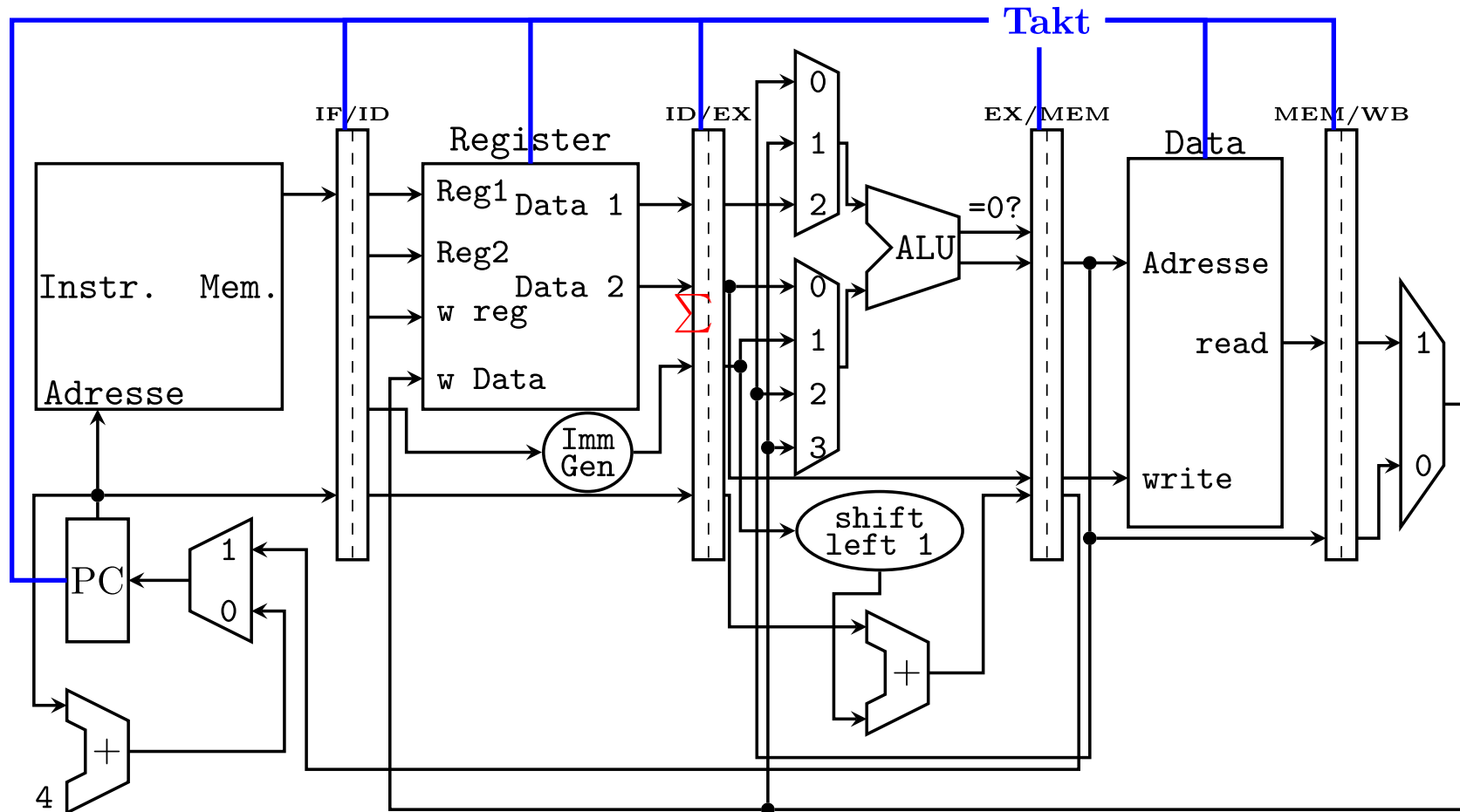
Taktung zur Behandlung des data hazards bei or



Zyklus 5

xor x10,x1,x11
or x8,x1,x9
and x6,x1,x7
sub x4,x5,x1
add x1,x2,x3

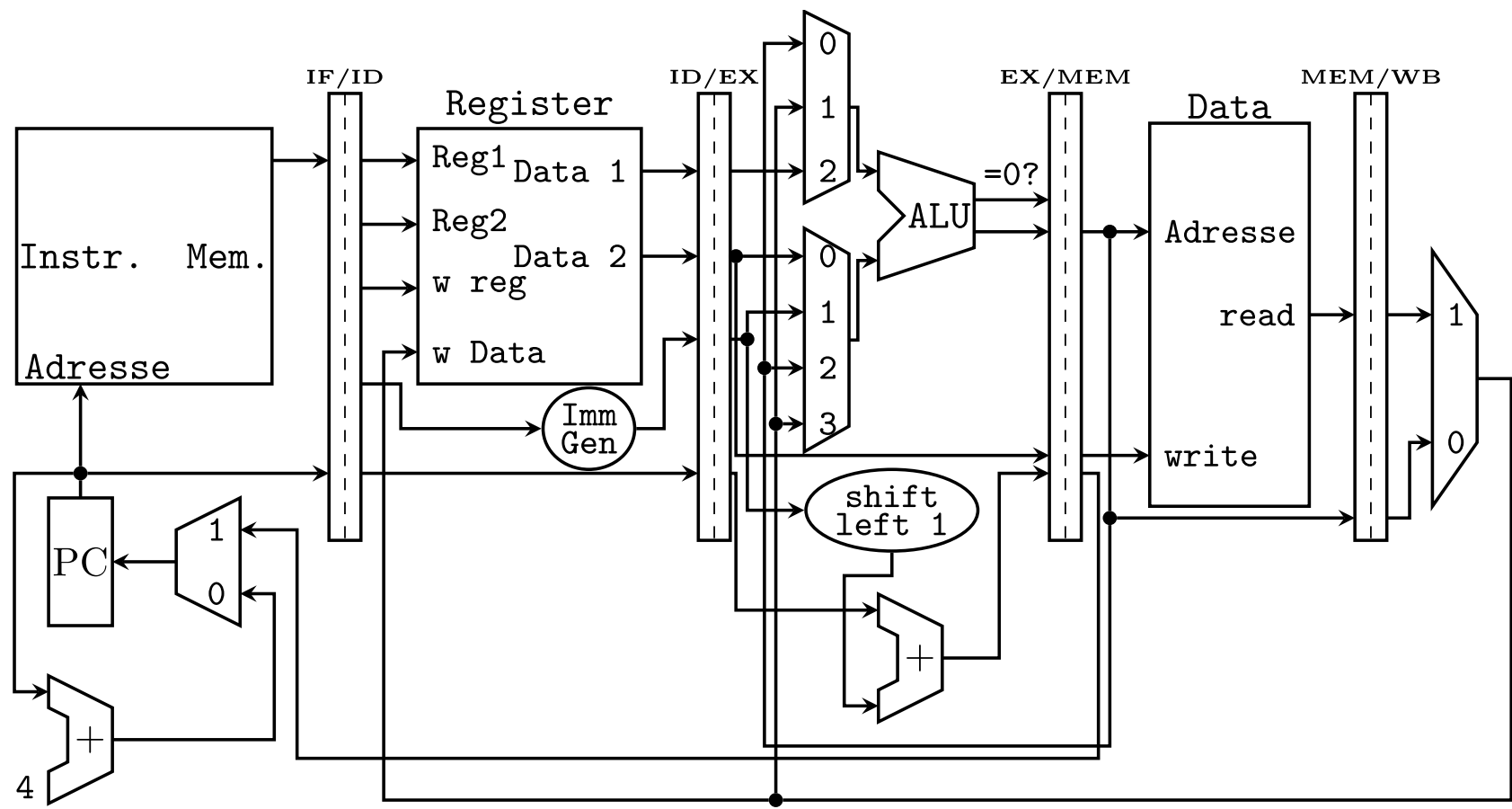
Taktung zur Behandlung des data hazards bei or



Zyklus 6

?
xor x10,x1,x11
or x8,x1,x9
and x6,x1,x7
sub x4,x5,x1

Alle data hazards durch Bypässe behandelbar?

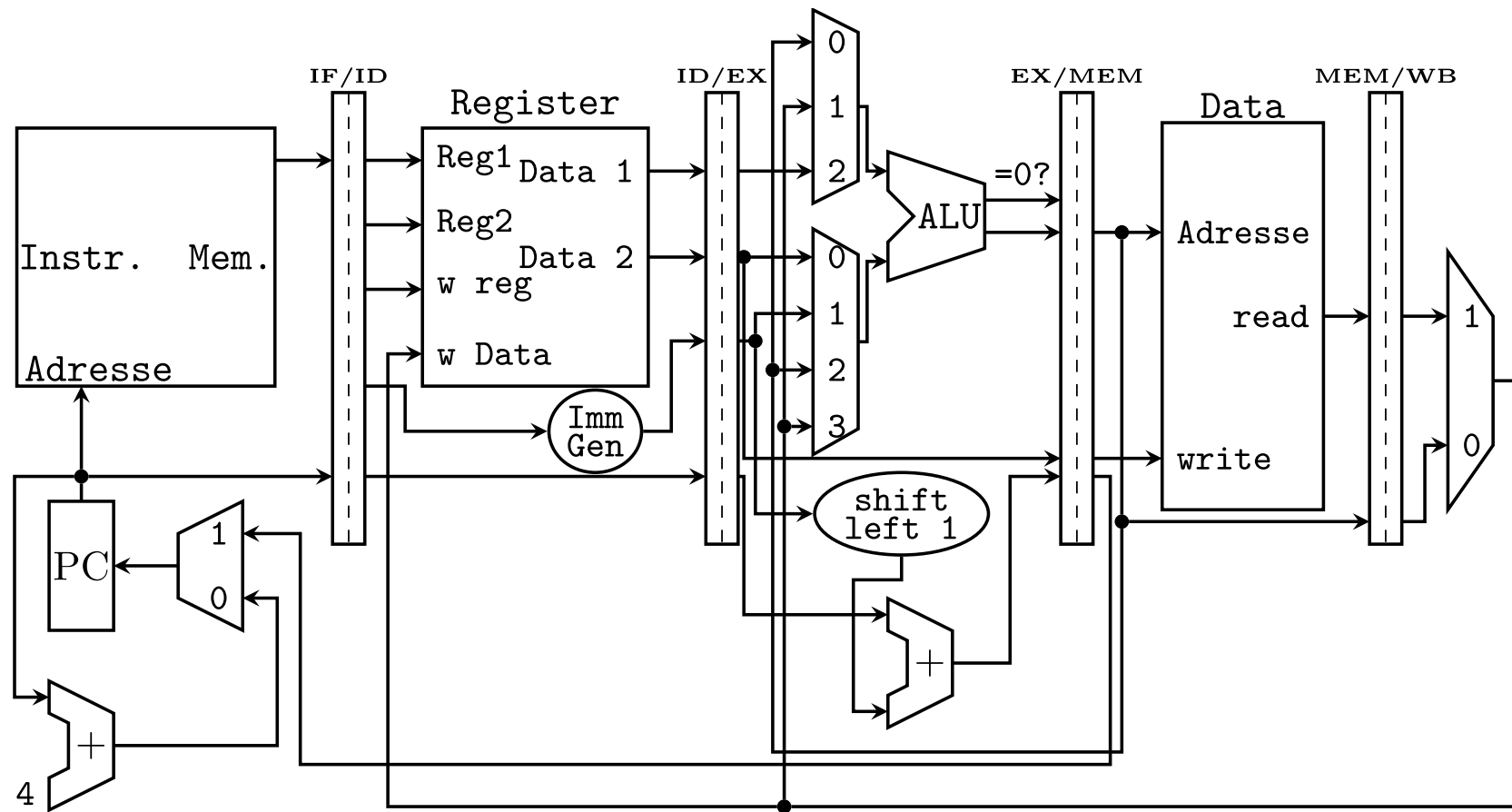


Zyklus 6

?
xor x10,x1,x11
or x8,x1,x9
and x6,x1,x7
sub x4,x5,x1

Speicherwort wird am Ende des Zyklus 4 gespeichert, steht für Differenz noch nicht bereit

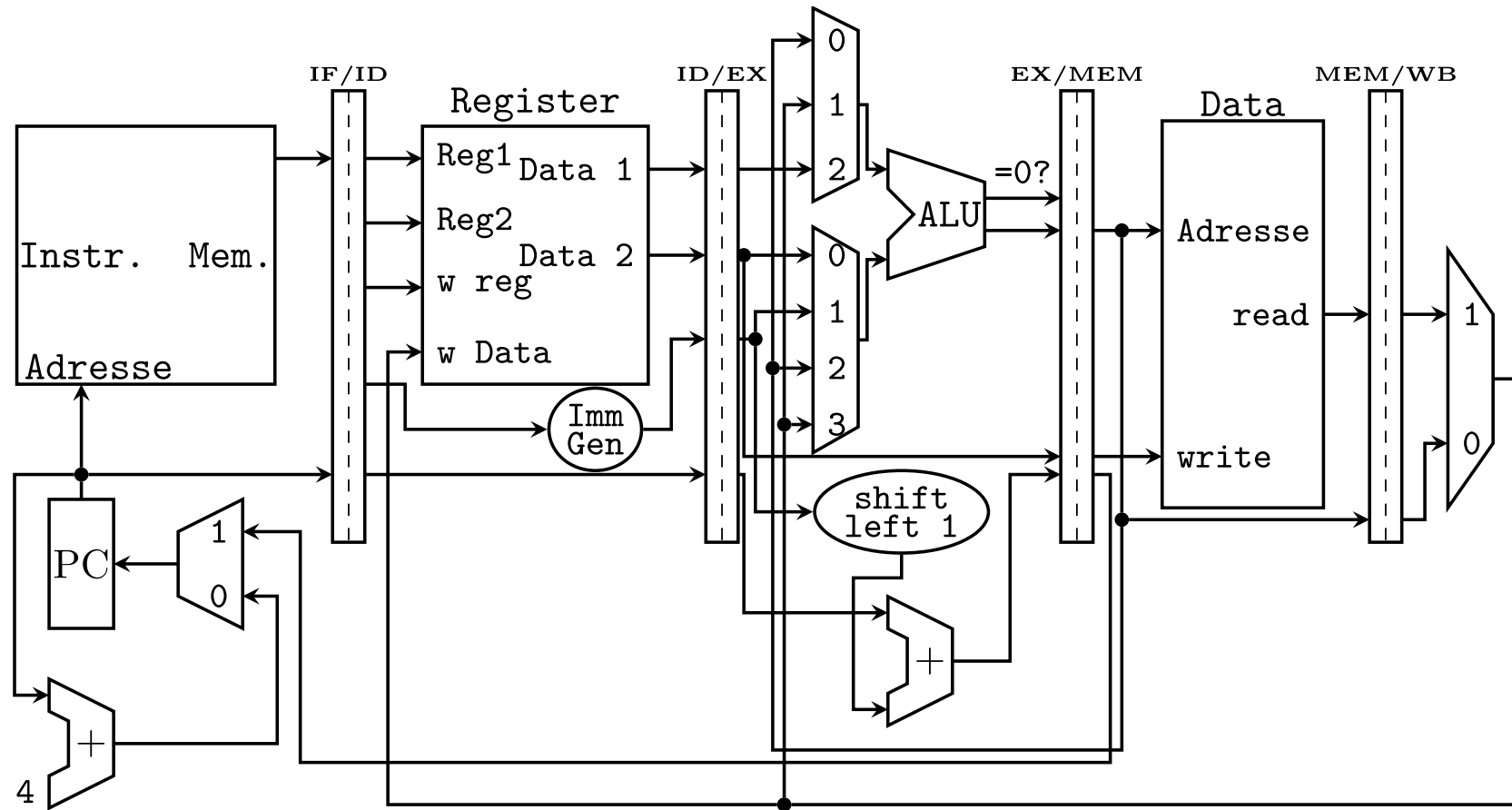
Lösung durch Anhalten des Fließbandes (pipeline stall, hardware interlocking, bubbles)



Zyklus 1

lw x1,0(x2)

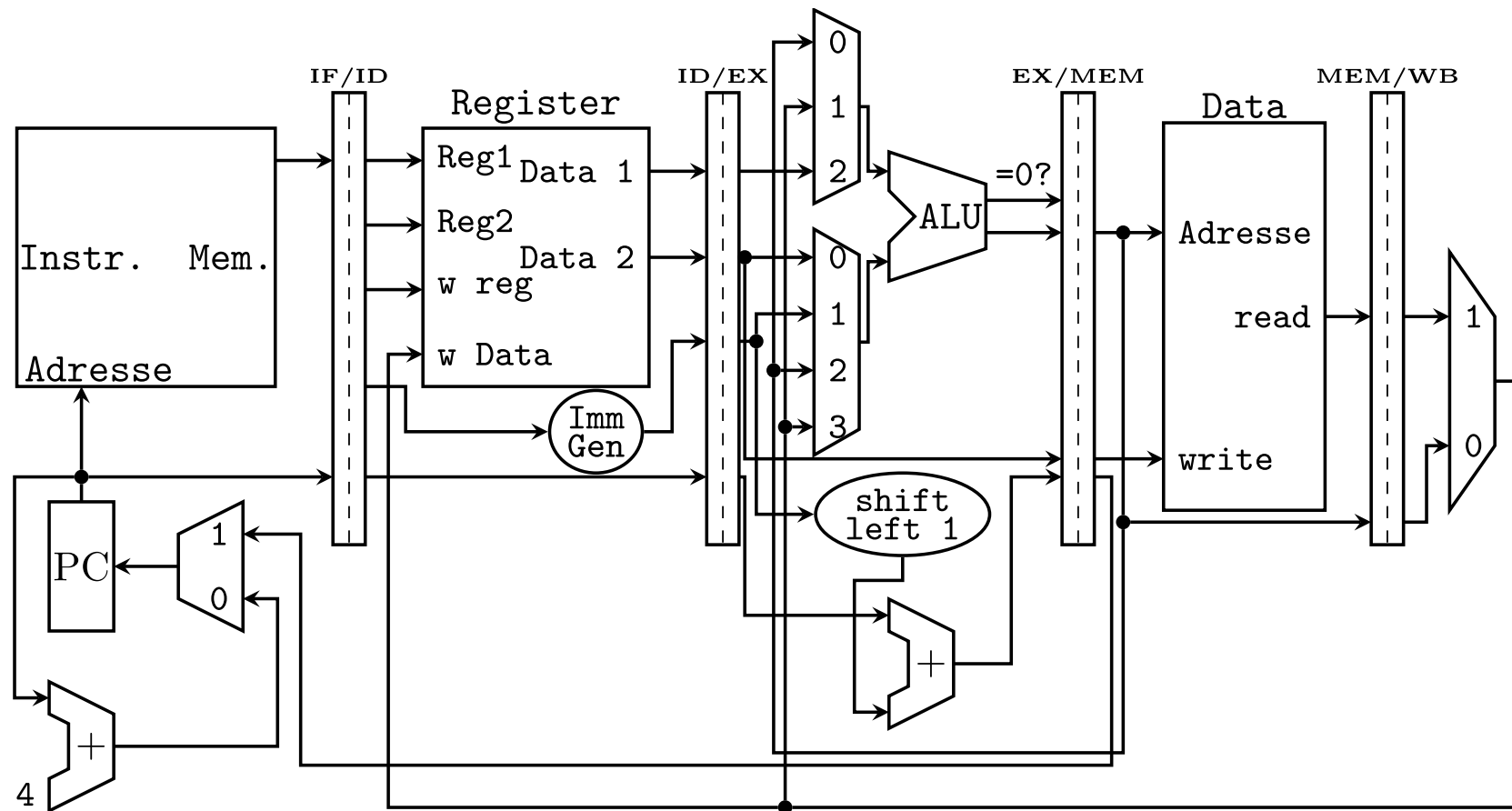
Lösung durch Anhalten des Fließbandes (pipeline stall, hardware interlocking, bubbles)



Zyklus 2

<code>sub x4,x5,x1</code>
<code>lw x1,0(x2)</code>

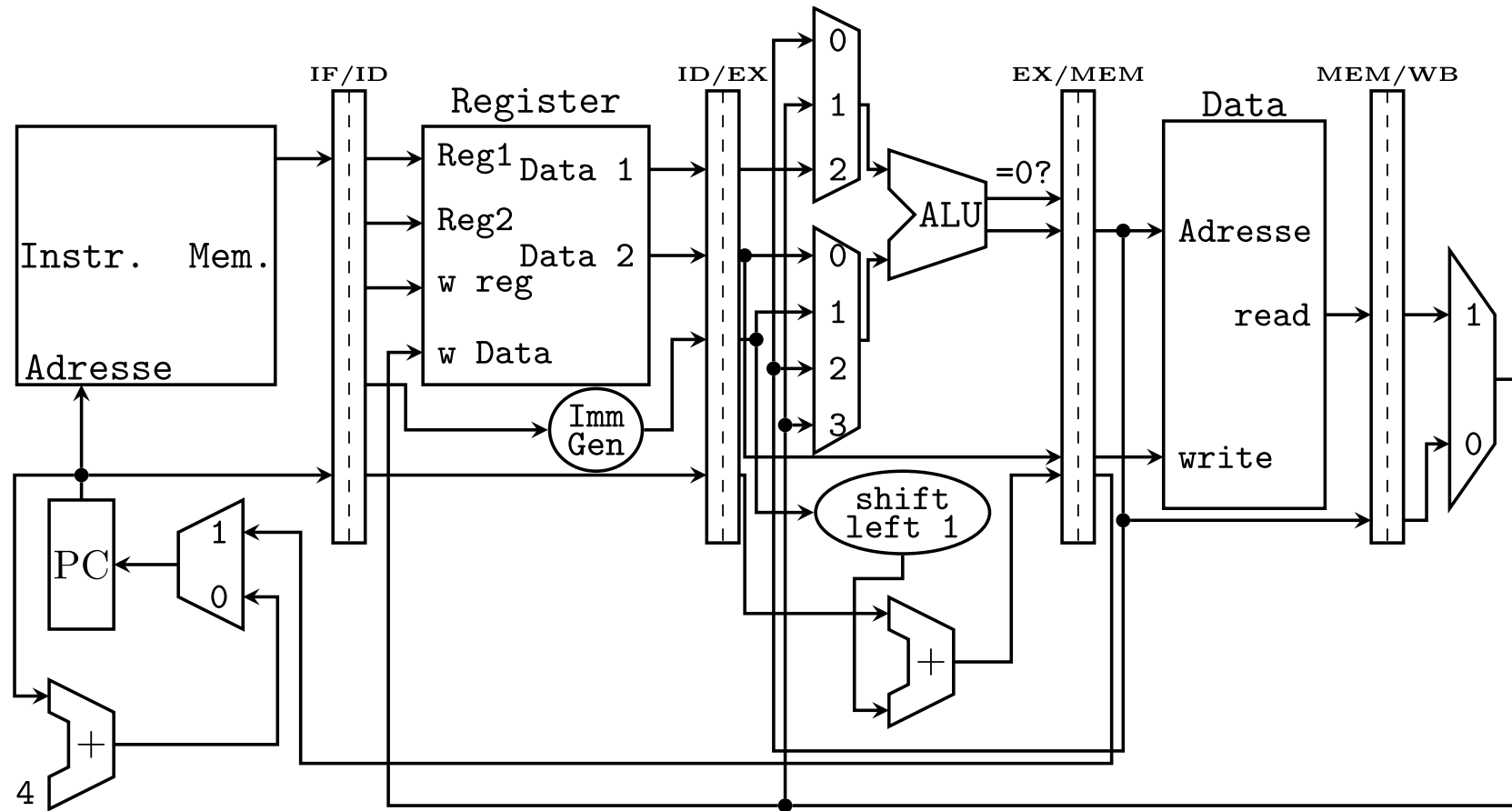
Lösung durch Anhalten des Fließbandes (pipeline stall, hardware interlocking, bubbles)



Zyklus 3

and x6,x1,x7
sub x4,x5,x1
lw x1,0(x2)

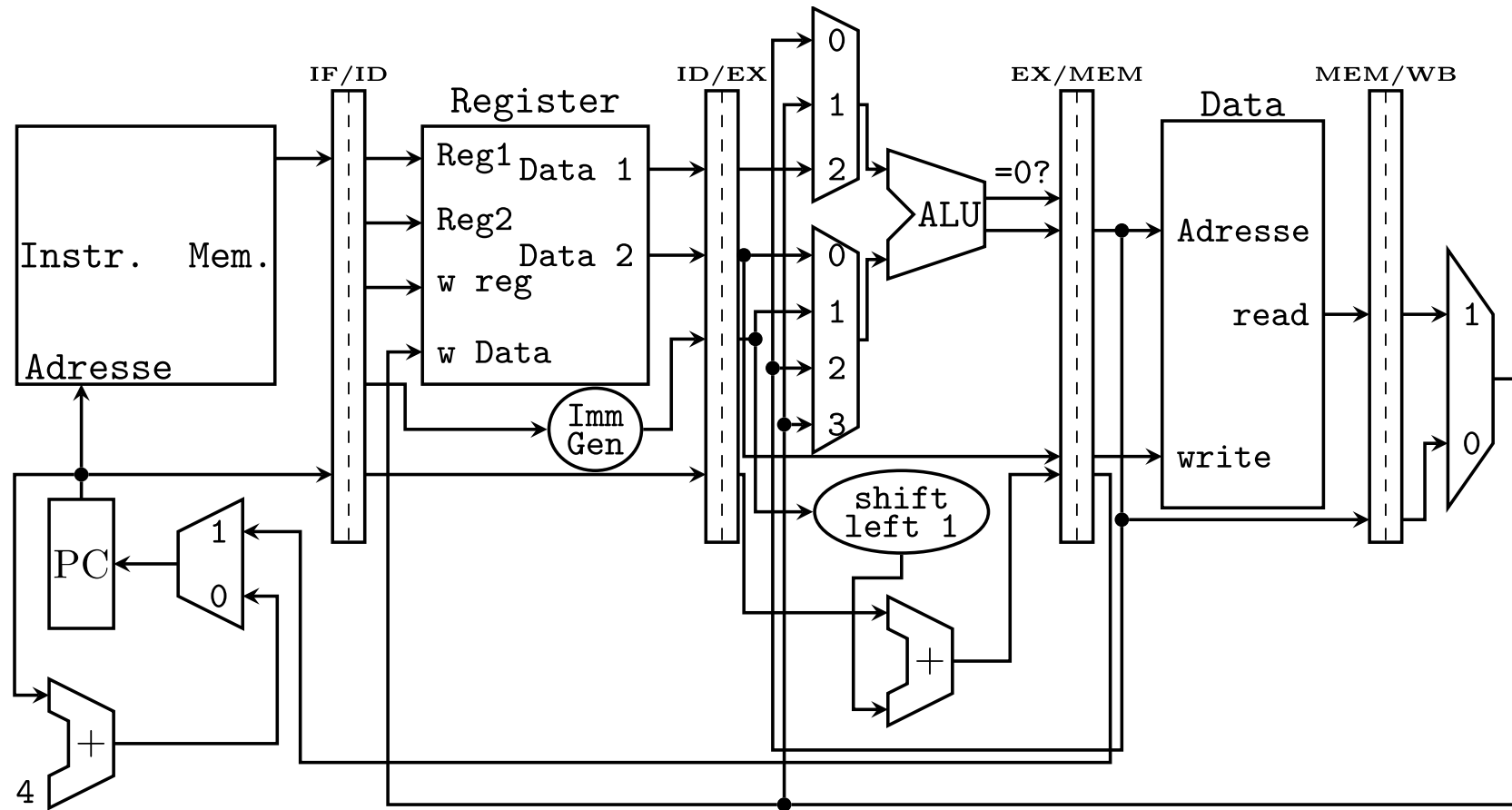
Lösung durch Anhalten des Fließbandes (pipeline stall, hardware interlocking, bubbles)



Zyklus 4

NOOP
NOOP
NOOP
lw x1,0(x2)

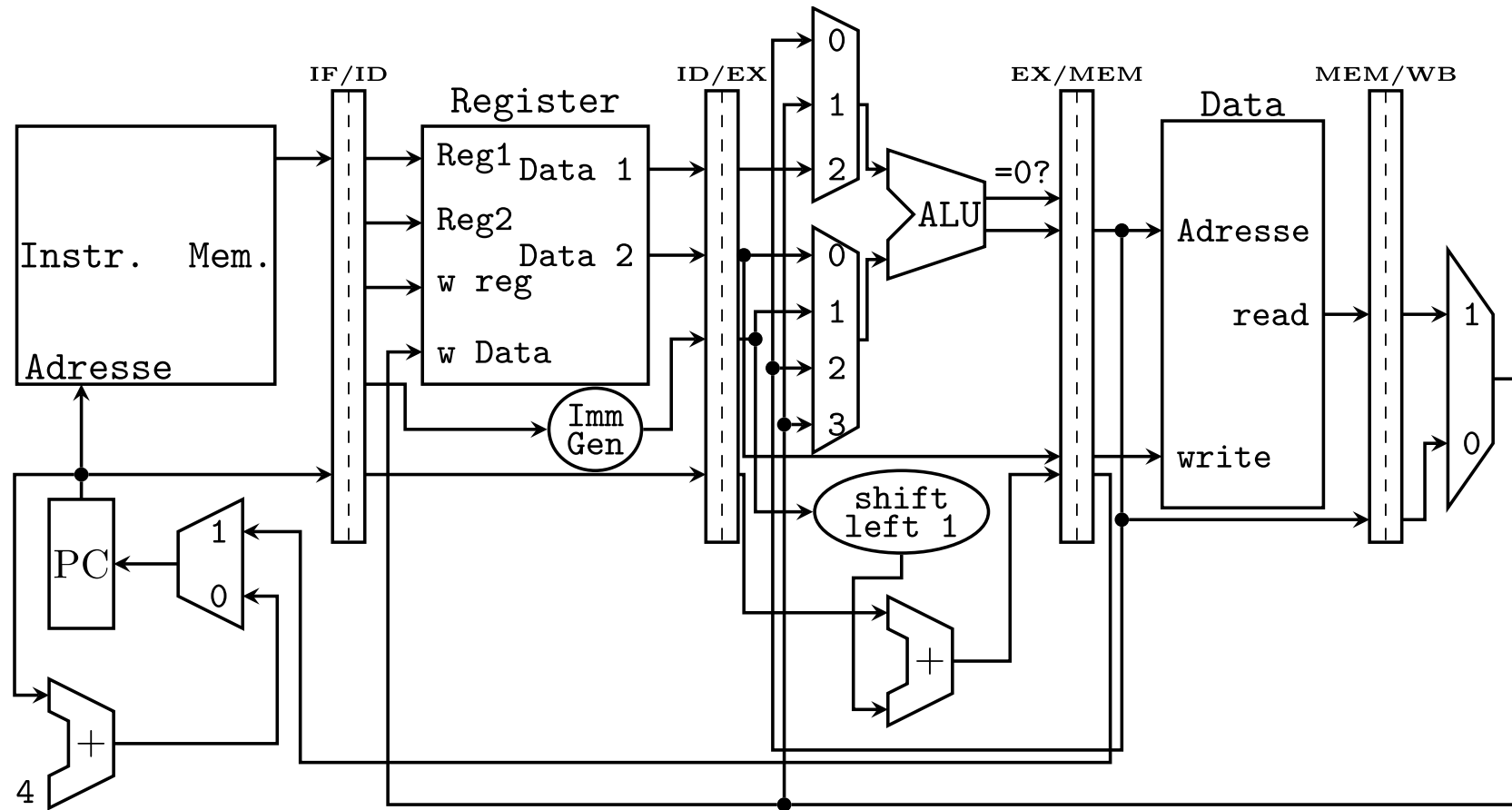
Lösung durch Anhalten des Fließbandes (pipeline stall, hardware interlocking, bubbles)



Zyklus 5

or x8,x1,x9
and x6,x1,x7
sub x4,x5,x1
NOOP
lw x1,0(x2)

Lösung durch Anhalten des Fließbandes (pipeline stall, hardware interlocking, bubbles)



Zyklus 6

xor x10,x1,x11

or x8,x1,x9

and x6,x1,x7

sub x4,x5,x1

NOOP

↑↑ “bubble”, durch
intelligente Compiler
vermeiden!

Kontrollfluss-Abhängigkeiten oder - Gefährdungen, control hazards (1)

Beispielprogramm:

```
beq  x12,  x8,  t  --springe zur Marke t, falls Reg[12]=Reg[8]
```

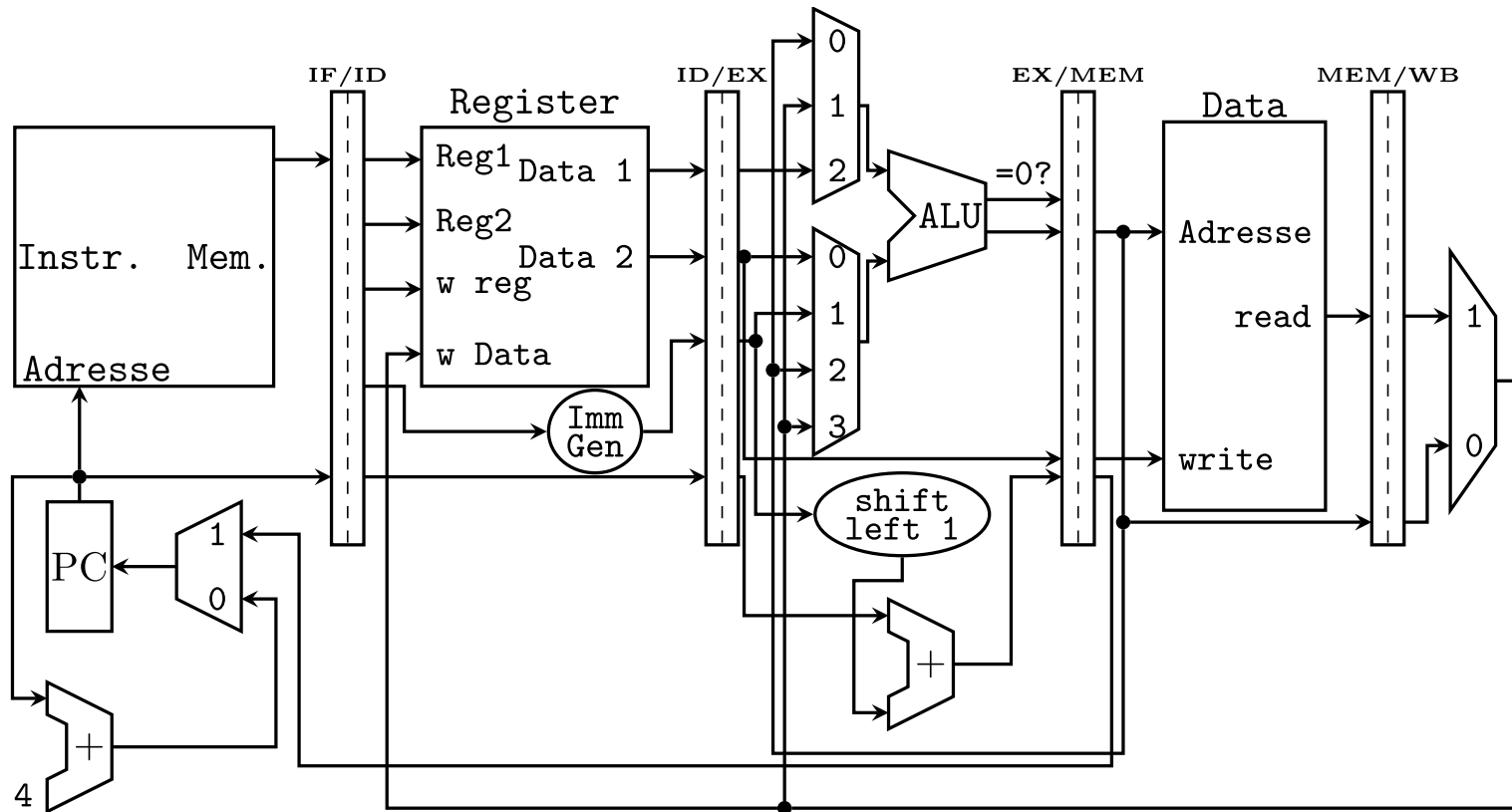
```
sub  ...
```

```
...
```

```
t:  add  ...
```

Wie versuchen zunächst, durch Einfügen von NOOPs, die intuitive Bedeutung des Programms zu realisieren...

Lösung durch Anhalten des Fließbandes (pipeline stall, hardware interlocking, bubbles)



Zyklus 6

...
...
sub oder add
NOOP
NOOP

- Takt 2: Sprung wird erkannt, deshalb werden zwei NOOP-Befehle eingefügt

- Takt 4: Mit fallender Flanke wird PC getaktet, mit steigender IF/ID-Register

Kontrollfluss-Abhängigkeiten oder - Gefährdungen, control hazards (3)

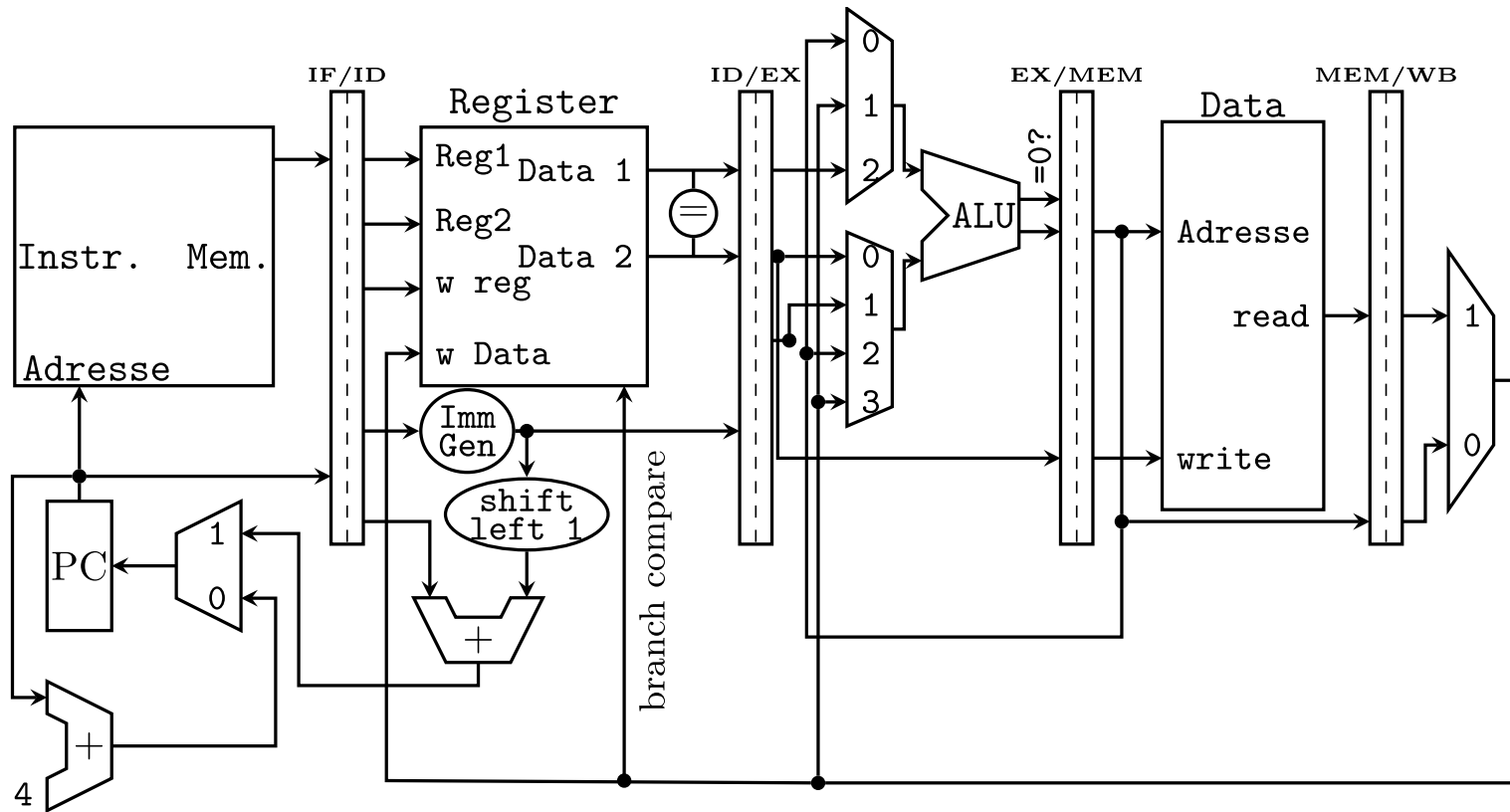
Probleme beim gezeigten Ansatz:

- Leistungsverlust durch 2 NOOPs (branch delay penalty).
- ALU/Multiplexer in der gezeigten Form nicht ausreichend, um Test und Sprungzielberechnung in einem Takt auszuführen.

Lösungsansatz:

- Gleichheit der Register wird schon in der instruction decode-Stufe geprüft.
- Sprungziel wird in separatem Adressaddierer ebenfalls bereits in der instruction decode-Stufe berechnet.
- Sofern weiterhin noch Verzögerungen auftreten:
 - nächsten Befehl einfach ausführen (delayed branch).
 - oder weiterhin NOOP(s) einfügen (stall).

Reduktion der branch delay penalty; "delayed branch"



- PC: Folgeadresse wird bei fallender Flanke übernommen

Zyklus 6

...
...
add
sub

- Addierer: Am Ende des Zyklus 2 gültig

Delayed Branches, verzögerte Sprünge

Beim gezeigten Beispiel wird der auf den Sprungbefehl folgende Befehl immer noch ausgeführt

```
beq  x12,  x8,  t    --springe zur Marke t, falls Reg[12]=Reg[8]
```

```
sub  ...           # Wird immer noch ausgeführt  
                        # ("It's not a bug, it's a feature")
```

```
...
```

```
t:  add  ...
```

Einen Platz für die Aufnahme eines solchen Befehls nennt man **delay slot**, die Sprünge **delayed branches**.

Manche Maschinen haben mehrere *delay slots*.

Delay slots sollten von Übersetzern mit nützlichen Befehlen gefüllt werden. Nur notfalls sollte es ein NOOP sein.

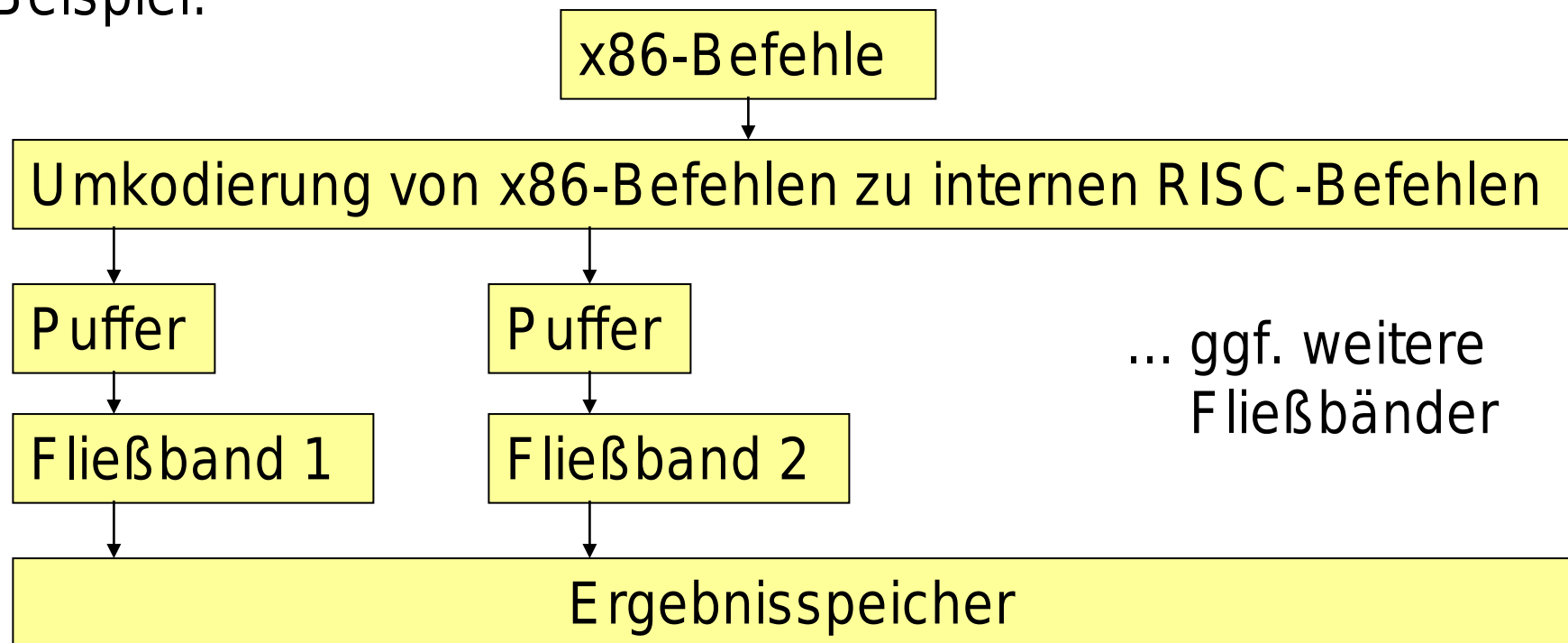
Die RISC-V-Maschine hat ein *delay slot*, welches aber vom Assembler verdeckt wird.

Typen von Fließband-Gefährdungen (hazards)

- **Strukturelle Abhängigkeiten/Gefährdungen (structural hazards)**
- **Datenfluß- Abhängigkeiten/Gefährdungen (data hazards)**
 - aufgrund von Datenabhängigkeiten (RAW)
→ forwarding, pipeline stalls
 - aufgrund von Antidatenabhängigkeiten (WAR)
(erst bei komplizierteren Systemen wichtig)
 - aufgrund von Ausgabeabhängigkeiten (WAW)
(erst bei komplizierteren Systemen wichtig)
- **Kontrollfluß-Abhängigkeiten/Gefährdungen (control hazards)**
 - delayed branches, pipeline stalls, spekulative Ausführung, Sprungvorhersage

Interne Struktur von Pentium-Prozessoren

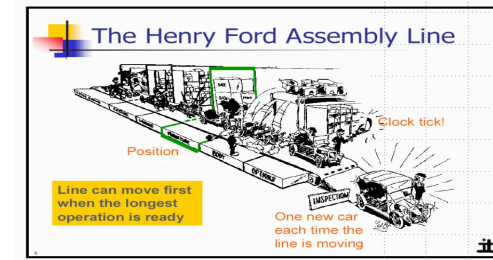
Fließbandverarbeitung bringt Performancegewinn.
Fließbandverarbeitung nur bei RISC-Befehlssätzen einigermaßen überschaubar.
Interne Umkodierung alter CISC-Befehle in RISC-Befehle.
Beispiel:



Zusammenfassung



- Die Fließbandverarbeitung (engl. pipelining) ermöglicht es, in jedem Takt die Bearbeitung eines Befehls abzuschließen, selbst wenn die Bearbeitung eines Befehls ≥ 1 Takte dauert.
- Bei mehreren Fließbändern ➡ pro Takt können mehrere Befehle beendet werden.
- 3 Typen von Gefährdungen des Fließbandbetriebs:
 - resource hazards
 - data hazards (RAW, WAR, WAW)
 - control hazards
- Gegenmaßnahmen:
 - pipeline stall
 - forwarding/bypassing, delayed branches
 - branch prediction, out-of-order execution, dynamic scheduling (➡ Rechnerarchitektur)
- Ggf. mehrere Fließbänder bei modernen Architekturen



www.it.lth.se/courses/dsi/material/Lectures/Lecture6.pdf

